



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INTELLIGENT SYSTEMS

ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

SYBIL-RESISTANT IDENTITY SYSTEMS IN DECENTRALIZED ENVIRONMENTS

SYSTÉMY IDENTIFIKACE ODOLNÉ PROTI SYBIL ÚČTUM
V DECENTRALIZOVANÝCH PROSTŘEDÍCH

MASTER'S THESIS

DIPLOMOVÁ PRÁCE

AUTHOR

AUTOR PRÁCE

Bc. MICHAL L'AŠ

SUPERVISOR

VEDOUCÍ PRÁCE

doc. Ing. IVAN HOMOLIAK, Ph.D.

BRNO 2026

Master's Thesis Assignment



173072

Institut: Department of Intelligent Systems (DITS)
Student: **Řař Michal, Bc.**
Programme: Information Technology and Artificial Intelligence
Specialization: Cybersecurity
Title: **Sybil-Resistant Identity Systems in Decentralized Environments**
Category: Security
Academic year: 2025/26

Assignment:

1. Get familiar with the principles of blockchains and smart contracts.
2. Study approaches related to avoiding Sybil entities and ensuring identity verification & uniqueness in decentralized and centralized environments.
3. Analyze the pros and cons of particular approaches and perform security analysis while assuming diverse attacker models.
4. Get familiar with zero-knowledge cryptography and zk-SNARKs in particular. Focus on the application in the context of identity verification and authentication.
5. Propose a novel approach for mitigating Sybil entities in decentralized environments or a significant improvement of an already existing one.
6. Implement the selected approach and evaluate its performance on a suitable case study.
7. Discuss the security of your implementation and further improvements.

Literature:

- BrightID. (n.d.). *BrightID*. <https://www.brightid.org/>
- Circles UBI. (n.d.). *Circles UBI – A basic income system for communities*. <https://joincircles.net/>
- Worldcoin Foundation. (n.d.). *World – The real human network*. <https://world.org/>
- ElSheikh, Muhammad, and Amr M. Youssef. "Dispute-free scalable open vote network using zk-SNARKs." *International Conference on Financial Cryptography and Data Security*. Cham: Springer International Publishing, 2022.
- Wood, Gavin. "Ethereum: A secure decentralised generalised transaction ledger." *Ethereum project yellow paper* 151.2014 (2014): 1-32.
- W3C, Decentralized Identifiers (DIDs), <https://www.w3.org/TR/did-1.0/>
- Homoliak, Ivan, et al. "The security reference architecture for blockchains: Toward a standardized model for studying vulnerabilities, threats, and defenses." *IEEE Communications Surveys & Tutorials* 23.1 (2020): 341-390.

Requirements for the semestral defence:
Points 1-4.

Detailed formal requirements can be found at <https://www.fit.vut.cz/study/theses/>

Supervisor: **Homoliak Ivan, doc. Ing., Ph.D.**
Head of Department: Kočí Radek, Ing., Ph.D.
Beginning of work: 1.11.2025
Submission deadline: 20.5.2026
Approval date: 3.11.2025

Abstract

Almost all identity management systems, whether centralized or decentralized, use a centralized approach to achieve Sybil resistance – protection against attacks that use a large number of fake identities. Typically, these systems require verification via a phone number or government-issued ID. This thesis aims to design and implement a decentralized identity management system that ensures Sybil resistance while preserving user anonymity and sensitive data. The proposed solution utilizes facial biometric verification and anti-Sybil social graph analysis. To ensure decentralization, anonymity, and protect privacy, it utilizes blockchain technology, zero-knowledge proofs, and trusted execution environments. The result is a robust system that ensures the uniqueness, anonymity, and privacy of registered users without relying on centralized authorities. This represents a significant step toward achieving self-sovereign identity management, which is crucial in today's world where personal data is a valuable commodity.

Abstrakt

Takmer všetky systémy pre správu identít, centralizované alebo decentralizované, používajú na zaistenie Sybil-rezistencie, ochrany pred útokmi, ktoré využívajú veľké množstvo falošných identít, centralizovaný prístup. Tieto systémy zvyčajne vyžadujú overenie prostredníctvom telefónneho čísla alebo dokladu totožnosti vydaného štátom. Cieľom tejto práce je navrhnúť a implementovať decentralizovaný systém pre správu identít, ktorý zabezpečí Sybil-rezistenciu, anonymitu používateľov a súkromie ich osobných údajov. Navrhované riešenie využíva overenie biometrie tváre a anti-Sybil analýzu sociálneho grafu. Pre zabezpečenie decentralizácie, anonymity a ochrany súkromia sa využívajú technológie blockchain, zero-knowledge proofs a trusted execution environments. Výsledkom je robustný systém, ktorý zabezpečuje jedinečnosť registrovaných užívateľov, ich anonymitu a súkromie bez spoliehania sa na centralizované authority. To predstavuje významný krok k dosiahnutiu samostatnej správy identít, ktorá je v dnešnom svete kľúčová, keďže osobné údaje sú cennou komoditou.

Keywords

digital identity, decentralized identity, Sybil resistance, identity management, self-sovereign identity, blockchain

Klíčové slová

digitálna identita, decentralizovaná identita, Sybil rezistencia, správa identít, autonómna identita, blockchain

Reference

EAŠ, Michal. *Sybil-Resistant Identity Systems in Decentralized Environments*. Brno, 2026. Master's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor doc. Ing. Ivan Homoliak, Ph.D.

Rozšířený abstrakt

Digitálne identity sú neoddeliteľnou súčasťou online prostredia. Takmer každý vlastní nejakú formu digitálnej identity, ktorá môže zahŕňať online profily, digitálne identifikátory alebo decentralizované identity. Tieto identity zvyčajne spravuje centralizovaná autorita, ktorá riadi registráciu a spravuje užívateľské údaje. Centralizovaná správa však predstavuje riziká pre súkromie týchto údajov. V reakcii na tieto riziká sa objavujú decentralizované modely pre správu identít, ktoré dávajú užívateľom plnú kontrolu nad osobnými dátami a správou svojej identity.

Mnohé služby, ktoré používajú systémy správy identít, však vyžadujú unikátnosť digitálnych identít, v zmysle, že každá fyzická osoba je v systéme zastúpená iba raz. Táto vlastnosť sa často označuje ako Sybil-rezistencia – ochrana pred útokmi, ktoré využívajú veľké množstvo falošných identít. Väčšina existujúcich systémov, či už centralizovaných alebo decentralizovaných, rieši tento problém centralizovane pomocou overovania dokladov totožnosti, pasov alebo telefónnych čísel. Unikátnosť je teda zaručená dôveryhodnou tretou stranou.

Cieľom tejto práce je navrhnúť a implementovať systém správy identít, ktorý by umožnil užívateľom nezávisle preukázať svoju unikátnosť. Výsledné riešenie pre tento účel kombinuje overenie biometrie tváre s anti-Sybil analýzou sociálneho grafu. Na dosiahnutie decentralizácie, zachovanie anonymity a ochranu súkromia, riešenie využíva technológiu blockchain, Zero-Knowledge Proofs (ZKP) a Trusted Execution Environments (TEE).

Podstata riešenia spočíva v kombinácii registrácie biometrie tváre s jedinečným protokolom pre vytváranie vzájomných spojení, ktoré následne tvoria sociálny graf. Tento protokol vyžaduje osobné stretnutie, kde si zúčastnené strany navzájom overia registrované identity, čím sa eliminuje potreba centralizovanej autority. Počas tohto stretnutia sa užívatelia navzájom vyfotografujú a využijú decentralizovanú sieť TEEs pre overenie, že sú obaja registrovaní a ich fotografia zodpovedá registrovanej identite. Druhá časť riešenia využíva spomenutú anti-Sybil analýzu sociálneho grafu. Z dôvodov ochrany súkromia sa samotný sociálny graf nikde neukladá. Namiesto toho sa využívajú rekurzívne ZKPs dokazujúce existenciu nadviazaných spojení. Užívatelia poznajú iba svoje vlastné spojenia, o ktorých dokážu vytvoriť ZK dôkazy o ich existencii. Tieto dôkazy potom ďalej zdieľajú so svojimi spojeniami, ktoré ich rekurzívne používajú vo vlastných dôkazoch. Takýmto zrefazovaním sa vytvárajú dôkazy o cestách v sociálnom grafe. Cesty v sociálnom grafe sú dôležité pre finálnu analýzu. Ak je užívateľ schopný dokázať požadovaný počet odlišných ciest určitej dĺžky, tak dostane verifikáciu od systému. Tento spôsob analýzy vychádza z toho, že Sybil entity (falošné identity) nie sú schopné nadviazať dostatočné množstvo spojení a nezískajú tak verifikáciu.

Implementácia tohto riešenia dokazuje jeho funkčnosť a praktickú použiteľnosť. Pozostáva zo simulácie analýzy sociálneho grafu v jazyku Python, ktorá demonštruje, ako táto analýza funguje, implementácie rekurzívnych ZKP pomocou systému Plonky2 v jazyku Rust a implementácie konceptu celého navrhnutého riešenia v jazyku Python a Solidity. Testovanie pozostáva z praktických ukážok rekurzívneho vytvárania cesty pomocou implementovaného ZK obvodu, príkladov registrácie identít a vytvárania vzájomných spojení.

Táto práca prináša robustné riešenie problému unikátnosti digitálnych identít s dôrazom na decentralizáciu, súkromie a anonymitu. Jedná sa o decentralizovanú alternatívu k existujúcim takmer výlučne centralizovaným riešeniam. Táto iniciatíva je ďalším krokom k decentralizovanej správe identít, ktorá je dôležitá najmä vo svete, kde sú osobné údaje cenným aktívom.

Sybil-Resistant Identity Systems in Decentralized Environments

Declaration

I hereby declare that this Master's thesis was prepared as an original work by the author under the supervision of doc. Ing. Ivan Homoliak Ph.D. I have listed all the literary sources, publications, and other sources that were used during the preparation of this thesis. I have used Grammarly to check for spelling and grammar errors and to enhance the academic tone of this paper. Additionally, I have used Gemini for research, using its Deep Research functionality, and to help explain complex academic papers. Finally, I have used GitHub Copilot and Claude Code to help with implementation, bug fixes, testing, and code documentation.

.....
Michal Laš
May 20, 2026

Acknowledgements

I would like to sincerely thank my supervisor doc. Ing. Ivan Homoliak Ph.D., for all the advice and insightful comments. Additionally, I am deeply thankful to my family, particularly my parents, for their unwavering support during my studies.

Contents

1	Introduction	4
2	Decentralized Infrastructure	5
2.1	Blockchains	5
2.2	Smart Contracts	12
2.3	Oracles	12
2.4	Wallet	13
2.5	Decentralized Identities	13
2.6	Verifiable Credentials	15
2.7	Decentralized Autonomous Organization	18
3	Zero-knowledge Proofs	19
3.1	zk-SNARKs	20
3.2	zk-STARKs	24
3.3	Applications	24
4	Identity Management Systems	26
4.1	Centralized IDMSs	27
4.2	Federated IDMSs	28
4.3	Decentralized IDMSs	29
5	Sybil-Resistant Identity Systems	31
5.1	Classification of Approaches	32
5.2	Primary Properties	33
5.3	Centralized SRISs – Taxonomy of Approaches	34
5.4	Decentralized SRIS – Taxonomy of Approaches	36
5.5	Existing Solutions	44
6	Design of a Sybil-Resistant Identity System	57
6.1	Assessment of Approaches and Existing Solutions	57
6.2	Design Overview	58
6.3	Registration	63
6.4	Connections Establishment	67
6.5	Social Graph Analysis	72
7	Security Analysis	82
7.1	Properties of Proposed System	82
7.2	Facial Biometric Fraud, Forgeries, and Deepfakes	84

7.3	Facial Biometrics Theft	85
7.4	Attacks on Social Graph Analysis	85
7.5	Censorship Attacks	85
8	Implementation and Testing	87
8.1	Social Graph Simulation	87
8.2	ZK Random Walks	92
8.3	ZK Proof of Paths	94
8.4	Registration and Connection Establishment	95
9	Discussion	99
9.1	Facial Scan False Positives	99
9.2	Liveness Detection in ROFL	99
9.3	Trust in the Social Graph	99
9.4	Social Network Groups and Bootstrapping	100
9.5	Persistence of Merkle Proofs	100
10	Conclusion	101
	Bibliography	102

List of Figures

2.1	Evolution from a traditional ledger to a blockchain [4].	5
2.2	Linked chain of blocks.	7
2.3	Example of Merkle hash tree.	8
2.4	Example of decentralized identifier [83].	14
2.5	Data model of verifiable credentials [82].	16
3.1	General steps of converting a high-level program to a zk-SNARK [51, 65]. .	22
4.1	Centralized identity management systems [38].	28
4.2	Federated identity management systems [38].	29
5.1	Virtual identity uniqueness vs. Sybil-resistance.	32
5.2	Visualization of Sybil attack trilemma. $A \cap B \cap C = \emptyset$ [68].	35
5.3	Hierarchical PKI vs. Web of Trust [20].	38
5.4	Graph-based Sybil detection assumption. Sybil region and honest region [2].	39
5.5	Composition of the World Network’s Orb [91].	45
5.6	Circles transaction using transitive trust relation [49].	47
5.7	Circles Sybil resistance example [49].	48
5.8	Example of Idena’s FLIP (Filter for Live Intelligent People) [41].	51
6.1	An example of creating a path through connected entities.	61
6.2	Actors of the proposed Sybil-resistant system and their interactions. . . .	63
6.3	Design of the profile registration process.	64
6.4	Connection establishment process between two entities.	68
6.5	Example of a Merkle Inclusion Proof [33].	71
6.6	Example of a Sparse Merkle Tree with a single inserted value.	72
6.7	Zero-knowledge random walks in the social network.	75
6.8	Illustration of the insertion and membership check operations of a Bloom filter [44].	77
7.1	An attack by a coordinated group on the proposed social graph analysis method.	86
8.1	Screenshot from the social network simulation visualizer.	91

Chapter 1

Introduction

Digital identities are a fundamental aspect of online life. Nearly everyone possesses some form of digital identity, which may include online profiles, digital IDs, or decentralized identities. Typically, these identities are managed by a centralized authority that controls registration and manages user data. However, centralized management poses risks to the privacy of this data. In response to these risks, self-sovereign identity management models are emerging, enabling individuals to retain complete control over their identities.

Nevertheless, many services that use identity management systems require unique digital identities to ensure that each physical person is represented only once within the system. This property is commonly referred to as Sybil-resistance – protection against the creation of numerous false identities. Most existing systems, whether centralized or decentralized, address this problem centrally by verifying government-issued IDs, passports, or phone numbers. In this way, uniqueness is guaranteed by a trusted third party.

The idea of this thesis is to design and implement an identity management system that allows users to prove their uniqueness independently. Such a system aims to be a decentralized alternative to existing, almost exclusively centralized solutions. This initiative is another step towards self-sovereign identity management, especially in a world where personal data is a valuable asset.

The proposed system combines facial biometric verification with anti-Sybil social graph analysis to ensure the uniqueness of registered users. To achieve decentralization, maintain anonymity, and protect privacy, the solution employs blockchain technology, zero-knowledge proofs, and trusted execution environments. Finally, a proof-of-concept implementation demonstrates the system’s practical feasibility.

Organization

Chapter 2 introduces the concepts of blockchains, smart contracts, decentralized identities, and verifiable credentials. **Chapter 3** explores zero-knowledge cryptography, while **Chapter 4** explains centralized, federated, and decentralized identity management systems. **Chapter 5** builds on the identity management topic by focusing on methods to achieve Sybil-resistance, particularly in a decentralized manner, and analyzes existing solutions. **Chapter 6** presents a detailed design of a novel system. Following this, **Chapter 7** discusses the properties and potential threats related to the proposed system. **Chapter 8** details the proof-of-concept implementation and testing. The thesis concludes with a discussion in **Chapter 9** and a final conclusion in **Chapter 10**.

Chapter 2

Decentralized Infrastructure

This chapter explains the key aspects of decentralized infrastructure needed to create a decentralized, Sybil-resistant identity system. Primarily, this involves blockchain technology, which is the cornerstone of decentralized systems, as well as other technologies, systems, and protocols such as smart contracts, oracles, and decentralized identities and credentials.

2.1 Blockchains

Blockchain is a unique technology that combines cryptography, networking, databases, and game theory into one secure, transparent, and decentralized system. At its essence, it is a digital, distributed, and decentralized ledger of transactions made by users, as illustrated in [Figure 2.1](#).

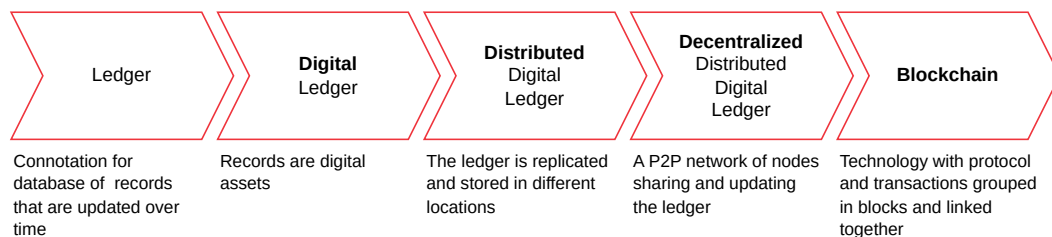


Figure 2.1: Evolution from a traditional ledger to a blockchain [4].

The key features of blockchains are [92, 72]:

- **Decentralization:** There are no centralized authorities managing the system; therefore, there is no single point of failure. All transactions and protocol changes are decided by users, who communicate via peer-to-peer connections. The information is not stored in a single node, and everyone in the network can access it.
- **Transparency:** All transactions are visible and can be verified by everyone in the network.
- **Immutability:** Once a transaction is verified and inserted into the distributed ledger, it cannot be changed or deleted.

- **Security:** Transactions are digitally signed to prevent falsification, and all ledger changes have to be verified by users. Any malicious changes to transactions or stored data can be detected and refused by other users.
- **Programmability:** Ability to execute smart contracts. This feature will be further discussed in [Section 2.2](#).

To give the term distributed ledger a more precise definition, it is just another way of referring to a distributed database, a data storage system in which data are logically located across multiple nodes [67]. In the case of blockchain, nodes are connected peer-to-peer and share data to achieve a synchronized, common state.

Staying with the concept of blockchain as a distributed database, there is one important theorem, called the CAP theorem, introduced by Eric Brewer [13] and later formalized by Gilbert and Lynch [37], which states that in distributed data systems, only one of the following three properties can be achieved [54]:

- **Consistency:** Every node has the same data at the same time.
- **Availability:** Every request receives a successful or unsuccessful response.
- **Partition Tolerance:** The system continues to operate even if part of the network fails to operate.

Blockchains usually aim for availability and partition tolerance, while consistency between nodes may be temporarily disrupted. This is important to keep in mind when designing new applications running on blockchains. The individual nodes do not have to have the same data at the same time. However, specific blockchain platforms, especially permissioned ones, can choose different combinations of properties.

The unique features of blockchain lay the foundation for different applications and usages. The original and most popular use of blockchains was introduced by Satoshi Nakamoto [59] in 2008, in which he proposed a decentralized payment system called Bitcoin. Since then, many other blockchain-based cryptocurrency systems have emerged.

However, the uses of blockchain technology are much broader. It has applications in voting processes, crowdfunding, supply chain management, healthcare, the internet of things, and the transportation industry [72]. From this work's perspective, an important application is decentralized identity management. With blockchains, users can individually manage their own identities and are not dependent on a centralized identity issuer. More about decentralized identities will be discussed in [Section 2.5](#).

2.1.1 Blockchain Classification

Blockchains can be classified into two categories based on access to the blockchain network into permissionless and permissioned.

Permissionless Blockchain

In a permissionless blockchain, anyone can join or leave the network at any time. The participants are equal and can perform the same actions. Everyone has access to the data stored on the blockchain and to the protocol's source code [72].

Permissioned Blockchain

In permissioned blockchains, there is usually a hierarchy among nodes regarding the actions they can perform. Access to the blockchain network itself may be restricted. This type of blockchain is typically used by private companies or organizations. There is a distinction between private blockchains (e.g., within a single organization), consortium/federated blockchains (managed by multiple organizations), and hybrid blockchains (a combination of private and public blockchain) [72].

2.1.2 Architecture of a Blockchain

For a decentralized ledger to function, it must have a **data model to represent the current state**, a **method of communication defined by transactions** that modify the ledger, and a **protocol for the network to reach consensus** on the ledger's final state [4].

Blocks

In blockchains, transactions are organized into blocks that are chained together through each block containing the hash digest of the previous block's header. This data model prevents any previous block from being modified without changing all subsequent blocks. In other words, if the hash of one block changes, the subsequent blocks will no longer chain to it. This model is illustrated in Figure 2.2.

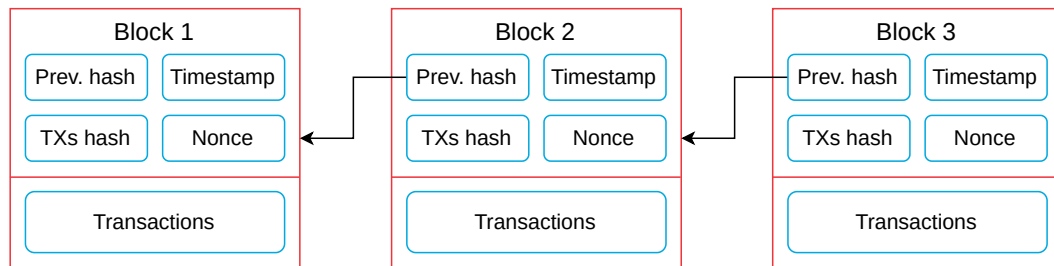


Figure 2.2: Linked chain of blocks.

The content of a blockchain block may vary across implementations, but in most cases consists of a header and a body. While the body contains actual transactions or other data specific to certain implementations, the block header contains metadata, which may be [92]:

- **The block number**, also known as a block height. It is the block's order number in the chain.
- **The header's hash of the previous block**. This value connects the blocks to each other.
- **A hash representation of the block data**. There are different methods for calculating this hash, but the most popular is by using a Merkle tree. This value makes it easy to verify the contents of the entire block. Changing a single transaction also changes the value of this hash, which in turn changes the block hash. In this way, a single change propagates throughout the entire chain, making it easy to detect.

- **A timestamp of creation.**
- **The size of the block.**
- **The nonce value.** This value is specific to blockchains using proof-of-work consensus. It is the result of a puzzle the miner had to solve to mine the block.

Merkle tree [56] is a data structure in the form of a binary tree, where values are iteratively hashed in pairs until a single final hash is created that represents all values. If the number of values is odd, the last value is duplicated and hashed with itself. Illustration of a Merkle tree is in [Figure 2.3](#).

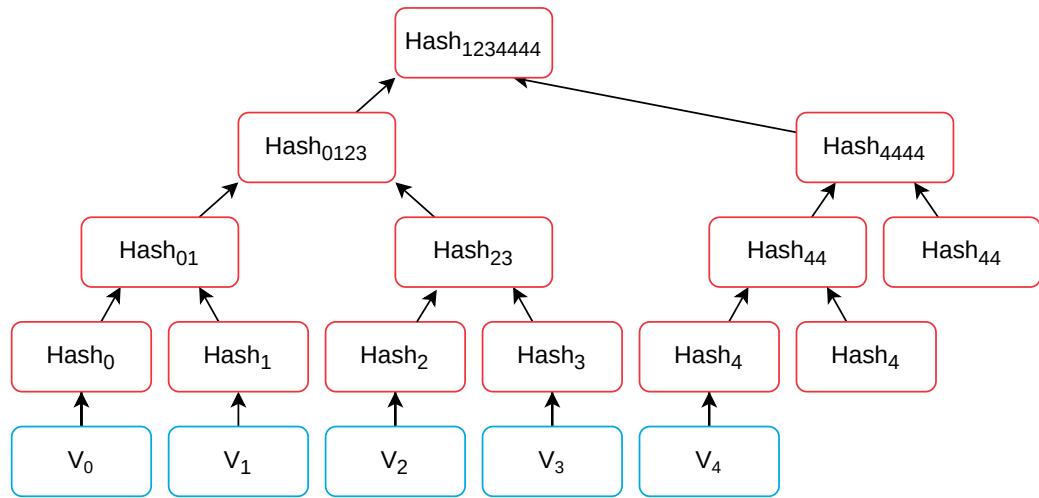


Figure 2.3: Example of Merkle hash tree.

Transactions

Transactions allow users to interact with and modify the ledger. Generally, they do not have to be strictly financial, but can also store, query, or share data. Because smart contracts can execute code, a transaction can be an executable program that performs various tasks.

To prevent fraud, each transaction must be digitally signed and verified. The entire process, from creating a transaction to confirming it and recording it in the blockchain, consists of the following steps [4]:

- **Creation:** The sender of the transaction creates a transaction according to a predefined data structure defined by the blockchain and signs it with its private key.
- **Propagation:** The transaction is propagated to the consensus/validating peers via peer-to-peer connections.
- **Validation:** This is the most crucial step. At this step, the transaction must meet various conditions defined by the protocol in order to be included in the block.
- **Propagation:** The block with the validated transaction is propagated through the network in order to let all nodes update their copy of the blockchain.

- **Confirmation:** After the consensus on the propagated block is reached, the transaction is executed and permanently written to the blockchain.

Node types

Blockchain is a complex mechanism. Therefore, individual nodes are usually organized hierarchically into three different types according to the actions they perform [40]:

- **Consensus nodes:** Active participants in the transaction validation and consensus protocol with a full copy of the blockchain. Their actions are most important for the proper functioning of the blockchain, as they can not only read but also write new transactions to the blockchain.
- **Validating nodes:** These nodes can read the blockchain, validate it, and disseminate transactions. They cannot write to the blockchain; however, they can detect malicious activity because they have a full copy of it.
- **Lightweight nodes:** These nodes rely on consensus and validation nodes. They do not have a complete copy of the blockchain. Therefore, they cannot read all data or validate all transactions, but they often have fragments of data in the form of block headers and can validate transactions that directly concern them.

2.1.3 Consensus Mechanism

Since all consensus nodes can create new blocks, there must be a strategy to ensure that they reach a common, consistent state. In decentralized systems, cooperation cannot be expected because nodes do not trust one another. Nevertheless, nodes can reach an agreement by applying principles from game theory. The two most commonly used concepts are lottery and voting, or eventually their combination. The principle behind both concepts is to select one node to publish the next block, while also guaranteeing fairness. Consensus mechanisms need to be Sybil-resistant, which means they must be able to withstand manipulation by fake identities. In other words, an entity cannot improve its chances of creating the next block by simply generating more fake identities. The concept of Sybil-resistance is crucial for this thesis and will be explored in detail in [Chapter 5](#).

In **lottery-based protocols**, consensus is reached by randomly choosing a leader that produces the next block. The random selection process is mostly non-interactive. There is minimal communication overhead between nodes, which improves scalability. On the other hand, a disadvantage is that multiple “winners” may emerge simultaneously, leading to inconsistency called *forks* [40]. This approach is used in consensus protocols such as Proof-of-Work (PoW) and Proof-of-Stake (PoS).

The **voting-based protocols** operate on the principle of elections, in which a leader is selected by votes from all participants. With this approach, there is a low probability of a fork, but scalability is problematic due to the need for extensive communication. Examples of such protocols include Byzantine Fault Tolerant (BFT) protocols, which require a majority quorum (usually $2/3$) to reach consensus [40].

Protocols combining lottery and elections work by selecting a subset of participants based on the lottery principle, who then vote among themselves. The second option is to divide the nodes into groups, each performing consensus independently, and then synchronize the groups using a separate mechanism.

Forks and Finality

Forks occur when multiple consensus nodes simultaneously publish a new block. All new blocks may be valid, but the network must decide which one will ultimately be included as the next block in the chain.

Blockchains solve this problem in two ways. The first is called the *longest chain rule*, and the second is called the *strongest/heaviest chain rule*. In the longest chain rule, the branch with the greatest length (number of blocks that follow each other) is considered correct. If a fork divides the network into two branches, it is assumed that one branch will naturally become longer over time, thereby resolving the conflict. The strongest chain rule considers not only the length but also the total “work done” in the branch. Even if a branch is shorter, the total number of blocks or “work done” in the branch may be greater (due to the occurrence of additional forks) than in the longest branch [54, 40].

Once conflicts are resolved, the block reaches a point called *finality*, from which all transactions in the block are considered permanent, irreversible, and unchangeable [54].

Types of Consensus Mechanisms

There are many different mechanisms for achieving consensus. Implementations vary according to the focus and needs of individual platforms. In permissionless blockchains, the most popular mechanisms are Proof-of-Work (PoW) and Proof-of-Stake (PoS). In permissioned blockchains, more centralized mechanisms such as Proof-of-Authority (PoA) are used. This work will mention another mechanism in [Section 5.5.4](#) called Proof-of-Personhood (PoP), which uses physical verification of a person’s presence to obtain a unique token that allows them to participate in the consensus.

Proof-of-Work (PoW) is a lottery-based consensus mechanism in which individual nodes compete in solving computationally intensive puzzles. The solution to the puzzle is a “proof” they have performed work. The puzzle is designed so that solving it is difficult, but verifying the result is easy. The popular method used by Bitcoin involves searching for a number called a nonce that, when hashed with the block hash, produces a hash that is smaller than the agreed target value. In practice, this means nodes try different combinations at random, which requires extensive hashing of different values. However, verification is very simple, as it is sufficient to calculate a single hash value [92].

Proof-of-Stake (PoS) is also a lottery-based consensus mechanism that is more computationally and energy-efficient than PoW. It is based on the idea that wealthy users who have staked their financial resources will have an incentive to behave honestly and not subvert the system. In practice, this works by the user staking a certain amount of cryptocurrency, based on which they receive a probability of being selected as the next leader. The disadvantage is that rewards for creating new blocks are concentrated among the wealthiest users who hold the highest stakes [92].

Proof-of-Authority (PoA) is a voting-based consensus mechanism in which a small number of selected users have the right to validate and create new blocks. The identities of the selected validators are mostly known, which motivates them to behave honestly. It is a more centralized scheme, but it allows consensus to be reached quickly and efficiently [54].

2.1.4 Privacy in Blockchains

Blockchain is often mistakenly attributed with anonymity. But in reality, it's pseudonymity, where individual users are represented by an address in hexadecimal format, usually derived from a public key. All transactions and data are public (in permissionless blockchains) and cannot be changed or deleted. Moreover, if someone discovers the real identity behind a user's address, they will also reveal all of their activities.

When working with sensitive data, privacy is an important concern. For example, in the case of identity management systems, it is certainly desirable that users' real identities and personal data cannot be traced. New innovative solutions to this problem are constantly emerging; some will be described below, and separately, zero-knowledge proofs will be described in [Chapter 3](#).

Mixing Services

Mixing services are solving the problem of linkability between senders and receivers of a transaction. They were originally designed by Chaum [22] in 1981 for sending untraceable electronic emails. A mixing service is an intermediary through which users can route their transactions, which are then forwarded to their actual recipients. Several such mixing intermediaries can be chained together.

Assume a transaction T that will be sent to recipient R via mixing service S . The transaction is first encrypted using public key K_R , address A is added, and the entire content is encrypted with public key K_S . This entire process is mathematically described in [Equation \(2.1\)](#) and extended to $i \in \{1, 2, \dots\}$ mixing services in [Equation \(2.2\)](#). The symbol $\rightarrow$ indicates the transformation of the mixing service input to its output (basically decrypting the part that is encrypted with the mixing service public key K_S). The parameters r_i are just random numbers (bit strings) that prevent replay attacks and guessing attacks.

$$K_S(r_0, K_R(r_1, M), A) \rightarrow K_R(r_1, M), A \quad (2.1)$$

$$\begin{aligned} K_{S_i}(r_i, K_{S_{i-1}}(r_{i-1}, \dots K_{S_2}(r_2, K_{S_1}(r_1, K_R(r_0, M), A)) \dots) \rightarrow \\ K_{S_{i-1}}(r_{i-1}, \dots K_{S_2}(r_2, K_{S_1}(r_1, K_R(r_0, M), A)) \dots) \rightarrow \end{aligned} \quad (2.2)$$

For such a service to work, it must be used by a sufficient number of users so that transactions are mixed and it is not possible to determine the connection between recipients and senders. In addition, if it is a centralized service, it is a single point of failure that can still track the relationship between senders and recipients. There are also decentralized mixing services that are achieving similar outcomes using multi-party computation (MPC) [31].

Multi Party Computation

Multi-party computation (MPC) is a cryptographic technique that allows two or more parties $P_i (i = 1, \dots, n)$ to perform a joint computation $f(x_1, x_2, \dots, x_n) = (y_1, y_2, \dots, y_n)$ without revealing their private inputs x_i and obtain their own corresponding output y_i . The MPC should guarantee the following security requirements [96]:

- **Privacy:** No one learns anything beyond the joint result.
- **Correctness:** The result obtained must be mathematically correct.

- **Independence of input:** A participant cannot cheat by choosing an input that affects the inputs of other participants.
- **Guarantee of output:** A corrupt participant cannot prevent an honest participant from obtaining their correct output.
- **Fairness:** A corrupt participant can only obtain an output if honest participants also obtain their outputs.

MPC is used, for example, in electronic voting to count voters' votes without revealing who voted for whom. In [Section 5.5](#), which discusses various Sybil-resistant identity systems, also lists several protocols that use this technique.

Ring Signatures and Linkable Ring Signatures

Ring signatures introduced by Rivest et al. [74] are a cryptographic technique that enables the creation of anonymous signatures. Any user can create such a signature non-interactively using a set of public keys of other users (potential signers) and their own pair of private and public keys. It is not possible to determine who signed the message from the resulting signature.

However, it is sometimes necessary to verify which users have already performed an action. For example, in electronic voting, ring signatures cannot be used to verify who has voted and who has not. Liu et al. [52] proposed a solution called linkable ring signatures, which preserves anonymity, i.e., it is not possible to determine who signed the message, but at the same time allows two signatures made by the same user to be linked.

2.2 Smart Contracts

The idea of a smart contract was introduced by Nick Szabo in 1994 [84] as a set of promises specified in digital form, with a specific protocol for performing them. Similar to contracts in physical form, for example, in business or politics, a contract can be represented as contractual clauses embedded in the hardware and software.

In blockchains, a smart contract is executable code and data that is part of a transaction, and individual nodes can execute this code and achieve a common result that is recorded on the blockchain. A smart contract can perform calculations, store information, or automatically transfer funds to other users. However, the condition is that the result must be deterministic. All nodes must obtain the same result by executing the same smart contract. For this reason, smart contracts cannot operate on external data (such as information from web services), but only on data provided directly via parameters or stored on the blockchain. For data from the outside world to be accessible on the blockchain, there is a special interface called an *Oracle* [92].

2.3 Oracles

Oracles [6] are third-party services that provide smart contracts with external information. They act as a bridge between blockchains and the outside world, because smart contracts themselves do not have access to data that is not stored directly on the blockchain. Oracles are not data sources but rather a layer that mediates, verifies, and authenticates external data sources. Oracles can be classified by data source into software, hardware, and human

oracles. Software oracles bridge data from external databases, such as web services. Hardware oracles bridge data from the real world, such as sensors, IoT devices, scanners, etc. Human oracles obtain information from people, such as experts with specific knowledge in a particular field.

2.4 Wallet

Users who want to use blockchain need software or hardware to securely store their private keys. Such software/hardware is called a wallet.

Its main task is to manage user keys and provide an interface for interacting with the blockchain, including creating and tracking transactions associated with stored keys. However, other functions are also increasingly included, such as the management of decentralized identity documents and credentials. Wallets can be categorized into two classes [40]:

- **Self-sovereign wallets** store private keys locally on a specific device that interacts directly with the blockchain. The advantage is that the private key remains in the user's hands, but the user must have the specific device with them to use the keys.
- **Hosted wallets** use a centralized party that provides an interface for interacting with the wallet and blockchain. The private key can be stored on the server side (server-side wallet) or in the user's browser (client-side wallet). If the key is on the server side, the user can use their key across multiple devices, but at the cost of relying on a third party.

2.5 Decentralized Identities

Decentralized identities (DIDs) represent the latest evolution in digital identity management. A more detailed explanation of digital identity management will be provided in [Chapter 4](#). The core concept of DIDs involves transitioning from centralized to decentralized models for managing digital identities. This shift is closely linked to the idea of **Self-Sovereign Identity (SSI)**, which empowers individuals to have complete control over their identity data.

DIDs [83], standardized by the World Wide Web Consortium (W3C), are a type of globally unique identifier referring to any subject, e.g., person, organization, thing, data mode, abstract entity, etc. In essence, DIDs are URIs (Uniform Resource Identifiers) that associate the entity identified by a DID with a *DID document* that describes the entity. Although third parties can assist in discovering DID-related documents or other operations, the entity that owns the DID maintains complete control and does not depend on any authority. Entities can create an unlimited number of DIDs. This allows them to maintain different DIDs tailored to specific contexts.

As illustrated in [Figure 2.4](#), a DID is a simple text string that consists of three parts:

1. The *did* URI scheme identifier.
2. The identifier for the DID method, which refers to a specification outlining the creation, resolution, updating, and deactivation of DIDs and their associated documents.
3. The DID method-specific identifier.

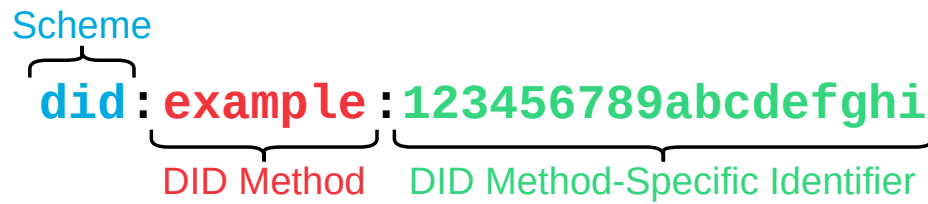


Figure 2.4: Example of decentralized identifier [83].

DIDs and DID documents are stored in a Verifiable Data Registry (VDR). While the W3C standard does not provide a specific definition for VDR, it serves as a foundational network technology for the public recording and retrieval of data. In practice, a blockchain is usually used as a VDR. Each DID can be resolved into a DID document through a process known as DID resolution. A DID document is a machine-readable JSON-LD document that contains information about the subject of the DID. This includes cryptographic public keys, service endpoints, authentication parameters, timestamps, and additional metadata [55, 83]. An example of a DID document is in Listing 2.1. It consists of the following parts:

- **@context:** Defines how to interpret the terms used in the DID document.
 - `https://www.w3.org/ns/did/v1` are just standard W3C rules for DIDs.
 - `https://w3id.org/security/suites/ed25519-2020/v1` are exact instructions on how to handle the cryptographic keys (in this case, the Ed25519)
- **id:** This is the DID of the entity.
- **The Authentication Section:** Defines how an entity proves the ownership of the ID.
 - `id` (Key ID) is a specific nickname for the key (`#keys-1`). An entity can have multiple keys for different purposes.
 - `type` defines the cryptography used.
 - `controller` specifies who has the power to change the key. In the example, the DID is its own controller (self-sovereign).
 - `publicKeyMultibase` is the actual public key.

Note that the DID document does not include any personal identity information. Personal data is stored in a digital wallet. When an entity wishes to share this data, they sign it using the private key linked to the public key in the DID document. This signature verifies their ownership of the DID.

Zooko's Triangle and DIDs

Zooko's Triangle [57] is a concept published by Bryce Wilcox-O'Hearn in 2001, assessing that it is not possible to have human-memorable, decentralized, and secure identifiers. Only two of these three properties can be achieved simultaneously.

```

1 {
2   "@context": [
3     "https://www.w3.org/ns/did/v1",
4     "https://w3id.org/security/suites/ed25519-2020/v1"
5   ]
6   "id": "did:example:123456789abcdefghi",
7   "authentication": [{
8
9     "id": "did:example:123456789abcdefghi#keys-1",
10    "type": "Ed25519VerificationKey2020",
11    "controller": "did:example:123456789abcdefghi",
12    "publicKeyMultibase": "zH3C2AVvLMv6gmMnam3uVAjZpfkcJCwDwnZn6z3wXmqPV"
13  }]
14 }

```

Listing 2.1: Example of a decentralized identifier document [83].

- **Human-memorable** means that identifiers are human-readable and easy to remember.
- **Decentralized** means that the allocation of identifiers is not controlled by a single central authority.
- **Secure** means that identifiers are globally unique and cannot be manipulated by an attacker.

DIDs are decentralized and secure; however, they are not fully human-memorable. Initiatives exist to map DIDs to human-memorizable identifiers, similar to the DNS system.

2.6 Verifiable Credentials

Verifiable credentials [82], standardized by the W3C, are a specific way to express a set of claims, such as driver's licenses, university degrees, educational certificates, or government-issued passports. The model for utilizing VCs to prove claims is illustrated in [Figure 2.5](#). In essence, it is also a model of a decentralized identity management system, which will be discussed in [Section 4.3](#). There are three main actors, and a VDR is the same as in the case of DID:

- A **holder** is an entity that controls and stores the VCs, typically using a digital wallet. The holder can present these VCs to prove their claims via Verifiable Presentation (VP). In most instances, the holder is subject to the conditions of the VCs they possess, although this is not always mandatory.
- An **issuer** is typically an authority figure, such as a corporation, non-profit organization, trade association, or government, but may also be an individual. This authority creates and signs a VC regarding one or more subjects and delivers it to a holder.
- A **verifier** receives VCs and verifies the stated claims. To accomplish this, the verifier must check several key aspects: the VC's expiration date, whether the VC has been revoked, the issuer's signature and DID, and confirmation that the individual

presenting the VC is the legitimate owner of the DID listed in the credential. This verification is conducted through a signing challenge, where the holder signs a random string using their private key, and the verifier uses the holder's public key from their DID to validate the signature.

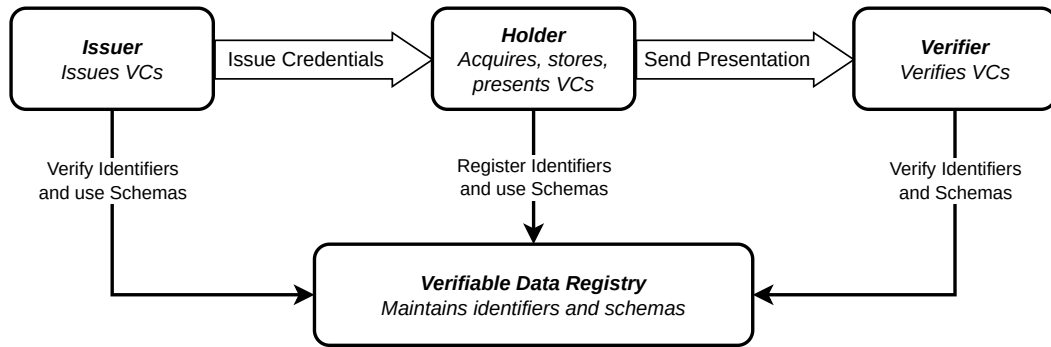


Figure 2.5: Data model of verifiable credentials [82].

Listing 2.2 shows an example of a VC for a bachelor's degree. As with DIDs, VCs can be represented in JSON-LD or JSON formats, and they can also be encoded as JSON Web Tokens (JWTs) [55]. The provided example of a VC consists of these parts:

- **@context:** As with DIDs, it defines how to interpret the terms used in the VC.
- **id:** A unique URI for this specific credential.
- **type:** Tells what kind of credential this is.
- **issuer:** DID of the entity that granted the degree. A verifier can look up this DID to find the public key and verify the signature.
- **validFrom:** The exact timestamp when this credential became active. Credentials can also have a **validUntil** field if they are meant to expire.
- **credentialSubject:** The actual claim being made. **id** is the DID of the Student (the holder) and **degree** is the object containing the specific data.
- **proof:** Proof of authenticity of the VC created by the issuer. It consists of the following attributes:
 - **type:** The used W3C standard for digital signatures.
 - **cryptosuite:** Tells what cryptographic algorithm is used.
 - **verificationMethod:** Link to the public key associated with the private key used to sign the VC.
 - **proofPurpose:** Means the issuer is using this key specifically to assert that the information inside is true.
 - **proofValue:** The actual digital signature for this VC produced by the issuer.

```

1  {
2    "@context": [
3      "https://www.w3.org/ns/credentials/v2",
4      "https://www.w3.org/ns/credentials/examples/v2"
5    ],
6    "id": "http://example.gov/credentials/3732",
7    "type": ["VerifiableCredential", "ExampleDegreeCredential"],
8    "issuer": "did:example:6fb1f712ebe12c27cc26eebfe11",
9    "validFrom": "2010-01-01T19:23:24Z",
10   "credentialSubject": {
11     "id": "did:example:ebfeb1f712ebc6f1c276e12ec21",
12     "degree": {
13       "type": "ExampleBachelorDegree",
14       "name": "Bachelor of Science and Arts"
15     }
16   },
17   "proof": {
18     "type": "DataIntegrityProof",
19     "cryptosuite": "eddsa-rdfc-2022",
20     "created": "2021-11-13T18:19:39Z",
21     "verificationMethod": "https://university.example/issuers/14#key-1",
22     "proofPurpose": "assertionMethod",
23     "proofValue": "z58DAdFfa9SkqZMVPxAQp...jQCrfFPP2oumHKtz"
24   }
25 }

```

Listing 2.2: Example of a verifiable credential [82].

Verifiable Presentation

Verifiable Presentation (VP) [82], also standardized by the W3C, specifies methods for the holder to sign and present VCs. As with VCs, VPs can be described using JSON-LD, JSON, or JWT. They can be utilized to consolidate information from various VCs. Furthermore, they should be short-lived and tied to a challenge provided by the verifier to prevent replay attacks, as outlined in the description of the verifier’s role [55]. The example of a VP is in Listing 2.3. It consists of the following parts:

- **@context:** As with DIDs and VCs, it defines how to interpret the terms used in the VP.
- **id:** This is a unique ID for the specific instance of sharing. In every presentation, a new, unique UUID is generated. This also helps prevent replay attacks and keeps the sharing history organized.
- **type:** Used to express the type of VP. The value `VerifiablePresentation` must always be presented with VP, but there may also be other additional types.
- **verifiableCredential:** The list of VC presented in this VP.

Selective Disclosure

Selective disclosure is one of the key privacy features of decentralized identities. Individuals selectively disclose only a subset of the information contained in their VCs. This allows

```

1 {
2   "@context": [
3     "https://www.w3.org/ns/credentials/v2",
4     "https://www.w3.org/ns/credentials/examples/v2"
5   ],
6   "id": "urn:uuid:3978344f-8596-4c3a-a978-8fcaba3903c5",
7   "type": ["VerifiablePresentation", "ExamplePresentation"],
8   "verifiableCredential": [{}]
```

Listing 2.3: Example of a verifiable presentation [82].

them to share only the necessary details with chosen entities, maintaining complete control over their identity data. Additionally, advanced cryptographic techniques, such as Zero-Knowledge Proofs (ZKPs), further enhance privacy.

2.7 Decentralized Autonomous Organization

A Decentralized Autonomous Organization (DAO) [16] is an entity that exists on the internet and operates autonomously according to a defined protocol. However, certain tasks may still require the involvement of individuals. The essence of a DAO is that it operates as a decentralized network of entities that communicate with one another, and some processes can be executed automatically.

If the organization lacked automation, it would be classified as a Decentralized Organization (DO). The crucial aspect of a DAO is its autonomy, which is typically ensured through the code defined in a smart contract.

For instance, consider a scenario of decentralized elections in which entities vote for candidates. The DAO's autonomy allows individual entities to cast their votes, with the smart contract automatically determining the winner. In the absence of automation, the entities would need to collaborate to agree on the winner, making it just a DO.

Chapter 3

Zero-knowledge Proofs

A zero-knowledge proof (ZKP) [8] is a cryptographic technique that allows one to prove a statement without revealing any information beyond whether it is true or not. Typically, there are two entities involved: the prover and the verifier. The prover attempts to convince the verifier that their statement is true. The main advantages of ZKPs are privacy protection and simplified verification of computations. For example, in the case of a complex computation, it may be easier to verify the correctness of the ZKP than to perform the computation itself. ZKP must satisfy three properties:

- **Zero knowledge:** A malicious verifier must not be able to obtain any information other than whether the statement is true or false.
- **Completeness:** An honest prover must be capable of proving a true statement.
- **Soundness:** A malicious prover cannot be capable of proving the truth of a false statement to the verifier with sufficient probability.

To give an intuition of how a ZKP works, consider the following example called *two balls and the colorblind friend* [21] introduced by Chalkias and Hearn in 2019. Imagine two friends, Alice and Bob. Alice has two identical balls, red and blue, that differ only in color. Bob is colorblind, and Alice is trying to prove to him that the balls are different colors and that she can tell them apart without revealing which ball is which color. Bob takes one ball in each hand, hides them behind his back, and can swap them around as he likes. Bob then shows Alice one ball and asks her whether he has swapped the balls. Alice can tell whether Bob has swapped the balls based on their color. However, if the balls are the same color, Alice still has a 50% chance of guessing whether Bob has switched the balls. Therefore, Bob repeats this experiment n times. In this case, $P(\text{the balls are different colors}) = 1 - 0.5^n$. When n is sufficiently high, Bob can be sure that Alice's statement that the balls are different colors is indeed true, even though 100% probability cannot be achieved.

This proof demonstrates that it is **zero-knowledge** because Bob learns nothing beyond the fact that the balls are different colors. **Completeness** is satisfied, as Alice successfully convinces Bob of her statement. Additionally, **soundness** is also fulfilled. If the balls were the same color, Alice would not be able to accurately guess n times in a row whether Bob had switched the balls. Consequently, she would not be able to convince Bob of her statement.

Note that the proof between Alice and Bob is interactive. However, in practice, it is desirable for ZKPs to be non-interactive. The two most commonly used ZKP protocols

are **zk-SNARKs** and **zk-STARKs**, which incorporates different properties. Both these protocols will be discussed in the following [Sections 3.1](#) and [3.2](#).

3.1 zk-SNARKs

SNARKs are the most commonly used ZKP protocol, where SNARK stands for **S**uccinct **N**on-Interactive **A**rgument of **K**nowledge. *Succinct* means that verifying a proof is much more efficient than reexecuting the computation, and *non-interactive* means that a prover can provide a single proving object that can be verified by everyone without interaction with the prover [8].

The zk-SNARK generation and verification can be described at the high level using the following algorithms [51, 78]:

- $Setup(pp) \rightarrow (pk, vk)$: In many schemes it is also called a *trusted setup*. This algorithm takes public parameters (pp) and generates a **proving key** (pk) and a **verification key** (vk). pp are the initial security parameters used to seed the setup.
- $Prove(pk, x, w, C) \rightarrow \pi$: The prover uses the proving key, a **publicly known statement or input** (x), a secret data only the prover knows called **witness** (w), and a **circuit** C . The circuit is a program code translated into a mathematical format that the ZKP system can understand. More theoretical papers often use the abstract relation R instead of the circuit C , but they have the same meaning. Together, these elements allow the prover to generate a proof π .
- $Verify(vk, x, \pi) \rightarrow \{0, 1\}$: The verifier uses the verification key, the public input, and the proof to output 1 if the proof is valid or 0 otherwise.

Trusted setup is a ceremony in which several parties contribute random elements that are combined to define parameters for ZKP. If at least one party destroys the random elements they contributed, then creating proof using such parameters is safe. These random elements are called *toxic waste* because if someone obtained all of them, they could create false proof [8]. In many traditional zk-SNARKs, such as Groth16, a new trusted setup is required every time the circuit changes. However, modern systems such as PLONK use a universal, updatable trusted setup [78].

3.1.1 Construction of a zk-SNARK

The first step is to create a program in a specific high-level language that serves as a blueprint for how public inputs and private witnesses must interact to be considered valid. It can be understood as a mathematical formula $f(x, w) = y$ where the program defines the function f , and the proof demonstrates that the prover knows the correct values to make the equation true without revealing the secret parts. This program is then compiled into a **arithmetic circuit** [51]. The arithmetic circuit is composed of mathematical operations, specifically addition and multiplication, which are represented by nodes known as gates. These individual gates are interconnected by edges that carry values [65]. An example of an arithmetic circuit is illustrated in [Figure 3.1](#), along with additional steps in the zk-SNARK construction.

Once the circuit is compiled, it must be represented as a mathematical statement that can be proven. This representation is called a constraint system. The most commonly

used systems are Rank-1 Constraint Systems (R1CS) and Plonkish circuits. Afterward, the constraint system is converted into a set of polynomials, such as a Quadratic Arithmetic Program (QAP), or into a single polynomial [51].

The core of the zk-SNARK is the proving system, which uses cryptographic techniques to prove the correctness of the constraint system. The utilized techniques determine the properties of the final zk-SNARK, which is why there are various systems that might be suitable for different applications. There are multiple techniques, but the most widely used are **Polynomial Commitment Schemes (PCS)** and cryptographic assumptions, such as **bilinear pairings (Groth16)**. A bilinear pairing is a cryptographic mapping function that takes elements from two different groups and maps them to a target group. It can be used directly with a QAP or as part of a PCS. In the context of PCS, a prover commits to a polynomial and later demonstrates that it was accurately evaluated at a specific point [51, 78].

Depending on the methods used, the proof can still be interactive. The final step is to convert it into a non-interactive format. This is typically achieved using the Fiat-Shamir transformation, in which the prover generates random-looking messages (hashes) that simulate the verifier’s challenges, thereby eliminating the need for back-and-forth communication [51]. All steps of generating the zk-SNARK are illustrated in [Figure 3.1](#).

3.1.2 Recursive zk-SNARKs

Recursive zk-SNARKs enable the verification of multiple zk-SNARKs within a single zk-SNARK. Each zk-SNARK verification can be viewed as a computation represented by a circuit that can be used within another zk-SNARK. This feature allows for the combination of multiple proofs into a single proof. Although the prover must invest additional effort to generate such a proof, the verifier benefits from having to verify only one zk-SNARK [78].

Systems like Nova [46] utilizes the efficient concept of Incrementally Verifiable Computation (IVC). This cryptographic primitive enables a prover to demonstrate the correct execution of a computation. Essentially, it allows for the creation of proofs for other proofs. At each step, a new proof is generated that validates the current computation step and the validity of the previous proof.

3.1.3 Existing Proving Systems

There are many different zk-SNARK systems, each with its own advantages, disadvantages, and characteristics that can be utilized for various applications. From the developer’s perspective, it is crucial to select the right system that aligns with the specific application needs. However, this requires expert knowledge of the underlying construction of zk-SNARKs. The description of five different zk-SNARK systems below should serve just as an overview. The brief summary of the systems described below is provided in [Table 3.1](#).

Groth16

Groth16 [39] utilize the QAP to represent arithmetic constraints into polynomial equations. It offers the smallest proof size and the fastest verification available, making it ideal for recursive proof composition. However, a unique trusted setup is necessary whenever the circuit changes, and the prover’s computational burden becomes significant for large statements. Since it relies on the hardness of the discrete logarithm, it is also not post-quantum secure.

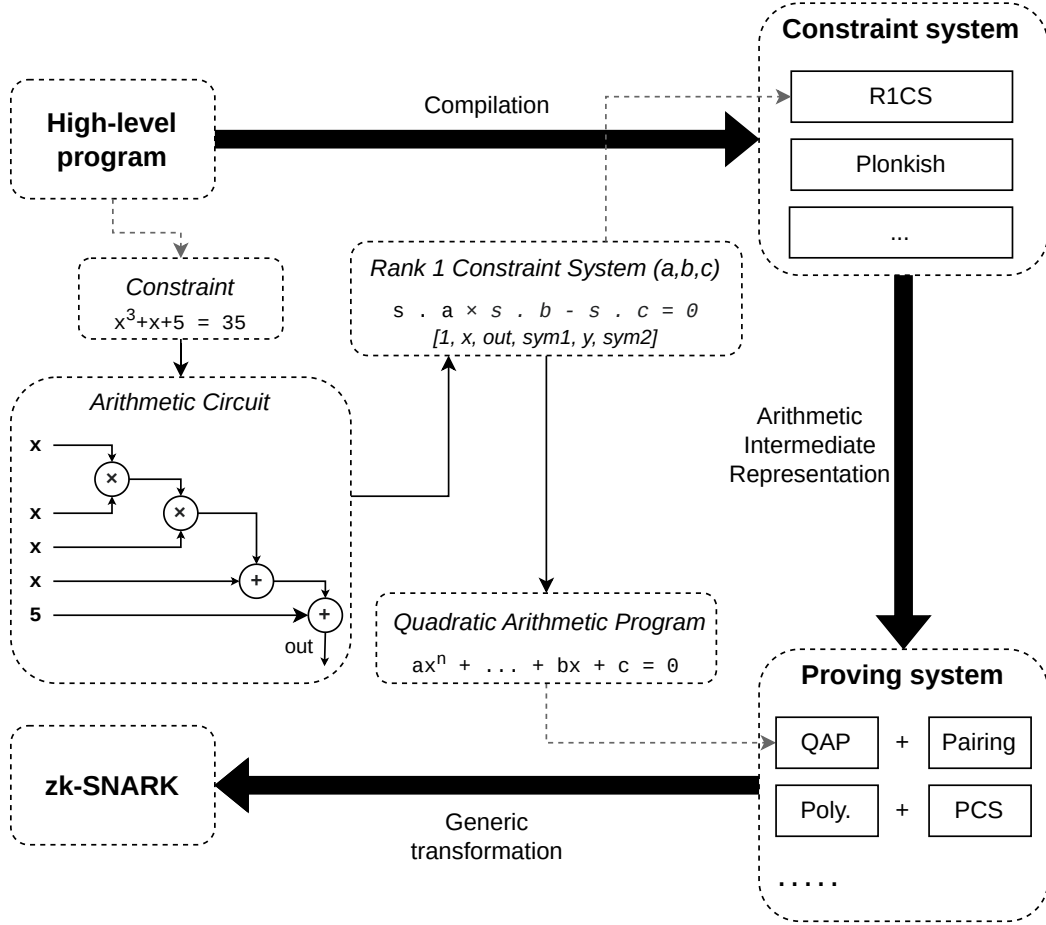


Figure 3.1: General steps of converting a high-level program to a zk-SNARK [51, 65].

PLONK

PLONK (Permutations over Lagrange bases for Oecumenical Noninteractive arguments of Knowledge) [36] is a universal and updatable zk-SNARK. Unlike Groth16, PLONK allows for a single large trusted setup that can be used for any circuit up to a certain size, and this setup can be updated by any party to maintain security. The size of PLONK remains constant regardless of circuit complexity, and it offers a fast prover running time. However, the computational burden on the prover is still 1.1 to 3 times greater than that of non-universal schemes like Groth16. Additionally, similar to Groth16, PLONK is not resistant to quantum attacks.

Plonky2

Plonky2 [70] is a framework designed for very fast recursive compositions. It achieves this by arithmetizing TurboPLONK and replacing the typical polynomial testing scheme found in PLONK with the Fast Reed-Solomon Interactive Oracle Proofs of Proximity (FRI) protocol. Plonky2 relies heavily on symmetric cryptography and hashing, utilizing the

Protocol	Proof Size	Prover Eff.	Verifier Eff.	PQ	Setup	Main Advantage
Groth16	Constant (~128–192 B)	$\mathcal{O}(n \log n)$	$\mathcal{O}(1)$	No	Per-circuit	Smallest proof and fast verification.
PLONK	Constant (~0.5-1 KB)	$\mathcal{O}(n \log n)$	$\mathcal{O}(1)$	No	Universal, Updatable	Flexible custom gates and single universal setup.
Plonky2	$\mathcal{O}(\log^2 n)$ (~43 KB)	$\mathcal{O}(n \log n)$	$\mathcal{O}(\log^2 n)$	Yes	Transparent (No Setup)	Fast proving time and recursion via FRI.
Bulletproofs	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	No	Transparent (No Setup)	Efficient range proof aggregation.
Halo & Halo 2	$\mathcal{O}(\log n)$ (~3.5-4 KB)	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	No	Transparent (No Setup)	Practical recursive proof composition.
Nova	$\mathcal{O}(1)$ folding; $\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	No	Transparent (No Setup)	Fast recursive proofs with minimal overhead via folding.

Table 3.1: Summary of the characteristics of different zk-SNARK systems. The variable n corresponds to the number of gates/constraints/rows, depending on the scheme.

Poseidon hash function within a sponge construction. This recursive approach allows it to condense arbitrarily large proofs into a constant size. Additionally, Plonky2 significantly optimizes the hashing and verification processes for Merkle proofs. Since this framework is not based on elliptic curves but instead on the FRI protocol and hash-based cryptography, it is considered to be post-quantum secure or at least highly resistant to quantum attacks.

Bulletproofs

Bulletproofs [18] are zk-SNARKs that do not require a trusted setup. They rely solely on the discrete logarithm assumption, which allows for full transparency without the risk of toxic waste. However, because they rely on elliptic-curve discrete logarithms, they are not secure against quantum attacks. The size of the proof scales logarithmically with the size of the witness. A unique advantage is their aggregation capability, which allows multiple range proofs (ZKP, which proves that the hidden value is within some range) to be merged into a single proof with only a small additive increase in size. However, their verification time scales linearly with circuit size, making them notably slower than other systems such as Groth16.

Halo & Halo 2

Halo [11] and Halo 2 [95] are zk-SNARKs that eliminate the need for trusted setups by utilizing discrete-log assumptions over cycles of elliptic curves. They also support effective recursive proof verification and a compact proof size of 3.5 to 4 KiB that remains constant regardless of the depth of recursion. The primary difference is the shift from R1CS arithmetization in Halo to the more flexible PLONKish model in Halo 2, which supports custom gates and efficient lookup tables for complex logic. However, neither system is

post-quantum secure, and verification is not fully succinct, as the final step requires linear work proportional to the circuit size.

Nova

Nova [46] is a system for Incrementally Verifiable Computation (IVC) that introduces **folding schemes** to replace traditional recursive zk-SNARKs. Instead of verifying a full proof at each step, Nova allows for the folding of two claim instances into one, deferring the work of creating and verifying the final proof. It avoids using zk-SNARKs in its core recursive process, but the resulting IVC proof is linear in size, so to make it succinct and zero-knowledge, a zk-SNARK (like a variant of Spartan) is used as a final step to compress the accumulated proof. Nova provides fast proving with a small, constant-sized circuit within the recursion. The final compressed proof typically measures around 8 to 9 KB. The verifier performs constant work during the recursion, but the total verification time is logarithmic or linear, depending on the circuit size and the commitment scheme used. Furthermore, no trusted setup is required, but as it relies on the hardness of the discrete logarithm problem, it is not post-quantum secure.

3.2 zk-STARKs

zk-STARKs [5] are a framework for creating zero-knowledge proofs that ensure computational integrity without requiring a trusted setup. STARK stands for **Scalable Transparent AR**gument of **K**nowledge. All the zk-SNARKs mentioned earlier utilize cryptography based on elliptic curves and rely on the hardness of the discrete logarithm problem. In contrast, zk-STARKs use collision-resistant hash functions, which makes them secure against potential quantum attacks. However, zk-STARKs generate proofs that are significantly larger in size, and the process of generating these proofs is also slower. Despite these drawbacks, zk-STARKs scale better and can eventually outperform other systems. Additionally, verifying zk-STARK proofs is very fast [65, 8, 58].

3.3 Applications

ZKPs have gained popularity mainly in blockchain decentralized applications. Their main advantage is ensuring privacy and improving computational efficiency.

While zero-knowledge proofs (ZKPs) are quite versatile and useful in many applications, they may not be the best option for every scenario. In some cases, it might be more advantageous to employ other cryptographic tools, such as MPCs or homomorphic encryption. The key consideration is the need for a source of trust, which standalone ZKPs are incapable of establishing. ZKPs can be used to ensure the confidentiality of input data and the cryptographic verifiability of computations, but cannot be used to establish trust between parties [28].

Blockchain and Decentralized Applications

- **Privacy-Preserving Transactions:** The sender of the transaction can use ZKP to prove that they are authorized to make the payment without having to reveal their identity, the recipient's address, or the amount [54].

- **Scalability Enhancements:** To increase the scalability of blockchains in terms of transactions, they use so-called zk-rollups. Transactions are batched outside the main chain, and a single ZKP is generated to prove their validity [54, 8].
- **Privacy-Preserving Consensus:** In consensus mechanisms where the next validator is known in advance, it is necessary to ensure their anonymity, as they may become the target of a denial of service (DoS) attack. ZKP can be used to preserve the anonymity of the next validator until a new block is created [8].
- **Blockchain Storage:** Storing data on a blockchain can be costly. Using ZKPs, it is possible to verify the existence and correctness of data stored off the blockchain while only placing the proof and a reference to the data on the chain [50].

Identity Management Systems

- **Selective Disclosure:** Users can prove ownership of specific attributes without revealing sensitive personal information about themselves. For example, a user can prove they are over 18 years old without revealing their date of birth. However, the user must have a VC (or equivalent) for the attribute they are trying to prove [54].
- **Identity & Authentication (zk-Login):** zk-Login [3] is a privacy-preserving authentication mechanism that allows users to access blockchain wallets using their existing OpenID Connect (OIDC) credentials, such as those from Google, Facebook, or Apple. The system leverages ZKPs to authenticate transactions without revealing users' off-chain identities.

Machine Learning

- **Model Verification:** A service provider can use ZKP to prove that a specific output was produced by running a particular model on a given input. This can be useful for detecting deepfakes and verifying content sources [50].
- **Privacy-Preserving Training:** It is theoretically possible to train large language models on a community-wide basis. An individual could utilize their own data, which they prefer not to publish, to privately train a model. After training, they could publish the resulting model and create a ZKP to demonstrate that the model was trained correctly [50].

Economic and Social Use Cases

- **Voting:** A person can prove they have cast a valid vote without revealing the specific content of that vote [8].
- **Financial Solvency:** Companies can use ZKPs to prove their solvency to regulators or the public without revealing their entire balance sheet [8].
- **Image Authentication:** It is possible to create a ZKP about the authenticity of digital images after permissible edits, such as cropping, while ensuring they have not been tampered with in any other way [50].

Chapter 4

Identity Management Systems

Digitization has enabled the transfer of many assets and resources into the online space. To allow individuals to securely manage these resources, digital identities have been developed to reflect a person's real-world identity. This chapter discusses approaches and systems for managing digital identities.

Understanding Identity

The concept of identity involves distinguishing individuals based on their attributes or affiliations. For example, name, origin, nationality, ethnicity, education, certificates, wealth, connections, achievements, and so on. For identity to be utilized in the digital world, these attributes and affiliations must be digitized and verified. Ideally, identity in the digital form should be something that cannot be stolen, lost, or misused, and the only way to lose one's identity is by death [34]. However, this is a rather utopian idea. In reality, such a thing probably does not exist, and even if it did, it would be very difficult to transfer it into digital form. People usually create their digital identity based on attributes of their real identity, such as name, date of birth, hobbies, etc. For the sake of consistency in terminology in this thesis, two types of identities will be distinguished: **personal identity** and **virtual identity**.

- **Personal identity** refers to a physical person in the real world with unique biological attributes (biometrics like fingerprints or iris patterns) and a lifelong social history that, ideally, cannot be lost, stolen, or shared.
- **Virtual identity** refers to a person's digital representation in a system (e.g., a user account). A single personal identity can technically be represented by multiple virtual identities.

Overview of IDMSs

Identity Management Systems (IDMSs) manage identity proofing, registration, verification, and issuance. They control access to online applications and resources, ensuring the protection of confidential and personal information from unauthorized access. Processes in IDMS typically involve three actors [32]:

- **Users:** They are clients of both the SP and the IdP. To access the service, users must have valid identities.

- **Service Providers (SPs):** They provide various services to users, such as online shopping platforms, social media access, and healthcare record management systems. SPs rely on IdPs for user verification.
- **Identity Providers (IdPs):** They serve as a reliable entity responsible for registering user identities, verifying their authenticity, and storing identity data. Furthermore, the IdP manages authentication requests from SP.

Classification of IDMSs

IDMSs can be classified into three categories based on the position of IdP to **centralized**, **federated**, and **decentralized**. Historically, individual categories have developed gradually. In the early days of digitization and the internet, there were almost exclusively centralized systems, which were very simple. Authentication relied solely on a username and a password. However, as the number of SPs on the internet increased, this approach became ineffective. Users found themselves needing different credentials for each service they accessed [38, 32].

This situation led to the emergence of federated IDMSs, which allow users to access multiple SPs through a single IdP. Such IdPs include Google, Microsoft, and Meta, which centrally manage virtual identities or use third-party providers such as ID.me, Okta, or Auth0 [38, 32].

With the advent of blockchain technology, decentralized IDMSs emerged as a third category. The main issue with federated IDMSs is that they aggregate a large amount of users' private information, posing potential privacy risks. In contrast, decentralized IDMSs emphasize user privacy. There is no centralized authority managing virtual identities; instead, users control their own identities, deciding what data to share and with whom [38, 32].

4.1 Centralized IDMSs

In centralized IDMSs, one centralized authority manages all identity-related tasks. Furthermore, the roles of SP and IdP are usually unified, meaning that each service provides both service access and identity verification [32]. An illustration of centralized IDMSs is shown in [Figure 4.1](#).

Centralized IDMSs are still relatively common. It is primarily used in environments where maintaining control over user identities is crucial for the effective operation of services [38]. Typical examples include major companies like Google, Microsoft, and Meta, which handle large volumes of virtual identities and user data. Smaller service providers also adopt this approach, often in conjunction with a federated model. However, this class of IDMSs also has disadvantages [32]:

- **Single point of failure:** Due to centralization on a central server or database where identities are stored, system failures or issues can cause the entire service to become inoperable.
- **Data breaches:** Identities and the sensitive user data associated with them are often stored in a centralized database, which is an attractive target for cyberattacks. If such a database is compromised, the sensitive information of many users can be stolen or exposed.

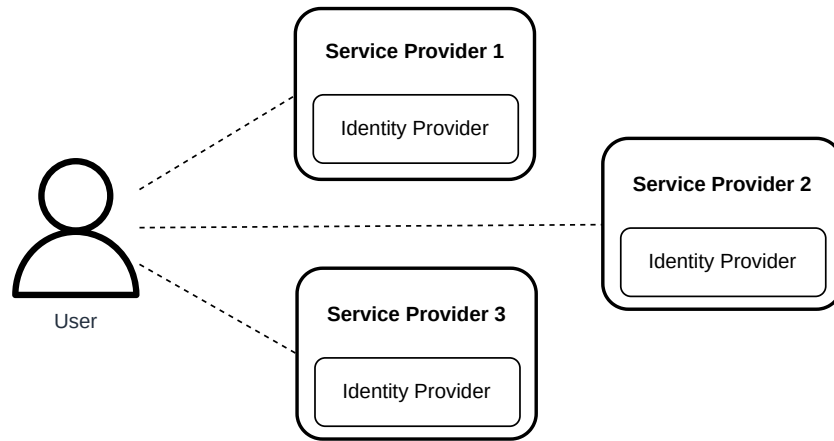


Figure 4.1: Centralized identity management systems [38].

- **Lack of interoperability:** Individual services are isolated and independent from each other, which makes it difficult to exchange identity data across different services. Users must maintain separate accounts and credentials for each service.

4.2 Federated IDMSs

In federated IDMSs, an identity registered with one IdP can be used across all organizations (SPs) within the federation. While this system remains centralized, individual organizations can authenticate identities against various IdPs. This model enables interoperability via a trust broker that manages trust relationships between organizations. Each organization's IdP registers with the trust broker, which ensures that the organizations trust each other's authentication tokens. Notable protocols used in federated IDMSs include SAML (Security Assertion Markup Language), OpenID, and OAuth. For example, a Google account can be used to log into multiple services [38, 32]. An illustration of federated IDMSs is shown in Figure 4.2.

With federated IDMSs, users can access multiple services with the same credentials, making virtual identities less fragmented and easier to use. However, problems with single points of failure and data breaches persist, as with centralized IDMSs, because identities are stored in a centralized database. In addition, there are other challenges [38, 32]:

- **Inconsistent updates across systems:** When a user accesses multiple SPs through a single IdP, the IdP must share user data with all SPs. Changes to the virtual identity need to be updated across all SPs to prevent inconsistencies, and the same applies when revoking a virtual identity.
- **Scalability:** As the number of users grows, so does the volume of data and authentication requests.
- **Lack of Flexibility:** Due to the need to maintain interoperability, it can be problematic to make major changes or quick upgrades.

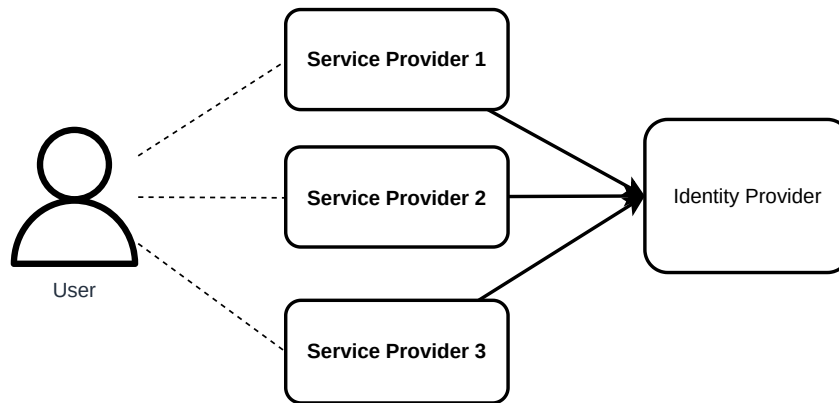


Figure 4.2: Federated identity management systems [38].

4.3 Decentralized IDMSs

The model of decentralized IDMSs is fundamentally different from centralized and federated models. It emphasizes Self-Sovereign Identity (SSI) and incorporates technologies like blockchain. The decentralized approach is closely linked to decentralized identifiers (DIDs) and verifiable credentials (VCs), as described in Sections 2.5 and 2.6. Essentially, the VC’s usage model, illustrated in Figure 2.5, represents a decentralized IDMS model.

Although the actors and their roles have changed in the VCs’ usage model compared to traditional IDMSs, there is still a clear mapping between them. Issuers are trusted authorities responsible for creating and revoking VCs. Verifiers are service providers that verify user information through their VCs. Users, as holders, have full control over their identities and VCs, allowing them to decide with whom they wish to share their personal information. While identifiers are publicly stored, typically on a blockchain, each user keeps their identity data in a digital wallet.

Decentralized IDMSs address many problems with centralized and federated IDMSs. There is no single point of failure, as the system operates across multiple nodes instead of relying on a central server that could malfunction. Additionally, blockchain technology ensures the security, immutability, and transparency of DIDs. Major data breaches are significantly less likely because identity data is stored on the user’s side rather than in a central location. The standardization of DIDs and VCs also enhances interoperability between different systems. However, there are also considerable disadvantages [38]:

- **Dependence on authorities:** While the main idea of decentralized IDMSs was to eliminate centralized control, there is still a need for authorities issuing VCs that, in the majority of cases, remain centralized.
- **Compatibility with centralized systems:** This issue arises primarily because of the different technologies utilized by each system.
- **Loss of private key:** Users are responsible for managing their own identities, which includes ensuring the security of their data and, specifically, their private keys – these are crucial for maintaining security. Losing a private key is comparable to losing one’s identity, as there is no way to recover it.

4.3.1 Existing decentralized IDMSs

Although centralized and federated IDMSs are well known, decentralized IDMSs have not gained much popularity to date. Therefore, here are some examples of such systems [38, 32]:

- **Hyperledger Indy** is a blockchain specifically built for decentralized identity. It provides the technical tools to create and manage digital identities with privacy-preserving features such as ZKP. The goal of the Hyperledger Indy is to promote trust, security, and compliance with rules such as GDPR.
- **Sovrin** is a global, public identity network that is actually built on top of the Hyperledger Indy framework. It is managed by a non-profit foundation and relies on a group of trusted organizations, called stewards.
- **Blockstack** is a system built on the Bitcoin blockchain. Instead of using traditional web addresses (like *.com*), it uses the Blockchain Naming System (BNS) to link human-readable names to public keys. While the proof of identity is recorded on the Bitcoin network, actual files and personal data are stored separately in a system called Gaia.
- **uPort** is an identity system based on the Ethereum blockchain. It works by creating smart contracts on Ethereum that serve as a unique, permanent digital ID. User identity data is stored on Inter Planetary File System (IPFS), while the hash of this data is recorded on the Ethereum blockchain. The private keys are stored directly on the user's smartphone, and the uPort app serves as a link between the user and the contracts. A unique feature of uPort is its recovery mechanism: if a user loses their phone or keys, they can regain control of their identity through a Recovery Quorum made up of trusted friends or delegates.

Chapter 5

Sybil-Resistant Identity Systems

Centralized, federated, and decentralized Identity Management Systems (IDMSs) do not necessarily address the issue of fake identities. Users can often create multiple virtual identities. However, certain systems require that a virtual identity corresponds to a real personal identity or that we can detect and link virtual identities belonging to a single personal identity. Sybil-Resistant Identity Systems (SRISs), which are a subset of IDMSs, address these challenges.

The term **Sybil** comes from the article *The Sybil Attack* by John R. Douceur [26]. A Sybil attack is a type of attack in which an attacker forges multiple fake identities to gain an unfair advantage. In this case, Sybil is a forged account, and a Sybil-resistant system is one that resists such attacks.

To clarify terminology, the following definitions of Sybil-resistance and identity uniqueness are provided; their relationship is also illustrated in [Figure 5.1](#):

- **Sybil-resistance** refers to the ability of a system to prevent fake virtual identities from gaining a quantitative advantage in the system. An entity can have multiple virtual identities, but they must be associated with a single personal identity that can stay private from the public. In other words, there exists a surjection from the set of virtual identities to the set of personal identities. In this case, multiple virtual identities for the same individual will be referred to as **pseudonyms**. Allowing pseudonyms enhances privacy and security. Users can present themselves through different virtual identities in various contexts, while the system still recognizes that they are the same individual.
- **Uniqueness** is a stricter requirement, ensuring that each virtual identity corresponds to exactly one personal identity. There exists a bijection between the set of virtual identities and the set of personal identities. The uniqueness property binds the entity session always to the same identifier. On the other hand, Sybil-resistance can map the entity session to a different identifier each time, which is linked to a single personal identity.

While it is likely impossible for any system to guarantee 100% uniqueness or Sybil-resistance in a virtual environment, such systems are often essential in today's digital society. They are used whenever there is a need to verify identity, such as in electronic voting, online auctions, social networks, airdrops, or Universal Basic Income (UBI) solutions [80].

This chapter discusses specific approaches and examples of achieving Sybil resistance in centralized and decentralized IDMSs. The organization of this chapter is as follows: in

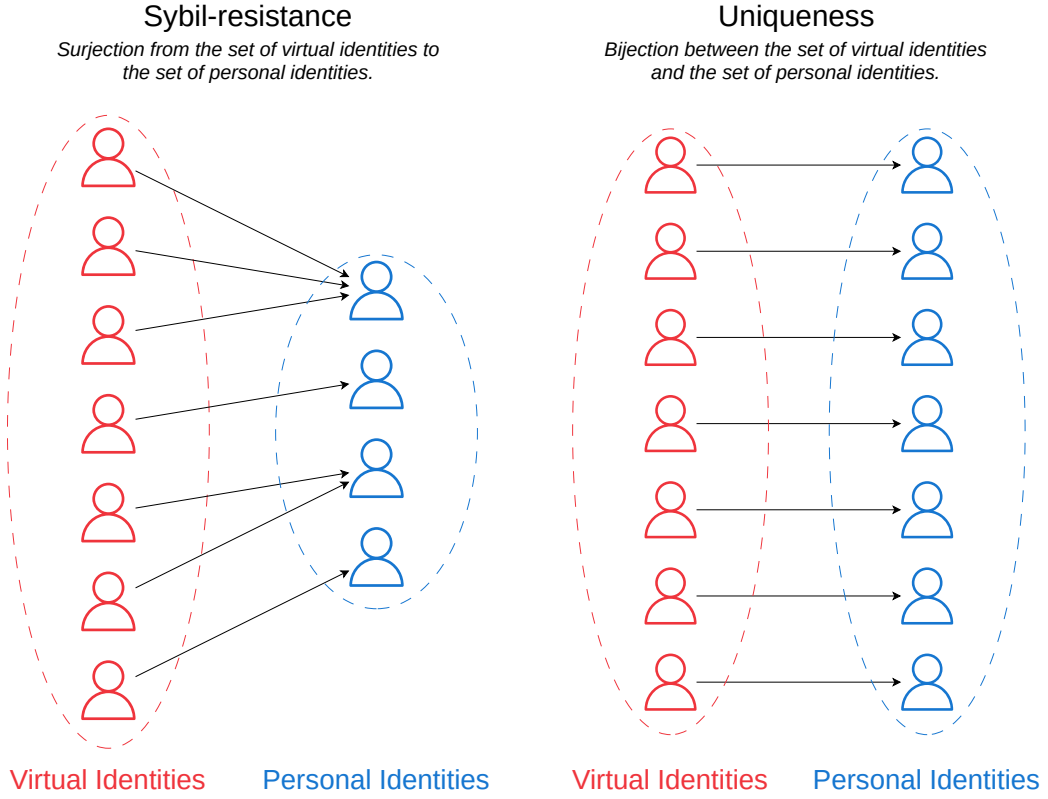


Figure 5.1: Virtual identity uniqueness vs. Sybil-resistance.

Section 5.1, a more precise classification of SRIS is explained. Section 5.2 discusses the essential properties of SRIS. Section 5.3 focuses on centralized approaches for achieving Sybil resistance, while Section 5.4 addresses decentralized approaches. Finally, Section 5.5 reviews existing decentralized SRIS solutions.

5.1 Classification of Approaches

As already mentioned, SRISs are a subset of IDMSs. This means that SRISs encompass the same classifications as IDMSs: centralized, federated, and decentralized. However, in both centralized and federated IDMSs, Sybil-resistance is guaranteed by a centralized IdP. Therefore, the approaches for achieving Sybil-resistance in SRISs can be divided into only two categories: **centralized** and **decentralized**. They employ essentially the same approaches to solving Sybil attacks, but their implementations differ.

Centralized SRISs are controlled by a single authority that manages and issues new identities. This makes system management much easier, but users must have a high level of trust in this authority, as it manages their sensitive personal data and is a single point of failure.

This work focuses primarily on decentralized SRISs, which utilize blockchain technology. However, achieving complete Sybil resistance in decentralized identity systems is significantly more challenging; according to Douceur [26], it is practically impossible. Nev-

ertheless, the idea of avoiding centralized identity management is relevant; numerous such systems exist, and new ones continue to emerge.

For completeness, it is also advisable to introduce a third class of SRISs, namely **SRIS with Trusted Third Party (TTP)**. This class is derived from the decentralized SRISs. Essentially, these systems correspond to traditional decentralized IDMSs, in which trusted issuers of VCs, such as public institutions, governments, or other authorities, issue credentials. When achieving Sybil-resistance, it is important to distinguish between decentralized SRISs with TTPs and purely decentralized SRISs without TTPs.

5.2 Primary Properties

Before diving into the existing approaches and protocols, let us define the primary properties of SRISs. There are quite a few such properties, so they will be divided into four categories: **Core Sybil-Resistance Properties**, **User Sovereignty & Privacy**, **System Architecture & Governance**, and **Social & Operational Goals**.

Core Sybil-Resistance Properties

Except for humanness, these properties are necessary to solve the problem of Sybil identities. If these fail, the system is not Sybil-resistant.

- **Sybil-Resistance/Uniqueness:** Ability of system to ensure one virtual identity per entity or ability to link multiple virtual identities of the same entity [80, 53].
- **Humanness:** The system must verify the entity is a living human, not a bot.
- **Non-transferability:** The system must prohibit virtual identity transferring to different entities [81, 53].
- **Integrity:** Ensuring that virtual identity data is accurate, consistent, and tamper-resistant throughout its lifecycle. In decentralized identity systems, this is often achieved via the immutability of the blockchain [1].

User Sovereignty & Privacy

These properties protect users against attackers and the system itself. They are particularly important for user privacy and security.

- **Anonymity & Unlinkability:** Anonymity means that it is impossible to identify an individual. We also distinguish pseudonymity, i.e., appearing under a pseudonym instead of one's own name. Unlinkability means that it is impossible for an attacker to find a connection between a virtual identity and a personal identity or their transactions [80, 34].
- **Coercion Resistance:** Entities holding identities should be protected against attackers forcing them to compromise their own virtual identity through bribery or violence.
- **Key-Recovery & Revocation:** Users must be able to recover access if keys are lost (resilience) or revoke keys if stolen to maintain control [81, 53].
- **Persistence:** The virtual identity should last as long as the user wishes.

System Architecture & Governance

These properties define who controls the system and the level of trust required.

- **Decentralization:** Absence of a single point of failure or control [80].
- **Transparency:** The system’s operations, code, and governance should be accessible for auditing, or even better, allow for active participation [1].
- **Accountability:** Accountability can be understood on two levels. There should be a mechanism that controls the entities operating the system, such as verifiers, oracles, or trusted third parties. At the same time, there should also be accountability towards the entities using the system. Users who harm the system should be held accountable for their behavior [53].
- **Interoperability:** The identities provided by the system should be usable across different services and blockchains [1].

Social & Operational Goals

- **Inclusiveness:** The system must be accessible to everyone, regardless of their wealth, hardware, or location [34].
- **Equality:** All participants must be treated equally for democratic deliberation and decision-making purposes [34].
- **Scalability:** The system should be sufficiently robust and scalable to be used by billions of users [80].

No system can fulfill all of these properties. Douceur [26] argued that Sybil-resistance is not possible without a centralized trusted authority. Moritz Platt et al. [68] formalized the notions and proved the statement by Douceur, that in a free (i.e., requires expenditure of only a negligible explicit monetary cost) system without a trusted authority, Sybil attacks are always possible. Note that this was achieved only for strongly permissionless blockchain systems, and not for distributed systems in general. For example, proof of resource protocols used in blockchains, such as PoW or PoS, can achieve Sybil-resistance in permissionless systems because they are not free. The visualization of the Sybil attack trilemma is in [Figure 5.2](#). Furthermore, the non-transferability of identities is a major problem that none of the further-discussed decentralized systems can fully solve. However, this is out of scope for this thesis.

Nevertheless, attempts to create new SRIS that will incorporate as many of the above-mentioned features as possible are still ongoing.

5.3 Centralized SRISs – Taxonomy of Approaches

In centralized SRISs, approaches to ensuring Sybil resistance are quite intuitive, and most people have already encountered them. The principles are very similar to those of decentralized systems, as described in [Section 5.4](#) with more details. They primarily utilize **biometric information**, **personal data** (including name, address, age, date of birth, and identification numbers), **social connections**, **behavioral analysis**, and **entity resolution** processes. The summary of these approaches and their characteristics is in [Table 5.1](#).

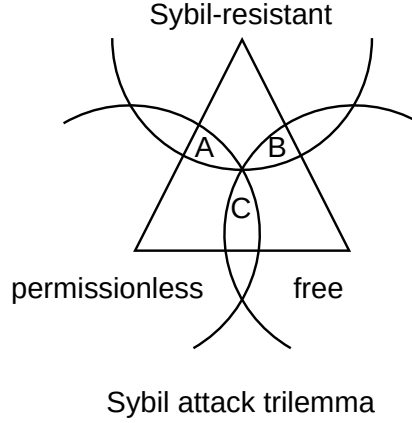


Figure 5.2: Visualization of Sybil attack trilemma. $A \cap B \cap C = \emptyset$ [68].

Since this work primarily focuses on decentralized systems, the description of centralized system approaches is less detailed.

Category	Primary Mechanisms	Trust Model	Example Systems
Biometrics & Demographics	Fingerprints, iris scans, facial recognition, gov. IDs	Government, central authorities	Aadhaar (India), eIDAS (EU), ID.me, Onfido
Social Graph & Behavioral Analysis	Graph connectivity, activity patterns, manual moderation	Private social network platforms	Meta (FB/IG), X, LinkedIn, Google
Entity Resolution	Attribute matching (email, phone, ID), cross-database linking	Public institutions, service providers	Banks, insurance, credit bureaus

Table 5.1: Summary of centralized Sybil-resistance approaches.

5.3.1 Biometrics & Demographic Records

This approach is primarily used by national identity systems, such as passports, ID cards, and driver’s licenses. The information collected for identity deduplication, mainly biometric and other personal information, is highly sensitive. The best-known system in this area is the Indian Aadhaar project [87], which is the world’s largest centralized Sybil-resistant system. This system uses all 10 fingerprints, scans of both irises, and a facial scan for deduplication. In recent years, several regulations and initiatives have been introduced in the European Union, including eIDAS [29] and eIDAS 2.0 [30], which also introduce the concept of the European Digital Identity Wallet (EUDIW). These regulations require Member States, among other things, to establish a system for the unique identification of persons and to integrate it into a pan-European identification system. The European Digital Identity Wallet should also give users full control (self-sovereignty) over their data. User data should be encrypted and can only be decrypted by the user’s digital wallet. The use of ZKPs is also assumed, allowing users to make verifiable statements without disclosing the data itself [76, 29, 30]. Almost every developed country now has similar national systems.

However, the problem of inconsistency remains. The United Nations has already launched the UN Digital ID initiative [86], which aims to unify the individual approaches of different countries and create a universal system.

In the commercial sector, there are solutions like **ID.me**, **Onfido**, and **Jumio** that IDMSs can integrate to ensure Sybil-resistance. These solutions verify official documents, such as passports or ID cards, along with users' biometrics. Existing IDMSs can use these services without developing their own verification systems.

5.3.2 Social Graph & Behavioral Analysis

Among centralized systems, this approach is primarily used by large internet platforms such as social networks owned by Meta (Instagram, Facebook, Snapchat, etc.), LinkedIn, X (formerly known as Twitter), and Google products [80]. Detecting Sybil accounts on these platforms is a major problem for these systems. Therefore, special methodologies have been developed for this purpose. While some methodologies, such as graph-based algorithms, can also be used in decentralized identity systems (some of these algorithms will be described later in Section 5.4.1), others are specific to centralized systems. Centralized systems can collect large amounts of personalized data. On the one hand, such data can be used to develop effective algorithms for detecting Sybil accounts, but this raises a major concern about user privacy invasion. Such methods include monitoring account activity (such as the number of requests, responses, and messages sent), searching for various patterns of behavior characteristic of Sybil accounts, or manual verification by moderators [2, 77].

5.3.3 Entity Resolution

In addition to national identity systems and social networks, various other companies and organizations (such as banks, insurance companies, consumer credit bureaus, etc.) also address the problem of Sybil accounts. In order to ensure the uniqueness of their users, they usually use documents or identifiers issued by national identities for this purpose, but users can mislead or lie when filling in such data.

Entity Resolution (ER) is a process that attempts to identify different entities that represent the same real-world object. In this case, ER is used to identify users who have multiple accounts under fake information that appears to be real. ER uses entity attributes (such as emails, phone numbers, identification numbers, etc.) and attempts to determine whether multiple entities are linked by the same attributes. Entities can be organized into a graph where nodes represent individual entities, and their connections to identifiers, accounts, and attributes form the edges. The topic of ER is widely discussed in academic literature. There are many methods for resolving ER, but the process itself is computationally difficult [89].

5.4 Decentralized SRIS – Taxonomy of Approaches

This taxonomy was primarily developed through the analysis of existing solutions, as there are no comprehensive surveys on this topic. Existing decentralized SRISs employ one of two main principles. Either some form of **human entropy** or **objective incentive** [80]. Human entropy-based approaches encompass solutions utilizing **social graphs**, **biometrics**, or **verifiable claims** about a specific individual. In contrast, approaches using material

Category	Primary Mechanisms	Underlying Principles	Example Systems
Social Graph	Web of Trust, graph-based algorithms (SybilLimit, SybilRank), path finding	Selectivity in befriending; detection of honest and Sybil regions	BrightID, Circles
Biometry	Biological template comparison (iris, facial, fingerprints)	Global uniqueness of biological features	World Network
Physical Presence	Pseudonym parties (physical or virtual simultaneous events)	Physical body locality; CAPTCHA	PoP protocol, Enconter, Idena
Community Verification	DAOs, Verifiable Credentials (VCs), Soulbound Tokens (SBTs)	Community verification, reputable issuers	CanDID, LinkDID, Proof-of-Humanity, HumanityDAO
Mechanism Design	Economic incentives, Proof-of-Resource (PoW, PoS)	Cost of forgery > potential gain	Upala, Token Curated Registries

Table 5.2: Summary of decentralized Sybil-resistance approaches.

incentives are based on **mechanism design**. These are often economic or reputational incentives. The summary of these approaches and their characteristics is in [Table 5.2](#).

5.4.1 Social Graph

Social graph approaches are based on the idea of a web of trust, which originates from the context of Public Key Infrastructure (PKI). PKI is a system for managing public-key encryption and digital certificates. One of its primary tasks is to ensure that communication occurs between authenticated entities. This means that both parties to the communication can be sure who they are communicating with. Today, PKI is mainly centralized around certification authorities, but alternatives such as the web of trust have also emerged. [Figure 5.3](#) illustrates the principal difference of the traditional hierarchically organized PKI, in contrast to the distributed nature of the web of trust. The idea of the web of trust is based on a distributed database of user identities, their public keys, and signatures from other users. Signatures from other users express trust in a specific identity. It is also possible to choose a level of trust, thereby creating a reputation system of sorts. Using this shared database, it is possible to verify a user’s identity and, based on signatures from other users, decide whether they are sufficiently trustworthy. Such a shared database essentially represents a **social graph** of users and their relationships [20, 80]. Since its introduction, the concept of the web of trust has been expanded to include other approaches that aim to enhance Sybil resistance and key recovery. It is no longer just about signing certificates, but about mutual sureties that form a social graph [75].

Graph Algorithms for Sybil Detection

All Sybil detection graph-based algorithms are based on the same assumption that the graph is divided into two regions: an honest region and a Sybil region controlled by the

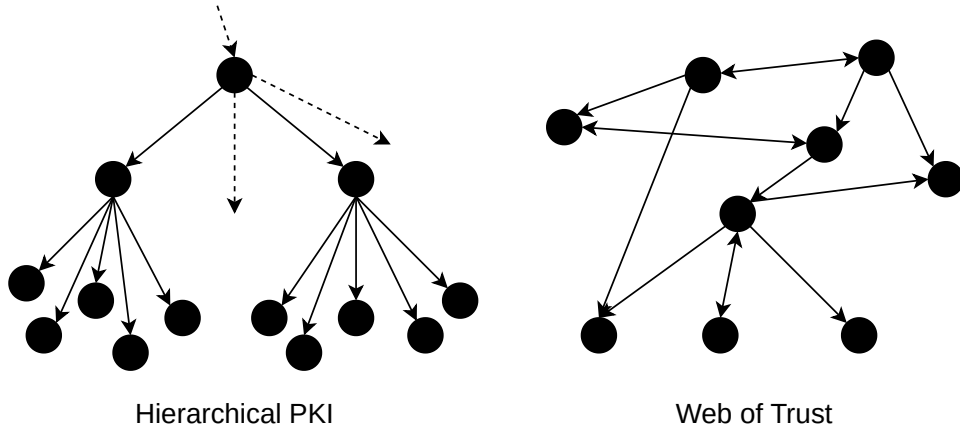


Figure 5.3: Hierarchical PKI vs. Web of Trust [20].

attacker. Because honest users are selective about whom they befriend, there are relatively few *attack edges* connecting the honest region to the Sybil region. A visualization of this assumption is shown in Figure 5.4. The honest region should have a property called **fast mixing**. This property essentially means that the number of steps a random walk needs to reach a stationary (uniform) distribution is small relative to the total number of nodes in the network. In the case of Sybil regions, the *attack edges* act like a bottleneck. A random walk will quickly traverse the honest nodes but will rarely cross these sparse *attack edges* into the Sybil region [2]. There are many algorithms of this kind, each with its own advantages and shortcomings. For example, SybilGuard [94], SybilLimit [93], SybilInfer [25], GateKeeper [85], SybilShield [79], SybilDefender [90], SybilRank [27], SybilFence [19], Integro [10], or SybilHeat [35]. **Notice that most of these algorithms require an honest region defined by some known trusted nodes to distinguish the honest region and the Sybil region.** There are also algorithms that aim to automate the distinction between honest and Sybil regions, but this introduces additional computational complexity, and the resulting inaccuracies have a dramatic impact on the final detection. Here are four examples of commonly used graph algorithms for Sybil detection to give a better idea of how these algorithms work [2, 77]:

- **SybilGuard:** It is employing a specialized type of verifiable random walk called random routes. It identifies Sybil nodes based on whether their random routes intersect with the routes of a trusted node in the graph.
- **SybilLimit:** This is an extension of the SybilRank algorithm. For a suspect node to be verified as legitimate, it must satisfy two primary requirements: the **intersection condition** and the **balance condition**. The intersection condition in SybilLimit requires that the last edge of one of the suspect's random walks must intersect with one of the random walks originating from a trusted verifier. Because the random walk length is $\mathcal{O}(\log n)$, where n is the total number of nodes within the social network, the last edge, also referred to as a **tail**, is like a unique entry point to the honest region. Every time a suspect is verified through a particular tail, that tail's load (the number of verified suspects) is incremented. This is where the balance condition takes place.

Once a tail reaches its pre-defined threshold, it can no longer be used to verify any more suspects. This effectively limits the number of Sybil nodes that can be verified through a single attack edge.

- **SybilDefender:** This algorithm consists of two steps. First, using short random walks from a known honest node to establish thresholds for how many nodes an honest user's walk should typically traverse. Subsequently, a random walk is performed from the suspect node, and if the number of nodes traversed is significantly lower than the threshold, then the node is marked as Sybil. In the second step, the algorithm attempts to isolate the entire Sybil cluster in which the detected Sybil node is located.
- **SybilRank:** This algorithm works on the principle of evaluating nodes based on their trust level. There are several trusted nodes in the network that serve as a source of trust, which spreads further from them. In this way, each node is assigned a level of trust. Due to the weak connectivity of the Sybil region with the rest of the network, Sybil accounts have a low trust value.

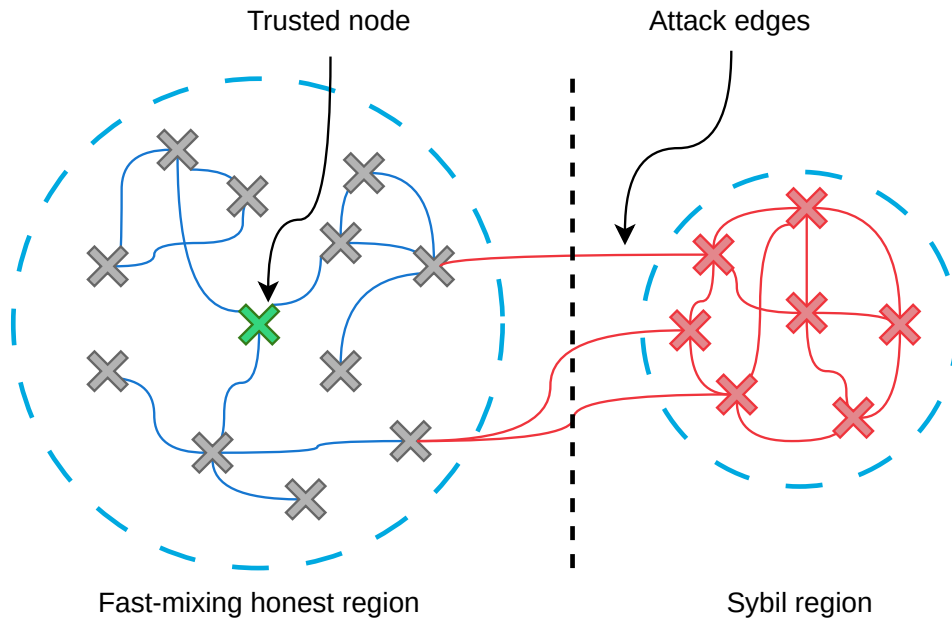


Figure 5.4: Graph-based Sybil detection assumption. Sybil region and honest region [2].

Different Approaches to Social Graph

Social graphs for Sybil resistance are utilized by solutions such as BrightID, as described in [Section 5.5.2](#), or Circles, as described in [Section 5.5.3](#). Although both solutions use graph-based Sybil detection under the same assumptions, they employ different implementations. BrightID, similar to the web of trust, uses a shared database of users and their relationships,

which is analyzed using various anti-Sybil algorithms. Circles, on the other hand, does not use a general social graph analysis but instead uses a path-finding algorithm to connect users through their transitive trust. If two users want to make a transaction, the algorithm must find a path through a chain of mutually trusting users. This enables transactions to occur even between users who do not have direct trust in each other. Although Circles primarily functions as a Universal Basic Income (UBI), it is possible to imagine any user interaction as a transaction, and the amount of tokens they can use on their path through other trusting users as a reputation metric.

Intersectional Identity

In the context of the web of trust, another approach, called *intersectional identity*, has emerged, proposed by Immorlica et al. [42]. Instead of unique identifiers or public keys, it is more natural to use singular claims that define a unique identity. For example, the identity of the first kiss, the place of marriage, places of residence, employment, coworkers, siblings, and so on. A set of such features can quite reliably identify a specific person. If user A wants to verify the identity of user B , A selects a set of features and obtains answers from B . A then verifies the answers from B with other users, such as friends, coworkers, employers, etc., who can answer the same features associated with user B . The number of features should be sufficient to ensure that the identity verified by the answers is unique, but also small enough that A cannot steal the identity of user B .

Issues with Social Graph Approach

Social graph approaches to Sybil-resistance do not prevent the creation of Sybils, but they provide methods for detecting Sybils and minimizing their impact. Special algorithms for Sybil detection in social graphs are not perfect. They are primarily designed to detect Sybil regions of sufficient size, which is a relatively narrow class of Sybil attacks. In real life, people interact with various groups, such as family, coworkers, or friends. It is not a problem to create multiple identities and link each one to a specific group. As long as there are enough good relationships within the group, it is very difficult to detect that someone has multiple identities. After all, creating specific identities (alter egos) in different social groups is very natural for humans. Additionally, even though the data representing the social graph is pseudonymized, it can still reveal sensitive information about users. If someone were to discover the identity of one node, they could use the social graph to find out the real identities of other nodes as well [34]. However, this problem could be solved by using ZKPs [17]. Another problem is determining reputation. Although there are many ways to calculate reputation, the answer to the question of how to correctly calculate someone's reputation is not simple. Furthermore, people living in remote areas may struggle to create a sufficiently representative social graph and reputation score. This may also cause Sybil detection algorithms to falsely report legitimate, but poorly connected users [80, 34].

5.4.2 Biometry

Biometric approaches utilize unique human biological features, such as fingerprints, facial recognition, DNA, iris, or retina images. In most cases, people can use these biometric features to register in a system that verifies that no other person has the same biometric features, thereby confirming the uniqueness of their identity. The used biological features

must be sufficiently unique across the population to reliably perform identification, i.e., verifying the stored template against billions of registered users, which is a 1:N comparison. Firstly, this is a very demanding operation, but it also narrows down the set of biometric features that can be used. For example, fingerprints and facial recognition may not be sufficiently unique on a global scale [91]. Biometric features for identification are already used in centralized systems, such as photos on passports and ID cards. Some ID cards also contain a chip that stores fingerprints and other biometric data. Biometric identity verification offers numerous benefits, including convenience for users, scalability, inclusivity, and the added security that biometrics cannot be lost, unlike private keys [34].

Protocols Utilizing Biometrics

Probably the best-known protocol that fully relies on biometrics as the primary method for ensuring Sybil resistance is World Network [91], as described in Section 5.5.1. However, the principles of biometric verification are also employed by various Sybil-resistant protocols based on decentralized verifiers, such as Decentralized Autonomous Organizations (DAOs). For example, individual DAOs in Upala [71], described in Section 5.5.6, can utilize these principles. Additionally, as previously mentioned, biometric features are frequently utilized in centralized identity systems.

Issues with Biometry Approach

The use of biometric features has its disadvantages and limitations. Although they cannot be lost like private keys, they can be damaged or destroyed. Fingerprints can wear off from hard manual work, and people can lose hands, arms, or eyes in accidents or violent conflicts. Furthermore, there are also many technical, security, and privacy challenges. Biometric hardware is the most important part. It should be well-secured and protected against possible compromise, but even then, there are nonzero chances of false-acceptance rates (FAR) and false-rejection rates (FRR). Another problem is sophisticated counterfeits. Nowadays, it is possible to create quite reliable counterfeit fingerprints and facial images that can even pass liveness checks. Also, a widely discussed topic is the privacy of biometric data. To verify identity using biometrics, all features must be publicly stored or, at the very least, stored in a coded form that allows comparison. This can be very difficult to achieve. Such databases are extremely sensitive, a prime target for hacking. Biometrics cannot be changed, which poses a permanent risk of identity theft to everybody whose biometric data was compromised [34, 91].

5.4.3 Physical Presence

Approaches based on physical presence are founded on the simple fact that a physical person cannot be in more places at the same time. Most of these approaches are based on so-called **pseudonymous parties**. The core idea of pseudonym parties is to utilize the physical presence of people at a specific time and place to verify their unique virtual identities. There is no centralized authority to trust, but some trust is needed towards the party organizers. In practice, a pseudonym party is a real-life event with specified rules, time, and place, organized by an entity (such as an organization, government, company, or interest group), where people can verify their physical presence by attending. Each attendee will obtain an identity token that represents both a physical and virtual identity. Such parties are held regularly, allowing users to renew their tokens [80, 9]. This approach

to protection against Sybil attacks is employed, for example, by the proof-of-personhood protocol [9], described in [Section 5.5.4](#), or the Encounter protocol [12].

Virtual Pseudonym Parties

There are also efforts to transfer pseudonym parties to the virtual environment. However, there is a problem with deep fakes and the ability of artificial intelligence (AI) to mimic human behavior. Parties in a virtual environment would therefore inevitably have to include AI-hard tests such as CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart). In addition, all parties would have to take place simultaneously, so that no one could attend multiple parties and thus obtain more identity tokens [80]. A solution for virtual parties is provided by the Idena blockchain [41], which is described in [Section 5.5.5](#). Idena utilizes the concept of testing users using a mechanism such as CAPTCHA simultaneously around the world. Users do not verify each other as in classic pseudonym parties, but reach a consensus on who has passed a CAPTCHA-like test.

Issues with Pseudonymous Parties

Pseudonym parties enable very reliable verification of membership in a given group. However, for such a system to be usable globally, it would have to be ensured that all parties participate at approximately the same time, so that no party can participate in multiple parties simultaneously. Additionally, physical participation may not be feasible or convenient for many users. Questions also arise regarding the financing, organization, and physical security at such parties. Regarding virtual pseudonym parties, they also require protection against increasingly sophisticated AI-generated forgeries.

5.4.4 Community Verification and Verifiable Claims

With the advent of smart contracts that provide Turing-complete computation, new ways of determining or guaranteeing uniqueness have also emerged. The idea of these approaches is to utilize community verification of identities. However, the community needs something to base its decisions on. One option is to use economic incentives. This method will be discussed in detail in [Section 5.4.5](#). The second option is to use verifiable claims, which can represent the reputation of individuals.

Decentralized Autonomous Organizations

One such method includes DAOs, which are used for automated resource management and governance. With DAOs, participants can control and manage shared resources in real-time or organize various types of elections, all according to pre-agreed-upon rules defined in smart contracts. Such a framework is used in various ways in different proof-of-personhood protocols. For example, new identities in HumanityDAO [73] and Proof-of-Humanity [45] must receive a sufficient number of votes from already verified users. Although it is not just about voting, applicants must pay a fee for their candidacy and provide their social media information in case of HumanityDAO. Alternatively, for Proof-of-Humanity, they must upload a photo, name, and a short video. Usually, DAOs as decentralized verifiers also use a combination of the other mentioned approaches. For example, some form of incentive, either for new candidates or for verified users who vote for them. Additionally,

these systems may require the presentation of biometric information, such as photos or video recordings [80, 34].

Decentralized Identities and Verifiable Credentials

Another method is to use DIDs and VCs. If a person can present themselves using multiple VCs created by reliable issuers, this can serve as proof of identity. The assumption is that an attacker is unable to obtain a large number of reliable credentials in order to create a large number of Sybil accounts. Protocols utilizing this approach include, for example, LinkDID, as described in Section 5.5.8, and CanDID, as described in Section 5.5.7, which focuses more on legacy identifiers than VCs.

Soulbound Tokens

Ohlhaver et al. [64] introduces an idea of **Soulbound Tokens** (SBTs), which are publicly visible, non-transferable, but potentially revocable by the issuer tokens representing the commitments, credentials, and affiliations of accounts called “Souls” (for example, educational credentials, employment history, or memberships). SBT tokens can be used to distinguish between real humans and bots. The more reputable SBT tokens a person owns, the higher their reputation score, which can be used in various anti-Sybil algorithms. The principle is to show a rich social context. For example, DAOs can compute over a Soul’s constellation of SBTs to determine voting power. However, such tokens can reveal private information; therefore, the authors also propose using ZKPs to verify properties without disclosing the underlying data.

Issues with Community Verification and Verifiable Claims

Although this approach certainly guarantees some form of Sybil resistance, it cannot ensure complete Sybil resistance. Individuals who wish to exploit such a system can create multiple identities and distribute their claims among them. Using DAOs, VCs, and SBTs can only determine a kind of reputation score. However, the problem remains of determining the threshold at which it is possible to assume a person is not a Sybil. Partial centralization is also a significant problem. VCs, SBTs, and other claims are issued by trusted issuers. These are usually centralized companies, institutions, or governments. In the case of DAOs, on the other hand, the problem may be the DAO’s insufficient reputation, meaning that even proof of identity verification by the DAO will not have much reputational value. There is also the issue of privacy, as various claims can reveal sensitive information about their holders. However, this problem can be easily solved using ZKPs. Overall, this approach can be combined with the other approaches mentioned to achieve better results.

5.4.5 Mechanism Design and Economic Incentives

Mechanism design, a concept within the field of game theory, along with economic incentives, sometimes referred to as proof-of-investment, seeks to limit users based on the resources they are willing to invest. These approaches cannot ensure the uniqueness of virtual identities, but they can help achieve Sybil-resistance.

Such mechanisms are often used today in popular blockchains, such as PoW or PoS protocols, discussed in Section 2.1.3, but in general any Proof-of-Resource (PoR) protocol [34]. In the case of PoW, the number of virtual identities is irrelevant because the ability to

produce a new block is tied to the computing capabilities of the hardware that the entity possesses. Similarly, in PoS, the ability to produce a new block is tied to the amount of an entity's stake.

In the context of Sybil-resistance, mechanism design is a process of designing a system in which the optimal strategy for a rational actor is to maintain a single virtual identity, or in which the cost of forging a virtual identity exceeds the potential gain. For example, various Sybil-resistant approaches that rely on reputational rather than economic incentives from a game-theoretic perspective have been discussed by Kumar and Bhuyan [47]. Nevertheless, existing Sybil-resistant systems often use economic incentives because they are easier to implement.

Protocols Utilizing Economic Incentives

A typical example of an economic incentive is a Token Curated Registry (TCR). It is a system that was originally designed to replace list owners by creating economic incentives for decentralized list curation. In this scheme, members in the registry hold tokens associated with the list of high-quality items, content, or entities. Members increase trust in the registry by staking tokens, voting, or vouching for other members, thereby motivating more people to add data to the list and increasing its value. Similar mechanisms have been employed in various Sybil-resistant systems, where verified users have an incentive to truthfully verify new users [80]. Such a mechanism is used, for example, by the anti-Sybil identity system **Upala** described in [Section 5.5.6](#).

Issues with Mechanism Design and Economic Incentives Approach

A significant disadvantage of systems based on economic incentives is that they favor wealthy individuals. One proposed solution to this problem is **Quadratic Voting (QV)**. The principle of QV is very simple: the cost of buying new votes increases quadratically, meaning the cost is the square of the number of votes purchased. This approach is used, for example, by various DAOs, but nothing prevents an attacker from distributing their resources among multiple Sybil accounts, thereby effectively bypassing QV [7]. Economic incentives can therefore be effective only when combined with other approaches that make it more difficult for attackers to create Sybil accounts.

5.5 Existing Solutions

This section discusses existing solutions for protecting against Sybil attacks that are in practical use or have been proposed in academic papers.

5.5.1 World Network

World Network [91] is a protocol using a biometric approach to issue and verify proof-of-human credentials. The mission of World Network is to create a globally inclusive identity and financial network.

Protocol Description

The World Network protocol is built on the Ethereum blockchain, but it also uses centralized backend servers. The core component is **the Orb**, shown in [Figure 5.5](#), which is a custom

biometric imaging device that captures high-resolution multispectral images of the iris and face to verify **humanness** and **uniqueness**. The Orb serves as trusted hardware. If a user wants to enroll, they must visit the location where this device is located. The Orb verifies the user and signs a message containing the user’s public key (identity commitment) and their generated iris code. Then, a backend service checks the new iris code against the database of all existing iris codes to ensure uniqueness. If unique, the user’s identity commitment is inserted into an on-chain Merkle tree. The user can then prove their identity by generating a ZKP that they own the private key corresponding to a valid identity commitment in the Merkle tree. The Orb does not store the collected data and only shares the iris code with the backend server, not the actual image of the iris. The iris code is not a hash, but a condensed, binary mathematical representation of the iris texture. The public keys of verified users are stored on the blockchain in a Merkle tree smart contract. Currently, such verification lasts indefinitely, but credentials can be revoked if an Orb is found to be compromised or malicious. In the event of identity theft or loss, repeated verification using Orb enables users to restore their identity, serving as a **key-recovery** mechanism.

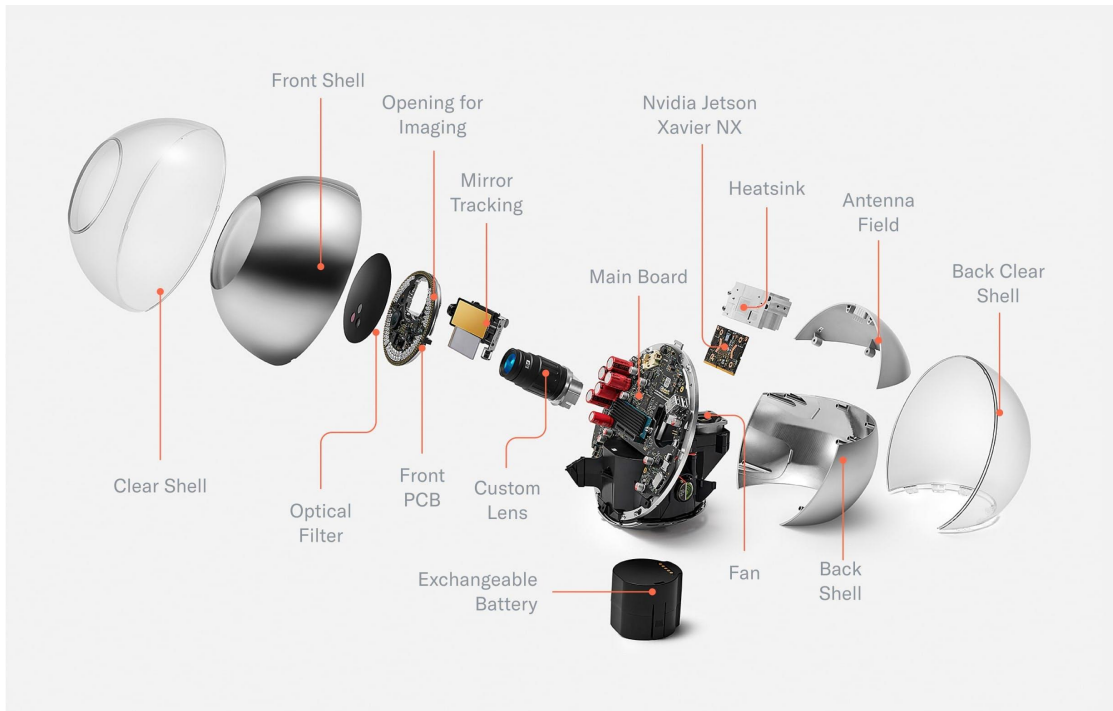


Figure 5.5: Composition of the World Network’s Orb [91].

Security Analysis

When it comes to security and privacy, Orb is the most vulnerable element. No hardware can be perfectly secured against physical compromise. It is very difficult to verify if it is constructed correctly and does not have backdoors. However, both the Orb design and protocol are open source, which contributes to transparency and allows Orb to be manufactured by other suppliers. Biometric data is a particularly sensitive issue, even though the protocol does not work with raw data, only with coded biometrics, from which it is

hard to gain any sensitive information. In addition, the production and processing of data from Orb is currently centralized, although the authors of the protocol have expressed their willingness to decentralize it in the future. Finally, the availability of the Orbs themselves is also problematic. There are not enough of these devices in the world, which makes identity verification difficult to obtain.

5.5.2 BrightID

BrightID [14, 15] is a decentralized social identity network with the primary goal of ensuring Sybil-resistance of the network.

Protocol Description

To achieve Sybil resistance, BrightID utilizes social graph analysis. The social graph is created independently by users, establishing cryptographically signed connections between themselves. The established connections are pseudonymized and stored in a decentralized manner on an Ethereum-based proof-of-authority blockchain called IDChain¹. The decentralization of connection data is essential for independent parties to conduct their own analysis. This also means that nodes can employ whatever analysis methods they want; it is also possible to compare and aggregate different methods. The personal data associated with pseudonyms (names, photos, etc.) is shared only among connections, and no data is shared with centralized organizations.

One of the problems with BrightID and social graph methods in general is the slow bootstrapping process. It takes new users some time to establish trustworthy connections and obtain a sufficient anti-Sybil score. BrightID aims to enhance bootstrapping by utilizing groups and seeds. Groups are small, combined efforts by connected users to help someone become verified. They provide better possibilities for interconnectivity than single connections. A user can belong to multiple groups, but each user chooses one as their primary, which represents their closest personal contacts. On the other hand, seeds are special users that serve as centers of trust. They were selected by the BrightID Main DAO in a permissioned manner and are generally trusted. Seeds can host connection parties over Zoom, where users can participate and connect with a specific seed, which improves the user's anti-Sybil rank. BrightID also uses multiple levels of trust in connections. For the system to function properly, it is essential that users manage their connections responsibly. If a user has someone in their connections who has been marked as untrustworthy, they also lose their anti-Sybil rank. This transitive dependency motivates users to maintain good connections. Additionally, a special set of trusted contacts can be used to restore access or revoke compromised keys.

Security Analysis

The main problems with BrightID stem from the shared social graph. Since user connection data is publicly available, it is possible to correlate public graph metadata with external social media information, potentially leading to the de-anonymization of specific BrightID pseudonyms. Additionally, the anti-Sybil analysis of the graph itself can be computationally intensive for large graphs. In some regards, such as the chosen blockchain or the introduction of special seed users, BrightID has more permissioned access. However, the BrightID

¹<https://idchain.one/begin/>

community is open and declares that seed users should only be a temporary solution until the network is large enough.

However, seed nodes are essential for social graph analysis, as they distinguish between Sybil and honest regions. The graph’s connections and structure alone are not sufficient for analysis. It is necessary to account for the fact that an attacker can create and connect a large number of Sybil nodes that can perfectly mimic the structure of an honest region. Without seed nodes whose trustworthiness has been verified by some other means, it is impossible to distinguish such a Sybil region from an honest region.

5.5.3 Circles

Circles [49] is a decentralized Universal Basic Income (UBI) protocol that features a Sybil resistance mechanism to safeguard against users creating multiple accounts.

Protocol Description

Before explaining the actual Sybil-resistant mechanism, let us introduce the idea of the Circles protocol. Circles allows every user to issue **their own tokens**, with a rate 1/hour. The token/currency is referred to as *CRC* overall, but **on the macro level, the tokens are different for each user** so they are non-fungible across accounts. We can imagine it as *Alice-CRC*, *Bob-CRC*, *Carol-CRC*, and so on. Users can accept others’ tokens if they **trust** the issuer of that token. The example of a transaction using a transitive trust relation is in Figure 5.6. Carol wants to buy something from Bob for 5 *CRC*, but Carol can not use its own *Carol-CRC* because Bob has no trust in them. The protocol finds a transitive path between Carol and Bob through Alice. Bob trusts Alice and Alice trusts Carol, so Carol can pay 5 *Carol-CRC* to Alice, and use 5 *Alice-CRC* to pay Bob. Overall, a longer sequence of transitive transfers may be required to make a transaction possible. The protocol assumes that utilizing transitive trust relations allows everyone to make transactions with each other without requiring direct trust between users to interact. So **on the micro level** the tokens/currency will appear just like one universal token called *CRC*. Note that these mechanics are fully abstracted away from the user, who only manages their own trust connections.

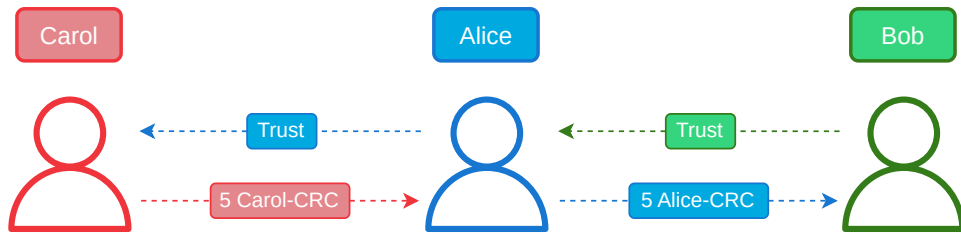


Figure 5.6: Circles transaction using transitive trust relation [49].

The reason why Circles does not simply issue fungible *CRC* but instead makes the tokens different for each user is part of the **Sybil resistance mechanism**. Account creation is not restricted in any way, but the main essence of the mechanism is **trust**. Figure 5.7 shows a fake (Sybil) account created by Alice. Alice intended to issue 2 *CRC*/hour to gain an advantage over others. However, Bob trusts only Alice’s primary account. This

makes Alice’s fake account useless, because *Fake-CRC* tokens can be swapped only with *Alice-CRC* tokens, but their amount is limited to a rate 1/hour. Therefore, Alice’s primary account will always limit the amount of *CRC* Alice can use unless somebody else gives trust to Alice’s fake account. While this ensures a relative Sybil-resistance, it also limits every transaction amount to account balances throughout the sequence of transitive transfers. In summary, the power of Sybil’s accounts (or any account) is limited by the trust honest users give them. The trust can always be revoked, which allows for isolating Sybil networks.

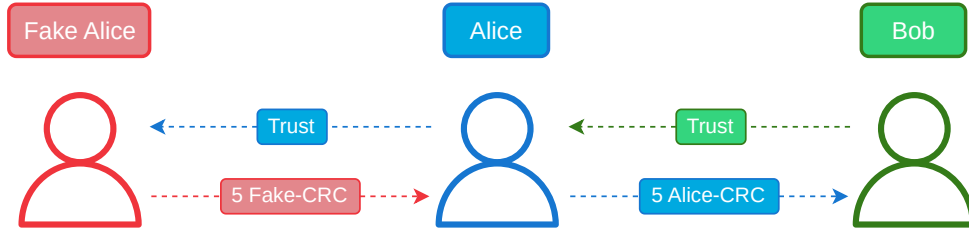


Figure 5.7: Circles Sybil resistance example [49].

Security Analysis

One of the biggest problems of Circles is that trust carries a risk of being exposed to a malicious user. Users need to be very careful to whom they give their trust. One bad connection can result in a big loss of valuable tokens in exchange for worthless ones. Why should users even trust anybody? Users benefit when someone trusts them, so what motivates them to give trust? Giving a trust makes users’ *CRC* more spendable across the network, which is attractive for other users to trust, as it means their tokens will be more spendable [49, 48].

In conclusion, Circles Sybil’s resistance is solely in the hands of users. For new users, building a reliable trust network can be challenging, as can establishing trust with others. Even an honest user’s account can be compromised and misused against others who trust them. This encourages users to be very cautious about whom they trust, which can make transactions across the network more challenging. Furthermore, because transactions are recorded on a public blockchain, it is possible to see trust connections between users, which can reveal sensitive privacy information.

5.5.4 Proof of Personhood

Proof-of-Personhood (PoP) [9], introduced by Maria Borge et al., is a mechanism for binding physical entities to virtual identities based on participation in real-life **pseudonym parties**.

Protocol Description

The party design is divided into three parts: *party setup*, *party operation*, and *party end*. During the *setup*, organizers of the party establish the place, date, expected start and end times of the party, the barrier time at which identity tokens will start to be issued, and the PoP token’s expiration date. Organizers then exchange their public keys and select a group of **observers** (an independent set of people) to record the party and produce a video file. The above-mentioned details of the party are included in a configuration file and made publicly available. A person who decides to attend the party downloads the configuration

file and creates a public and private key pair. The party *operation* has a strict set of rules designed to prevent anyone from obtaining more than one identity token for participation. At the start of the party, attendees are allowed to enter the place. Once the barrier time is reached, the doors are closed, and nobody is allowed to enter, but for safety reasons, anyone can leave. After this point, attendees form a line and **each organizer confirms one-by-one each attendee’s personhood by storing the attendee’s public key** and marking the attendee with an ink stamp to visually recognize that the attendee has already been confirmed, preventing gaining multiple tokens. At the *end* of the party, organizers stop collecting public keys and create a signed transcript, which is publicly available. The party transcript contains a hash of the video files recorded by the observers and a list of attendees’ public keys. The PoP-token received by attendees is composed of the attendees’ public and private keys, as well as the same list of attendees’ public keys as in the transcript [9]. The reason for storing the list of public keys of all attendees in a PoP token is to preserve anonymity through the use of a linkable ring signature scheme.

Additionally, PoP can ensure coercion resistance at the moment of the pseudonym party by providing participants with one real token and several fake ones. Only the attendee knows which token is real and which tokens are fake. They can safely give a fake token to a coercer or vote-buyer without the recipient knowing that it is a fake token [34]. The problem is that the device issuing tokens can be compromised, and the list of real and fake tokens cannot be transparent because an attacker could distinguish between tokens. Only special nodes can distinguish real tokens, which makes the system less transparent and more centralized.

Security Analysis

Notable issue is that this protocol design does not prevent the acquisition of multiple identity tokens if an individual attends multiple pseudonym parties. The original mechanism assumes local use in small groups, which could eventually form a federation and synchronize so that pseudonym parties are held at the same time (or approximately the same time) in different locations, so that no one in the federation can participate in multiple parties.

5.5.5 Idena

Idena [41] is a blockchain driven by a proof-of-person consensus mechanism. This consensus is based on verifying users’ humanity and ensuring Sybil-resistance. Every user verified in this way has the opportunity to participate in the consensus. The primary goal of Idena is to ensure the humanity and uniqueness of its participants while maintaining the confidentiality of personal data.

Protocol Description

To this end, Idena conducts regular Turing tests for users, which take place simultaneously around the world. These Turing tests, also known as FLIPS (Filter for Live Intelligent People), are tasks that are easy to solve for humans but difficult for artificial intelligence (bots). Example of a FLIP is shown in [Figure 5.8](#). The primary concept behind this mechanism is that only humans can pass the test, and the time limit for completion is extremely short, ensuring that humans cannot complete it multiple times. The frequency of testing is determined by the size of the network; the larger the network, the less frequently the tests are conducted. However, testing must take place at least once every 28 days.

Idena also defines an account hierarchy so that only users who have already successfully passed several tests can invite new candidates. If a user does not participate in regular testing, they will be unable to participate in consensus for the period until the next test and will gradually lose their verification class. To prevent identity trading from becoming profitable, Idena introduces an incentive in the form of a cryptocurrency that also functions as a Universal Basic Income (UBI). Identity owners can cancel their accounts and withdraw their coins. When selling an identity, buyers cannot be certain that the seller will retain their private keys and cancel their accounts. FLIPS are created by validated participants and stored as encrypted data in the network until the test begins. The nodes must agree on the correct answers for the FLIPS in the test as part of the consensus. If consensus is not reached, then the flip is disqualified. FLIPS can also be reported and disqualified. In such a case, FLIPS are not counted, and the authors of these FLIPS are not rewarded. The concept of creating FLIPS is very simple [41]:

1. A verified user receives randomly generated keywords from the protocol.
2. The user then uploads four images that represent a story in the correct order.
3. The user creates a second alternative order of images, which does not make sense in that order.
4. The test involves determining which of the two image sequences makes sense or tells a coherent story.

Security Analysis

With the mechanism used by Idena, it is not possible to achieve uniqueness among participants, and Sybil-resistance is also questionable. It is quite likely that some users will be able to solve multiple FLIPS within the test time limit and thus obtain multiple identities. Another disadvantage is the need for extensive coordination of nodes to enable simultaneous testing around the world. If a node fails to participate, it loses its identity verification and the ability to participate in the consensus [80]. According to analysis by Ohlhaver et al. [63] Idena is vulnerable to account trading and formation of collusive groups, where it is advantageous for users to cooperate or sell access to private keys for a reward. Ohlhaver et al. also highlight that while Idena has a decentralized architecture, by May 2022, about 0.6% of entities of the network controlled 40% of all accounts. The user-friendliness of this system is also questionable. It is challenging to envision users regularly participating in a test at a precisely defined time.

5.5.6 Upala

Upala [71] is an anti-Sybil system based on economic incentives. The core concept is to measure identity reliability in dollars, referred to as a **score**. This metric is called **Price-of-Forgery (PoF)**. The idea of the system is not to attempt answering the philosophical question *Is this a real human?*; instead, it answers the economic question *How much does it cost to forge this account?*

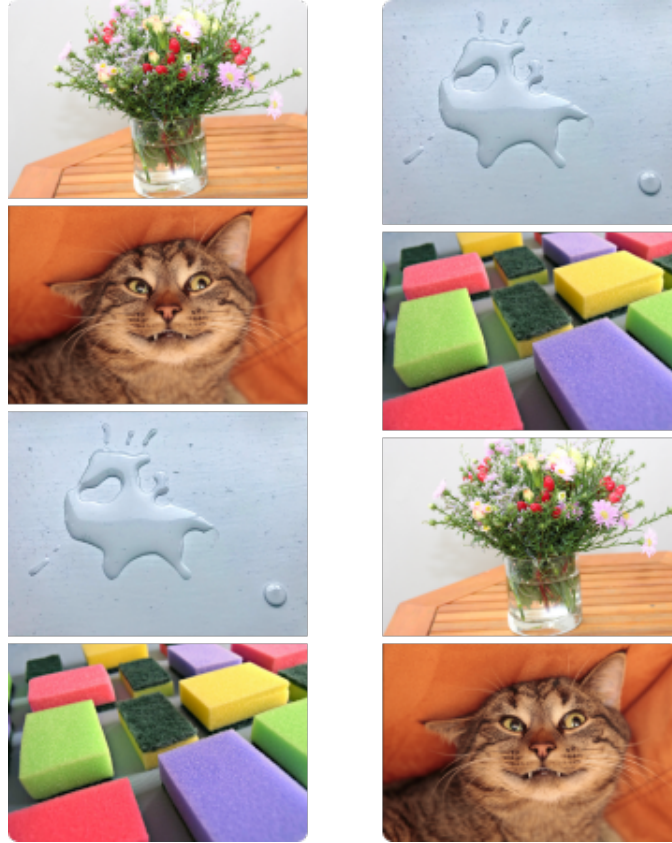


Figure 5.8: Example of Idena’s FLIP (Filter for Live Intelligent People) [41].

Protocol Description

The primary building block of Upala is a **group**. Groups are formed by users, typically represented as DAOs. These can be family, friends, colleagues, or anyone else accepted by the group. Each user must contribute to the group’s shared pool of money. At the same time, users in the group assign each other a score that reflects the level of trust they have in the user within that group. The score is a value in dollars that exceeds the contribution the user has made to the common pool. To ensure these scores are accurate, Upala uses a mechanism called “Explosion” (or self-liquidation). Any user can, at any time, “explode” their virtual identity. This action immediately pays them the cash value of their score, but permanently destroys their digital ID. Users must therefore be very careful about who they accept into the group and what score they assign them. Upala does not handle user verification; it just provides an economic trust model. Each group can determine its own method for verifying the identity of its members, for example, by utilizing other systems like BrightId or Idena. However, if a malicious user manages to gain access to a group, it is very easy for them to drain all the money from the shared pool. To accumulate a higher total score, a user can be part of multiple groups, and their total score is the sum of the scores from each group.

The system is realized as a set of smart contracts that manage the logic of scores, pools, and “explosions”. It ensures that if a user decides to “explode”, the funds are trustlessly transferred from the Group’s pool to the User.

Security Analysis

Economic incentives cannot fully guarantee Sybil resistance. A wealthy individual can create their own network of groups and obtain a high score. Ultimately, it is not only the score itself that matters, but also the reputation of the groups themselves, which is more similar to the concept of a social graph. A large part of security and privacy is placed on the groups themselves, which manage user verification. User scores and their group affiliations are publicly available, although users are listed under pseudonyms. In addition, bootstrapping such a system is also problematic. For such a system to be useful, user scores must be sufficiently large, which requires a substantial amount of capital.

5.5.7 CanDID

CanDID [53] is defined as a decentralized identity system that solves legacy compatibility with existing web services, Sybil-resistance, accountability, and key recovery. It is also compatible with the W3C VCs standard and blockchain-based DIDs.

Protocol Description

The Sybil-resistance mechanism is built around the decentralized CanDID committee. The committee is a decentralized set of nodes that act as a credential issuer. They collectively store a secret key for signing the user’s **master credential**. Besides the committee, there are also Legacy Identity Providers (LIPs) representing unmodified web services (such as social media and online bank accounts) that hold data about the user, and Oracles like DECO² or Town Crier³, which securely and privately port user identity and claims from the legacy providers. The process of issuing a Sybil-resistant master credential consists of two steps. First, the user uses the oracle to obtain data from a legacy provider that contains a unique identifier, such as a Social Security Number (SSN) in the US (the user can use multiple identifiers). The user then creates a verifiable commitment to the identifier, known as a pre-credential, which proves that the identifier is associated with them. In the second step, the user sends the created pre-credential to the committee. To maintain privacy, the user first “blinds” this unique identifier with a random secret factor and proves correct blinding using a ZKP. The committee nodes collectively execute a Multi-Party Computation (MPC) protocol to compute a hashed, unique representation of the identifier using a secret key shared among them. Afterward, the committee checks a shared database of registered users’ attributes, known as the IDTable, to determine if this unique hash already exists. If the hash is not found, the committee issues them a master credential and adds the new unique hash and the user’s public pseudonym to the IDTable. If the hash is found, the credential issuance fails. After successful registration, the user can present themselves using a signed master credential, and the verifier only needs to verify the signature from the committee and whether the user is on the publicly shared revocation list.

CanDID also features a mechanism for key recovery. Users can privately back up their keys via secret sharing with the committee and specify recovery accounts on web services of their choice.

²<https://www.deco.works/>

³<https://town-crier.netlify.app/>

Security Analysis

Although CanDID presents itself as decentralized, its functioning depends on unique identifiers issued by a centralized authority (such as the SSN). The proposed system also selects the committee in a permissioned model, and the chosen Oracle technology must be trusted to faithfully report data from the legacy providers and maintain privacy. However, the committee is the most important part of the entire system. Users must have a high level of trust in the committee. Compromising several committee members can lead to a loss of trust in the entire system and the leakage of sensitive user information. Another problem is that a user may have multiple unique identifiers from LIPs. In the current CanDID system, it would therefore be possible to create multiple master credentials. Deduplication across different identifiers from LIPs is challenging, and there is no single, universal, unique identifier. Polgar [69] highlights that each transaction requires communication with committee members, which can cause a computational bottleneck and also introduce privacy vulnerabilities where a single malicious node can link different users. Additionally, Polgar reviews CanDID’s MPC-based deduplication of identifiers as slow.

5.5.8 LinkDID

LinkDID [81] is not only a Sybil-resistant mechanism, but a whole web3-native identity scheme that eliminates reliance on centralized authorities. Besides Sybil-resistance, it also provides management of identities and credentials with properties such as selective disclosure, credential non-transferability, identity privacy, and key recovery and revocation. The Sybil-resistance is based on the assumption that people with more fulfilled credential records can be considered more trustworthy. The limit on the number of credentials a user should present can be dynamically adjusted according to the required level of Sybil-resistance. Although this scheme is completely decentralized, with each user managing their own virtual identity, a certain amount of trust is necessary among the issuer of a credential, its holder, and the verifier.

Protocol Description

A user can have multiple pseudonyms, i.e., multiple identities to which credentials are linked. The mechanism designed in LinkDID uses a smart contract, Verifiable Data Registry (VDR), and a decentralized ledger to associate credentials under different pseudonyms. It utilizes Zero-Knowledge Proofs (ZKPs) to ensure unlinkability of pseudonyms. The association of pseudonyms (identifiers) takes place in two steps. First, the user registers their identifiers using VDR. The user uses ZKPs to prove they possess the private key corresponding to the identifier without revealing the identifier itself. This mechanism also addresses **non-transferability**, as transferring virtual identity also means giving up all associated credentials. Subsequently, multiple registered identifiers are aggregated into a single associated identifier. ZKPs are used again to prove this aggregation to the VDR, which remembers the resulting aggregated identifier and invalidates the constituent identifiers using nullifiers to ensure they cannot be associated separately again. The user can then use ZKP to prove that the specific identifier used for the presentation belongs to an already recorded associated identifier and, at the same time, generate a unique campaign-specific nullifier for a specific campaign (e.g., Airdrop), which is based on the associated identifier and the campaign ID. The Verifier records this nullifier. If the user attempts to participate again in the same campaign using any other identifier that belongs to the same

associated set of user identifiers, the verifier will detect and reject such an attempt. The user can present various credentials required for a specific campaign, belonging to different identifiers, and add proof that the identifier to which the credentials belong is included in the associated identifier and has not yet been used in the campaign. Users can also recover keys by proving to the VDR their knowledge of the associated credentials. If proving the credentials was successful, the public key stored in the VDR can be restored.

Security Analysis

LinkDID’s design for protection against Sybil attacks can be effective if the verifier requires a sufficient number of credentials from trusted issuers. A user can create two disjoint sets of pseudonyms, each with a sufficient number of credentials, and associate two identifiers with each set. This allows the user to participate in a single campaign multiple times. However, maintaining any large number of disjoint sets of identifiers with credentials from trusted issuers can be very challenging. The mechanism is designed to make it strategically necessary and beneficial to merge identifiers, thereby permanently removing their ability to use those identities for Sybil attacks.

5.5.9 Summary of Existing Sybil-resistant Identity Systems

Not all SRIS properties have been described for each system mentioned in this section. Since these are decentralized systems, properties such as decentralization, transparency, and integrity in most systems derive directly from the properties of blockchains. The primary goal of these systems is to ensure uniqueness, but most systems do not strictly prevent the creation of multiple identities; rather, they aim to minimize or limit the creation of false identities. [Table 5.3](#) summarizes the properties of the discussed SRISs. The ● (full circle) means that the system fulfills the property, ◐ (half circle) means that the system partially fulfills the property, ○ (empty circle) means that the system does not fulfill the property, and ? means that the system is not discussing the property. When assessing uniqueness and Sybil resistance, no system can achieve 100% effectiveness. Therefore, this property is considered fulfilled if the system has very good Sybil-resistance and partially fulfilled if the system noticeably reduces the number of Sybils.

Non-transferability Problem

As can be seen in [Table 5.3](#), all decentralized SRISs share a common difficulty in solving the non-transferability of acquired identities to other individuals. Some systems try to incentivize users so that they do not voluntarily refrain from transferring their acquired identifiers. For example, if someone wanted to transfer an identity in BrightID, it would necessarily mean revealing their trusted connections. Similarly, in the case of CanDID, the transfer would also result in the loss of the associated legacy identifier, or in the case of LinkDID, the loss of associated credentials. However, it is always possible to hand over the private key to someone else, and technically, this problem cannot be solved.

Coercion and Identity Trading Problem

Coercion and identity trading are subproblems of non-transferability. Unlike a scenario in which users voluntarily give up their identities, there are potential solutions that leverage incentives to prevent users from selling their virtual identities and key recovery mechanisms.

Property	World Network	BrightID	Circles	PoP	Idena	Upala	CanDID	LinkDID
Uniqueness/Sybil-resistance	●	●	●	●	◐	◐	●	◐
Humanness	●	○	○	●	●	○	○	○
Non-transferability	○	○	○	○	○	○	○	○
Integrity	●	●	●	●	●	●	●	●
Anonymity & Unlinkability	●	◐	◐	◐	●	◐	●	●
Coercion Resistance	○	◐	○	◐	○	○	◐	◐
Key-Recovery & Revocation	●	●	?	?	?	?	●	●
Persistence	●	●	●	○	○	●	●	●
Decentralization	◐	◐	●	●	◐	●	◐	◐
Transparency	◐	●	●	●	●	●	◐	●
Accountability	◐	●	●	●	◐	●	◐	●
Interoperability	●	●	?	◐	?	●	●	●
Inclusiveness	●	●	●	●	◐	◐	◐	●
Equality	●	◐	●	●	●	○	○	●
Scalability	●	◐	●	◐	◐	●	◐	●

Table 5.3: Summary of the properties of individual Sybil-resistant Identity Systems.

In the case of coercion, even if an attacker can obtain an identity, its true owner can restore it. On the other hand, in identity selling, the buyer cannot be sure that the seller will not restore their key. Therefore, a strong key recovery mechanism is a crucial prevention against coercion. For example, in BrightID, a special set of trusted connections can help the rightful owner of the identity recover a stolen key. However, the problem persists as long as it is possible to relinquish the key recovery mechanism and transfer it to someone else.

Usability

For any system to be user-friendly, it must be practical for its users. In this context, the persistence of a user’s acquired virtual identity is crucial.

Systems like Idena and PoP tend to be user-unfriendly because users are required to regularly renew their virtual identities. Similarly, systems that rely on economic motivation present challenges in terms of user-friendliness, as users often need to keep assets pledged as collateral. This approach also lacks inclusivity, as not every user can afford to pledge a large amount of assets.

Additionally, systems that utilize a social graph approach encounter the “cold start” problem. New users take time to establish enough connections to receive verification from the system.

On the other hand, from a usability perspective, systems like CanDID and LinkDID are much more advantageous. They employ VCs that users can also utilize for other purposes.

Attacker Models

In summary, potential attackers on SRISs seek to achieve one of the following:

- **Obtaining multiple identities** – every SRIS tries to prevent this. The mechanisms for achieving this have been discussed throughout this chapter.
- **Stealing the identity of other users** – attackers can use blackmail or other types of fraud. This attack cannot be completely prevented, but it is possible to implement robust mechanisms for restoring lost identities.
- **Compromising the system in other ways** – malicious users can sell or lend their identities. For specific systems, there may be different ways to compromise the system. For example, sabotaging the pseudonym party in the case of PoP or sabotaging Orb in the case of World Network.

Chapter 6

Design of a Sybil-Resistant Identity System

This chapter discusses the design for a new Sybil-resistant identity management system. The goal of this system is to provide its users with a token or proof that they have successfully met the protocol requirements to be considered non-Sybil. The proposed system can work in conjunction with decentralized identifiers and also issue verifiable credentials.

6.1 Assessment of Approaches and Existing Solutions

Before diving into the design itself, this section discusses my views on which approaches and solutions have the most promising properties for achieving Sybil resistance and possible improvements. These considerations guided my decisions when designing the final system.

Existing Solutions

Existing solutions either do not achieve Sybil-resistance because they cannot prevent the creation of Sybil accounts entirely, or they are not fully decentralized. The most promising projects for me are BrightID and World Network, from which I drew the main inspiration for my solution, while striving to improve upon their shortcomings.

However, I also drew some inspiration from CanDID and LinkDID. These solutions rely on centralized systems and other trusted third parties. Specifically, in social graph analysis, it is necessary to have some sources of trust, and for this purpose, the approaches used in CanDID and LinkDID can be useful. Using the social graph, it's possible to leverage the existing trust of current users and extend it to others. This approach allows the network to build trust with a larger segment of its users without depending on third parties.

The other solutions that use pseudonymous parties are less usable, as they require continuous verification and cannot achieve reliable Sybil-resistance in a digital environment. That is why I did not consider such approaches in my design, nor did I consider money-based systems like Upala, which I consider to be non-inclusive. At the same time, I wanted to avoid money-based mechanisms and build a blockchain-based system that would not involve cryptocurrencies directly.

Approaches

All approaches for decentralized SRIS should be considered, except the physical presence approach. This concept is not user-friendly, so users are unlikely to adopt it; thus, creating such a system seems pointless.

Looking at the other four approaches, namely social graph, biometry, community verification, and mechanism design, none of them is perfect on its own. However, combining these approaches seems like a good way to create a robust system. These are my views on the individual approaches that drove ideas about the final design:

- **Social graph** is an effective approach for detecting Sybil entities while preserving privacy. It can be combined with all of the other approaches. Especially with community verification and mechanism design. Community verification can introduce a reputation score based on claimed credentials or SBTs, and mechanism design can ensure that users maintain only truly trusted connections. The only problem is that the social graph must be public to be analyzed; even with pseudonyms, it is possible to gradually de-anonymize parts of the graph and reveal connections between entities. However, it is possible to use Incrementally Verifiable Computation (IVC) with ZKPs to enable social graph analysis without keeping it public.
- **Biometry** is a powerful approach, but it requires a reliable entity or system, such as the Orb in the World Network, to authenticate users' biometric data. Given the sensitivity of biometric information, it cannot be decentralized in a typical manner, as it is crucial to keep this data private. However, there is a blockchain project called Oasis [61] that uses Trusted Execution Environments (TEEs) to enable privacy-preserving smart contracts and secure data processing. Protocols like this can be used to verify biometric data while preserving privacy.
- **Community verification** can be considered a fundamental approach to decentralization, in which people verify themselves within the community and, together, build trust in the system as a whole. However, in practice, there can be a certain level of centralization, and simply relying on verification from the community, VCs, or SBTs may not be sufficient. This approach works best when combined with the social graph method, as it helps establish a form of trust. The social graph alone lacks significance unless it includes recognized, trustworthy entities that can define regions that are not Sybil.
- **Mechanism design** can be used as a complement to other approaches. While it may not be very impactful on its own, applying well-defined game-theoretic principles can encourage users to avoid making decisions that could destabilize the system. In my opinion, a mechanism should rely on *personal reputation* rather than financial incentives, making it more inclusive. However, creating a robust and decentralized reputation system is quite challenging. In contrast, cryptocurrencies and tokens are intrinsic to blockchains, which makes them easier to implement.

6.2 Design Overview

The novel SRIS presented in this thesis primarily combines **biometry** and **social graph** approaches, but also marginally community verification and mechanism design.

As a biometric feature, **facial biometrics** is used because it is universal and can be obtained easily with a phone camera. The verification process can be divided into three parts, which will be described in detail in [Sections 6.3 to 6.5](#):

- **Registration:** An entity enrolls its facial biometrics, thereby creating a profile on the blockchain.
- **Connection establishment:** Registered entities can form connections with one another, thereby creating a social graph. These connections are private and visible only to the entities involved.
- **Proof of connection with other honest entities:** Entities generate proofs of their connections and forward them to other connections, which recursively use them in their own proofs. I call this process a **private random walk**, which ultimately forms **paths through the graph**. These paths can be used as proof that an entity is strongly connected to other trusted entities. However, the social graph itself is not stored anywhere.

The combination of facial biometrics and social graphs analysis offers promising capabilities. Different biometrics, such as iris or retina scans, could be sufficient for achieving Sybil resistance; however, because facial biometrics are not globally unique, they must be combined with another approach. Furthermore, to make the system as user-friendly as possible, users are allowed to perform face scans on their own devices, which poses a risk of fraud and scan forgery. There could be a situation where a user uses many different sophisticated masks and registers many profiles. This would be very time-consuming and resource-intensive, but possible. These shortcomings are addressed by social graph analysis and a specific method for establishing connections, which will be described in [Section 6.4](#).

The remainder of this section discusses the challenges encountered in the biometry and social graph approaches, as well as the final properties and actors of this system.

6.2.1 Biometric Verification

While achieving Sybil-resistance through biometrics is uncommon in decentralized SRIS, leveraging individuals' unique physical characteristics is an effective way to ensure uniqueness among enrolled entities. The only known systems utilizing this approach are World Network and some DAOs that may use it to authenticate their members. This limited adoption is largely due to concerns about privacy, global uniqueness, scalability, fraud, forgeries, and deepfakes.

Privacy

Biometric characteristics cannot be changed. If they are stolen, this poses a lifelong risk of identity theft. Therefore, it is important to protect biometric data, and certainly they cannot be stored publicly on a blockchain.

However, this raises the question of how to implement the recognition process itself in a decentralized manner while maintaining privacy. The only way I know of to achieve this is through a decentralized network of Trusted Execution Environments (TEEs), such as the Oasis Network [61]. With the usage of TEEs, even the entity performing the recognition does not have access to the biometric data, and the data itself can be stored in encrypted form so that only the TEE can decrypt it using its own key.

Additionally, regarding facial biometrics, the abundance of photos on the internet and social networks suggests that this type of biometric is less sensitive than others. Therefore, it can be assumed that users will be more open to facial biometric verification than to retinal or iris scans.

Global Uniqueness

Biometric characteristics, such as fingerprints or facial features, may not be unique across the entire population. On the other hand, iris or retinal scans require special equipment and may not be user-friendly. As mentioned earlier, this design uses **facial biometrics** because it is universal and can be obtained easily with a phone camera. The lack of uniqueness of this type of biometrics across the entire population is addressed by combining it with a social graph analysis. Entities that fail the facial biometrics uniqueness test must obtain endorsements from other registered users.

In its Face Recognition Technology Evaluation (FRTE) [60], the National Institute of Standards and Technology (NIST) compared over a hundred algorithms. The false positive rate – meaning the rate at which an algorithm incorrectly identifies a person – is relatively low, typically in the thousandths of a percent. However, these false-positive rates differ among algorithms by ethnicity.

Scalability

When registering a new template, it is necessary to check whether the same template has already been registered, which can be a demanding operation. A biometric template is typically stored as an embedding, which is a vector in a multidimensional space. This space can be divided into several clusters, and during recognition, only the necessary clusters are fetched instead of the entire database. This technique is called the **Inverted File Index (IVF)** [66]. Reference vectors, called centroids, divide embeddings into individual clusters. When a new vector is registered, its nearest centroid is determined, and only a few surrounding clusters are fetched.

Even if the database is encrypted, it is possible to perform such indexing and tag the embeddings within a TEE.

Fraud, Forgeries, and Deepfakes

The biggest challenge is ensuring that the scanned biometric characteristics belong to a specific individual. In facial biometrics, various methods, such as masks, photo editing, or AI-generated photos, can be exploited. Therefore, various checks must be performed during registration to prevent such fraud. However, there is likely no perfect way to prevent this. World Network uses Orb, a specialized piece of hardware that requires a physical visit. However, if the protocol allows users to register on their own devices, all possible forms of fraud must be considered.

To prevent photo editing or the use of AI-generated images, the Coalition for Content Provenance and Authenticity (C2PA) standard [24] specifies the use of cryptographically signed metadata attached to a photo or video. This metadata can prove that the content was captured by a specific app on a specific device, at a specific time, and that it has not been changed since it was signed.

This is a relatively new initiative, but the latest mobile phones, such as the Google Pixel 10, already support C2PA, which allows every photo to be cryptographically signed

at capture. It is not just a self-signature. The signing key is stored in the TEE and tied to the manufacturer's trust, which is what makes attestation meaningful. Essentially, it says that the signature was created using a key protected by secure hardware on a trusted device that passed integrity checks.

6.2.2 Social Graph

To use algorithms for social graph analysis, the entire graph must be available as input. For this reason, this design does not use the classical algorithms mentioned in [Section 5.4.1](#). However, the same principles are applied as in those analyses, which state that Sybil regions are connected to honest regions via a few honest entities.

If an entity is not a Sybil, it should be able to demonstrate multiple different paths between trusted entities. Such a proof can be made with Incrementally Verifiable Computation (IVC) [88] between entities.

The main idea is that each entity holds connections with other entities. Connections represent **endorsements** from other entities, and they are always mutual. These connections are never revealed to the public, but are committed on a public blockchain. Entities can use these commitments to prove connections to other entities using ZKPs. An important thing is that **the graph of the network is never stored**. Each entity knows only its direct connections.

Creating paths is quite simple. The process is illustrated in [Figure 6.1](#). Entity A is connected to entity B, and B is connected to C. Entity A begins creating a new path by generating proof of its connection to B and sending it to B. B verifies this proof, extends it the same way, and sends it to entity C. Entity C now holds proof of a path of length two. With such **recursive proofs**, entities can prove paths in the graph.

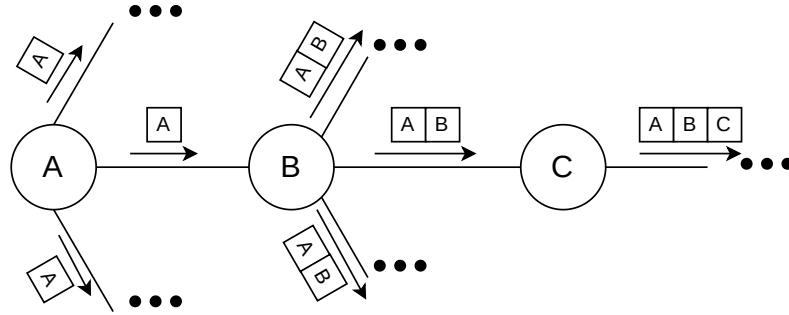


Figure 6.1: An example of creating a path through connected entities.

This approach addresses the potential leakage of entities' social connections. However, **there are still three main concerns**: how to determine that the connection has been revoked and that the path remains invalid, how to define honest entities, and how to ensure that each entity appears only once in a path while demonstrating multiple distinct paths with disjoint entities.

Connection Revocation

Connections can be revoked, which also means that paths built on those connections may become invalid. Retrospective validity checks on connections are not possible because

references to connections would reveal the graph structure. The only solution is to introduce a time limit on path validity. **Paths must thus be repeatedly proven to remain valid.**

Reputation on Paths

Having a path in the social graph is not enough to prove that an entity is not a Sybil. An attacker can mimic a graph's structure and create as many entities and paths as they want. Therefore, **there has to be some aspect of trust** on the path. It can be a known, trusted entity, or there can be a separate, decentralized reputation system for entities, and each entity along the path adds its reputation. For now, let us assume that each entity has a *reputation score*, and the methods for obtaining this score will be explained in [Section 6.5.4](#).

Reputation can change over time. So there is a similar issue here to that of connection validity. In other words, routes must be repeatedly verified, also for the sake of reputation.

Uniqueness of Entities on the Path

For this design to work, each entity can appear on a path only once. However, this creates a conflict, as storing a list of entities alongside the path would reveal connections between them, even if they are pseudonymized.

To solve this problem, a **Bloom filter** is used instead of a list of entities. Whenever a new entity is added to the path, its identifier is inserted into the Bloom filter for that path. If the entity is already in the Bloom filter, it cannot extend the path. The disadvantage is that false positives may occur; that is, even if the entity is not yet on the path, the algorithm may still evaluate it as if it were.

Additionally, proving a single path is not sufficient. Entities must prove multiple paths that do not share a common entity, because that shared entity can be a single attack edge. It is possible to verify whether two paths do not share a common entity using Bloom filters. The specific method will be explained in [Section 6.5.2](#).

6.2.3 System Actors

The proposed system includes the following actors, whose interactions are illustrated in [Figure 6.2](#):

- **Entities:** Users of the system. There are no special roles, so all entities are equal.
- **Oasis Sapphire:** Confidential smart contract layer that holds encrypted state.
- **Oasis ROFL:** Stateless, verifiable compute layer for heavy computation tasks, such as biometric verification.
- **Oasis Emerald or other Blockchain:** Public blockchain used to store commitments in a Merkle tree.
- **Database:** Stores encrypted embeddings of entities' facial biometric data.
- **Relay server:** Listens for events from Sapphire, and can batch commitments to the blockchain.

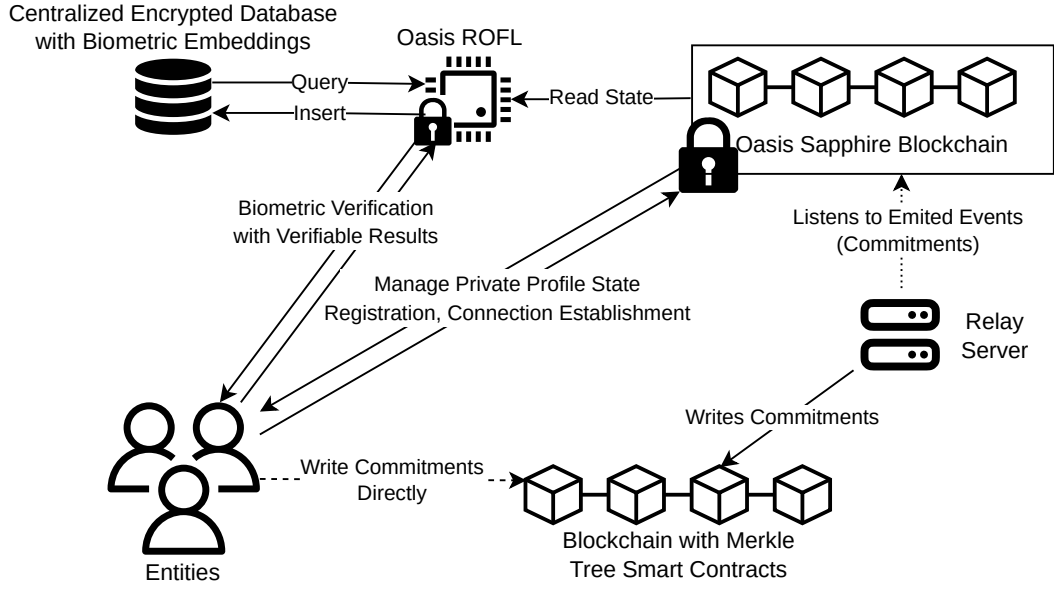


Figure 6.2: Actors of the proposed Sybil-resistant system and their interactions.

Oasis Network

Oasis Network [61, 62] is the most important component for biometric verification. It is a Layer 1 decentralized blockchain solution that features TEEs for private smart contract execution. The architecture consists of two main layers: the consensus layer and the ParaTime layer. The consensus layer uses PoS consensus and is similar to other blockchains. The ParaTime layer consists of **ParaTimes**, which are parallel execution environments that plug into the shared consensus layer. Each ParaTime has its own executor committee that performs the computation, and verifier nodes that verify correctness. The results of executions in ParaTimes are committed back to the consensus layer.

The most important ParaTime for the proposed system is the **Sapphire**, which can execute private smart contracts within a TEE and maintain a shared replicated encrypted state. Additionally, it is possible to use **Emerald** ParaTime, which offers standard, non-confidential smart contracts.

ROFL (Runtime Off-chain Logic Framework) is a framework on the Oasis Network for running off-chain computation that is still verifiable and optionally confidential via TEEs. It is designed for heavy computational tasks that are impractical to execute on-chain. The execution produces a result and attestation that can be verified on-chain. ROFL does not maintain a blockchain state, but it can interact with external data sources.

6.3 Registration

The registration process is illustrated in Figure 6.3. It is handled through Oasis ROFL and Oasis Sapphire in two steps:

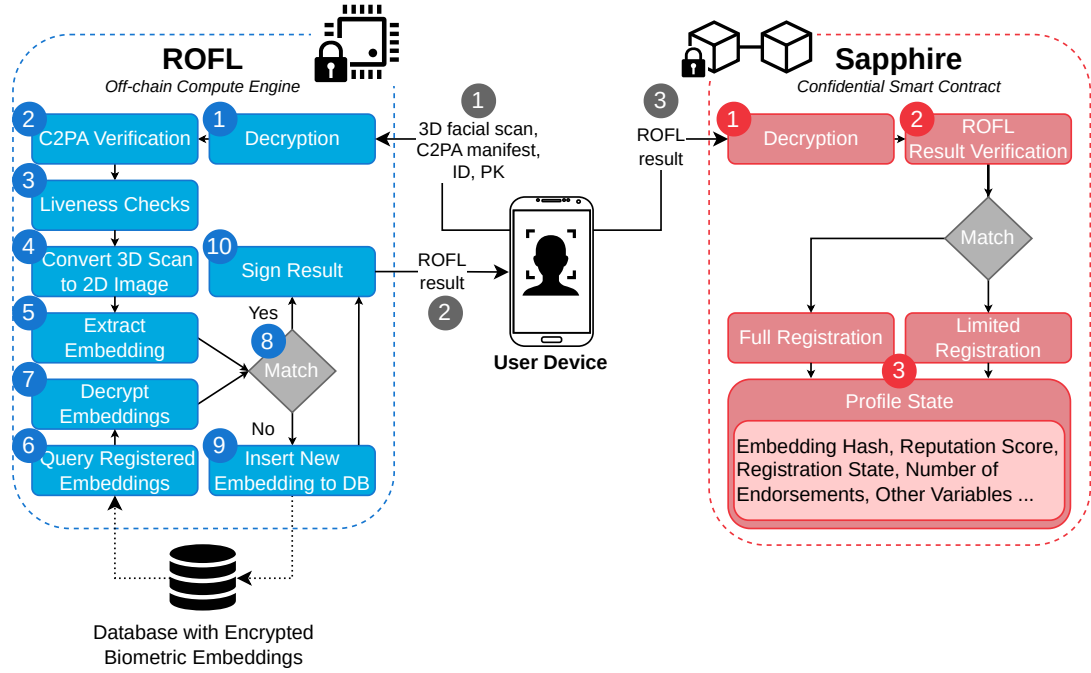


Figure 6.3: Design of the profile registration process.

1. **ROFL Face Biometric Verification:** ROFL verifies the new user's 3D facial scan and compares it against the database of registered users.
2. **Sapphire Profile Registration:** User uses the signed result from ROFL to create a profile on Oasis Sapphire.

6.3.1 ROFL Face Biometric Verification

The user generates a private (SK) and public (PK) key using their digital wallet. Using the camera on their mobile phone, they create a 3D facial scan with metadata that includes a C2PA manifest, the app used to create the scan, and a timestamp. Then the user sends the signed and encrypted PK , and the 3D scan, along with the C2PA manifest, to ROFL for verification.

ROFL processes the **input:** PK , 3D facial scan and produces the **output:** signed ID, facial embedding hash, timestamp, verification result in the following manner:

1. Decrypts the input (the user's request).
2. Verifies before processing:
 - The user's signature to verify the authenticity of their PK and prove they hold the SK .
 - The C2PA signature of the 3D scan to ensure that it has not been altered **since it was signed**. At the same time, requiring a valid C2PA manifest prevents the use of online 3D facial scans.

- That the 3D scan was performed by an authorized application. These must be trusted applications that best ensure the 3D scan is not altered before it is signed.
 - That the timestamp in the C2PA manifest is still valid (for example, no older than one hour).
3. Performs a **liveness check** and other possible checks to detect fraudulent alterations on the 3D scan.
 4. Converts a 3D scan into a synthetic 2D image.
 5. Extracts facial embedding from the 2D image and computes a hash of the embedding. Because embeddings are represented as floating-point numbers, inaccuracies can arise when **the same** embedding is repeatedly extracted from a 2D image. For this reason, the embedding is normalized before hashing, for example, by multiplying it by a large number, so that hashing the same embedding always produces the same hash. Note that different 2D images, even of the same person, yield distinct embeddings, so the hashes differ as well. The hash embedding from the registration is used later when establishing connections. The user provides the embedding from the registration, and this hash is used to verify that it is indeed the embedding from the registration.
 6. Makes an HTTPS call to an external database to query registered embeddings.
 7. Decrypts the queried embeddings.
 8. Performs a match against currently registered users.
 9. If no match is found in the database, ROFL adds a new embedding to the database. If the comparison matches another embedding already stored in the database, then **the user must obtain endorsements** from already registered users to complete the registration. By earning endorsements, the user confirms their authenticity. This mechanism is necessary so that, in the event of a truly random match of embeddings, the user can be registered. This also prevents the database from becoming overloaded with embeddings from users who did not successfully complete registration.
 10. ROFL uses its private key to sign the result and returns it to the user. The result consists of the user *ID*, **which is the wallet address derived from *PK***, the embedding hash, the timestamp (from the C2PA manifest), and the matching result. Additionally, ROFL returns the embedding, so the user can store it locally.

C2PA Manifest

C2PA manifest ensures that the media was not altered **since it was signed**. However, it does not ensure that the media was not altered before signing. There is a critical phase between taking the photo, processing it, and signing it. To a large extent, this depends on phone manufacturers and the capabilities of trusted hardware and cameras in their devices, ensuring that the created media cannot be tampered with before it reaches the application. The application that processes the media must also be trusted, as there is still a risk that the C2PA manifest could be removed from the media's metadata and replaced with a new one.

Embeddings Database

Face embeddings are stored in encrypted form in a centralized database. The user ID (**wallet address derived from PK**) serves as the primary key to the individual embeddings; however, it is salted by a secret S_{ROFL} , so the primary key to the database embeddings is $\text{key} = H(ID \parallel S_{ROFL})$. This ensures that it is impossible to determine which encrypted embedding belongs to which user, even for the database maintainer.

As mentioned, for better scalability, it is possible to use an Inverted File Index (IVF) and tag the embeddings so that, during recognition, it is not necessary to match against all stored embeddings. Tags can reveal information, since similar embeddings will be in the same cluster, indicating that people with those tags likely have similar facial features. However, this is not a major privacy concern.

Key Management

Oasis has a built-in Key Management Service (KMS). It can generate a symmetric key (K_{DB}) for encrypting and decrypting embeddings and encrypt it for approved ROFL instances with their TEE public keys. Individual ROFL instances can then ask KMS for the K_{DB} to decrypt the embeddings. No human ever sees the K_{DB} . It is generated and used within TEE and securely stored by KMS.

ROFL Authentication to the Database

Only Oasis ROFL should have access to the database. This can be done using the **Remote Attestation quote**. Upon startup, the ROFL agent generates an ephemeral cryptographic key pair within the TEE. The private key is used to self-sign a certificate with the public key and the Remote Attestation quote. This quote provides signed information about the TEE and the program running in it. When connecting to the database, the ROFL presents this certificate, and the database must be configured to verify it. If the presented certificate and quote are valid, ROFL gets access to the database.

6.3.2 Sapphire Profile Registration

The user uses the signed result from ROFL to create a profile on Oasis Sapphire. A smart contract on the Sapphire process the **input (parameters)**: *signed ROFL result* and registers the user in the following manner:

1. The smart contract on Sapphire decrypts the input (registration parameters).
2. Verifies:
 - The signature from ROFL to ensure that biometric verification was done by trusted hardware.
 - That the timestamp in the ROFL result is still valid (for example, no older than one hour).
 - Whether the embedding hash has ever been used before (the smart contract holds a list of registered embedding hashes).
3. The smart contract registers the user by storing the embedding hash, setting the reputation score, and recording the registration state, either **fully registered** or

limited registration (waiting for endorsements). There may also be other variables, for example, the number of endorsements gained and provided, as this number is limited.

4. **Once a user has obtained a sufficient number of endorsements** (e.g., 5), they can submit a request to ROFL, which retrieves and verifies the number of endorsements on Sapphire. If successful, it inserts the embedding into the database and returns a signed result containing the user *ID*, the embedding hash, the timestamp, and the computation result.
5. The user can present this result on Sapphire to complete the registration.

Endorsements

This mechanism is necessary because facial recognition is not perfect. In the event of a truly random match between multiple embeddings of different users, there has to be a way to allow all users to be registered, even if they have to take an extra step.

An endorsement is something extra beyond the standard connection establishment described in [Section 6.4](#). The difference is that the Sapphire smart contract tracks how many endorsements each user has received and how many they have granted. To prevent users from distributing endorsements to just anyone, **each user can give only one endorsement**. This is because facial biometrics can distinguish most users, and it is not anticipated that the need for endorsements will be too great. Users are thus motivated to save this endorsement in case they need it for a close friend.

Full and Limited Registration

A user without full registration cannot participate in creating paths in the social graph. However, they can still establish connections with other users. Since the smart contract's state is private and accessible only to the TEE, this contract can generate and store a private key and generate signatures. Upon request, a digitally signed claim from the contract can be obtained stating that the user has full registration.

6.4 Connections Establishment

Users with full or limited registration can create private mutual connections. No one can find out whether the two entities are connected. There can be only one connection between two entities. The process of connection establishment is illustrated in [Figure 6.4](#) and is performed in three steps:

1. **In-person Meeting:** Two people will meet to take a photo of each other and share information about their profiles.
2. **ROFL Mutual Verification:** Both entities use ROFL to verify that the embedding extracted from photos they have taken matches the other's embedding from the registration. In this way, both entities confirm each other's identities. **If one entity used a sophisticated mask or otherwise deceived during the registration, the other entity would notice it in the face-to-face meeting.**

3. **Sapphire Connection Registration:** If both profiles are verified, connection registration can take place via a smart contract on Oasis Sapphire, which ensures the uniqueness of the connection.

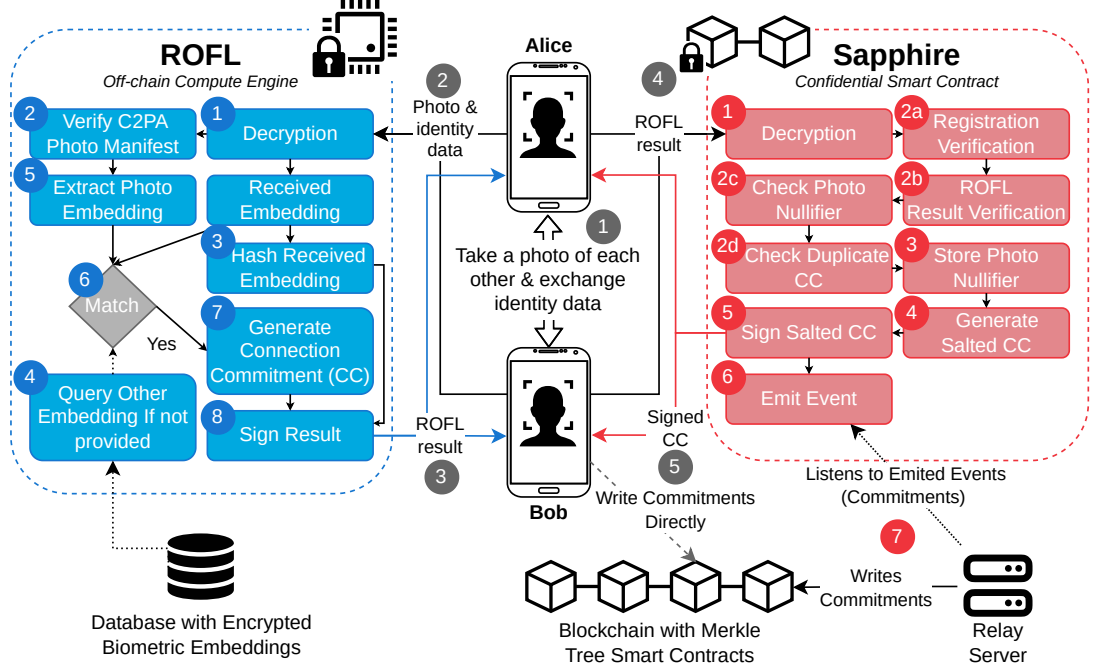


Figure 6.4: Connection establishment process between two entities.

6.4.1 In-person Meeting

During the in-person meeting, entities A and B take a face photo of each other (A takes P_B , B takes P_A), with the C2PA manifest. To complete the setup, A and B exchange the stored embeddings (E_A^s , E_B^s) obtained from the registration process, their wallet addresses (ID_A , ID_B) derived from their public keys. After exchanging data, both A and B send the signed, encrypted data to Oasis ROFL for verification.

6.4.2 ROFL Mutual Verification

Verification using ROFL proceeds in the same way for both entities. This is how it looks from the perspective of entity A with signed **input**: PK_A , ID_B , P_B , E_B^s :

1. Decrypts the input (request from A).
2. Verifies:
 - The A 's signature on the provided input to verify the authenticity of PK_A and prove they hold SK_A .

- The C2PA signature of the P_B . This verifies that the P_B has not been altered since it was signed.
 - That the P_B was taken by an authorized application based on the C2PA manifest.
 - That the timestamp (t_B) in the C2PA manifest is still valid (for example, no older than 10 minutes).
3. Computes the hash H_B^s of normalized E_B^s .
 4. If E_B^s was not provided, ROFL can fetch it from the embeddings database using ID_B .
 5. Extracts the embedding (E_B^c) and hash of the P_B (H_B^P).
 6. Matches the E_B^c against the E_B^s . Because the connection is established in person, there is no need to verify the authenticity of P_B .
 7. If there is a match between embeddings E_B^c and E_B^s , ROFL computes ID_A from the PK_A and generates a connection commitment $CC_{AB} = H(\min(ID_A, ID_B) \parallel \max(ID_A, ID_B))$. Notice that the commitment should be the same for both entities.
 8. Creates a **signed** (Sig_{ROFL}^r) **result of computation containing** CC_{AB} , t_B , H_B^s , and H_B^P .

6.4.3 Sapphire Connection Registration

Having the result from ROFL, both entities call the smart contract function on Oasis Sapphire to register a connection. This is how it looks from the perspective of entity A . The smart contract process the **input**: Sig_{ROFL}^r , CC_{AB} , t_B , H_B^s , H_B^P , ID_B , match result and registers the connection:

1. Decrypts the transaction parameters.
2. Verifies:
 - (a) That both A and B are registered.
 - (b) The validity of Sig_{ROFL}^r and that H_B^s is a registered embedding of B .
 - (c) The validity of the t_B (for example, no older than 10 minutes) and that the H_B^P was not used before.
 - (d) The CC_{AB} is not registered yet.
3. If all checks pass, Sapphire stores H_B^P for the duration of its validity, t_B , to prevent reuse.
4. If both entities pass successfully, the smart contract registers their connection and generates a random secret (S_{AB}^{CC}) for it, which serves as the salt for CC_{AB} . The salted commitment is $CC_{AB}^s = H(CC_{AB} \parallel S_{AB}^{CC})$.
5. The smart contract signs the salted CC_{AB}^s . On a standard blockchain, a smart contract cannot perform such a signature because the private key cannot be hidden. However, on Sapphire, the state is private and accessible only to the TEE, so a smart contract can generate a key pair while keeping the private key hidden. The Oasis Solidity libraries directly support this.

6. Sapphire emits a registration event with signed CC_{AB}^s , $Sig_{Sapphire}^{CC_{AB}^s}$.
7. A relay listens for these events and inserts them into an on-chain Merkle tree, enabling these commitments to be proven via ZKPs.

Any user can request their CC_{AB}^s and S_{AB}^{CC} from the smart contract and directly insert their commitment into an on-chain Merkle tree if the relay fails, but the smart contract must verify that the user requesting this is involved in the connection. The smart contract with the Merkle tree **must verify** the $Sig_{Sapphire}^{CC_{AB}^s}$ to prevent anyone from adding connection commitments.

Connection Revocation

Entities should be able to revoke established connections. However, deleting a connection commitment from the Merkle tree is a very resource-intensive operation that would also disrupt the tree structure for other commitments. Therefore, a second Merkle tree is used to store nullifiers for established commitments.

When proving a connection commitment, it is also necessary to prove that there is no nullifier for the connection commitment in this Merkle tree. However, proving non-inclusion in a standard Merkle tree is not possible. Therefore, a special Sparse Merkle Tree (SMT) is used, which makes this possible.

If A wants to revoke the connection with B , they use the connection revocation function of the smart contract on Sapphire, providing the following **inputs**: CC_{AB} , ID_B . Then the smart contract:

1. Verifies that CC_{AB} exists and that the sender of this transaction is part of this connection, i.e., $CC_{AB} = H(H(\min(ID_A, ID_B) \parallel \max(ID_A, ID_B)))$.
2. Using the stored S_{AB}^{CC} , it creates a nullifier for the connection that does not match the connection commitment, for example, $N_{AB} = H(CC_{AB} \parallel S_{AB}^{CC} \parallel S_{AB}^{CC})$.
3. The smart contract deletes the record of this connection from its state and emits an event with the signed N_{AB} , $Sig_{Sapphire}^{N_{AB}}$.
4. A relay listens for these events and inserts them into an on-chain **SMT**.

In the event that the relay fails, A still knows N_{AB} and $Sig_{Sapphire}^{N_{AB}}$, so it can directly insert N_{AB} into the SMT. Again, the smart contract with SMT must verify the $Sig_{Sapphire}^{N_{AB}}$ to ensure that no one can revoke existing connection commitments.

Merkle Tree and Sparse Merkle Tree

When proving connections using ZKPs, the question arises: why should we include connection commitments in a Merkle tree and use a **Merkle Inclusion Proof** instead of a simple digital signature from a smart contract on Sapphire that proves the connection's authenticity? There are two reasons for this:

1. A digital signature is impossible to revoke.
2. Verifying a Merkle Inclusion Proof in a ZKP circuit is computationally easier than verifying a digital signature.

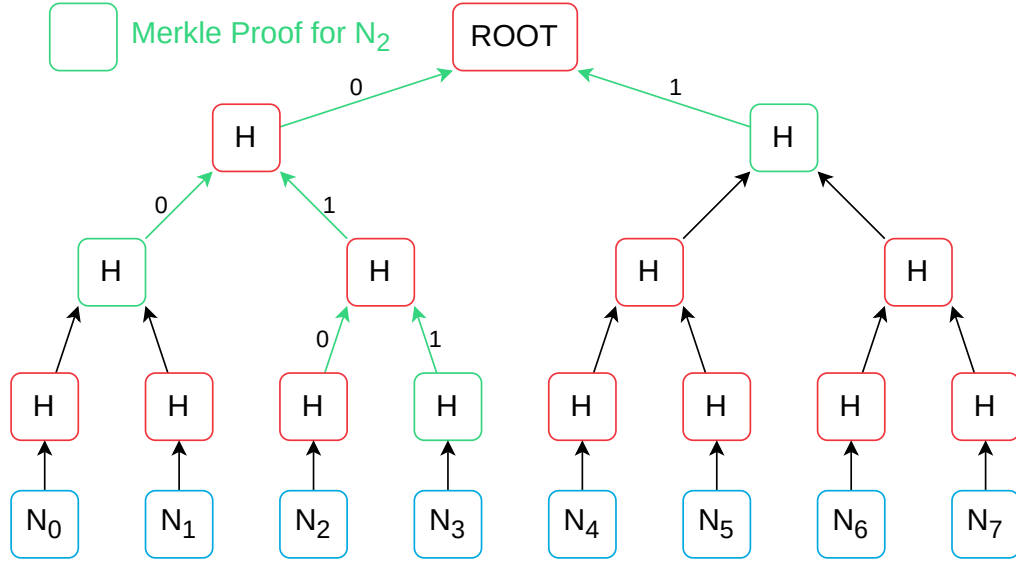


Figure 6.5: Example of a Merkle Inclusion Proof [33].

Merkle Tree (MT) was already described in Section 2.1.2. With this data structure, it is possible to prove that a specific leaf is present in the tree. To provide this proof, it is necessary to have all sibling nodes of the tree, from the leaf to the root, and a 0/1 bit array indicating whether each node is a right or left sibling, so that the hashing order is correct. Such a proof is called a **Merkle Inclusion Proof (MIP)**. An illustration of this proof is shown in Figure 6.5 [33].

Sparse Merkle Tree (SMT) [33] works exactly like a standard Merkle tree. The only difference is that the leaves are indexed, and from the very beginning, the tree contains all leaves that have a special **null** value. New values are inserted at the corresponding indices. Typically, the tree has 2^{256} leaves, so its depth is 256. For a hash function with a 256-bit output, each hash has its own index.

To make **Merkle Non-Inclusion Proof (MNIP)** in the SMT, one must prove that the index of that value contains the **null** value. This is an MIP, which always has 254 siblings and a 254-bit array indicating whether each sibling is a right or left sibling. An example of an SMT with a single nullifier and $2^{256} - 1$ **null** leaves is shown in Figure 6.6. Note that many siblings can be precomputed since the tree is initially empty.

The disadvantage of SMTs is that they are very large; however, most of the tree consists of **null** values for which the values of parent nodes in the tree can be precomputed, so it is not necessary to store the entire tree.

Alternative without Profile Registration

The process of establishing connections, which forms the basis for social graph analysis, can be performed even without profile registration or mutual biometric verification. This significantly simplifies establishing connections, but it raises a problem with their revocation.

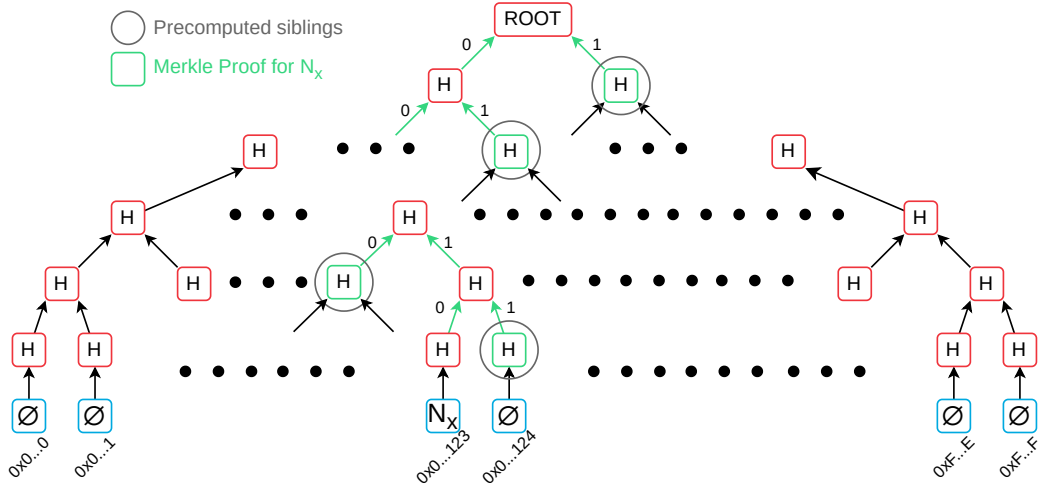


Figure 6.6: Example of a Sparse Merkle Tree with a single inserted value.

Entities can establish a connection commitment by creating a shared secret S_{AB}^{CC} . Such a secret can be generated using the Elliptic Curve Diffie-Hellman (ECDH). Subsequently, A and B can create the same connection commitment $CC_{AB} = H(\min(ID_A, ID_B) \parallel \max(ID_A, ID_B) \parallel S_{AB}^{CC})$. This CC_{AB} is inserted into the on-chain Merkle tree, and the rest of the process is identical.

To prevent multiple connections between two entities, the method used to generate the secret for A and B must always return the same result. ECDH satisfies this requirement, but one problem arises here. Once a connection is revoked, it cannot be re-established because the nullifier will be permanently stored in the SMT.

This problem is solved by a smart contract on Sapphire that always generates a different secret, even after multiple revocations and connection re-establishment. Otherwise, this alternative does not require any checks on insertion into the Merkle tree and SMT.

6.5 Social Graph Analysis

Registered entities that have established connections with one another can demonstrate their membership in a broader network by creating distinct paths among themselves. However, the paths alone do not prove whether they belong to the honest or Sybil region; therefore, an external reputation source is also required. Possible sources of this external reputation will be discussed in [Section 6.5.4](#). Proof of strong connections to the broader trusted network demonstrates that the entity is not a Sybil. Conversely, Sybil entities have few connections to the trusted part of the network and therefore struggle to establish such paths.

Proving a connection to the broader network involves two steps:

1. **ZK Random Walks:** Entities share ZKPs of their connections. These proofs are recursively used by other entities, thereby creating paths in the graph. Each entity on the path contributes its reputation R_{entity} to the path, which gives the path its own reputation $pathR$.

2. **ZK Proof of Paths:** An entity generates a single final ZKP that proves multiple **distinct paths**. The distinctness of paths is defined such that each path passes through a **set** of entities, and these sets **must be disjoint**. This assumption ensures that the Sybil region must be connected via multiple honest nodes.

6.5.1 Reputation Commitment and Epochs

Before diving into the details about path proving, this process requires a method for proving a reputation score. The reputation score value is part of the profile in the smart contract on Sapphire. This is important because the score is kept private and provable. To prove this score, a commitment must be created in the public on-chain Merkle tree, just as with connection commitments. An entity (A) requests the creation of a reputation commitment (RC_A); the smart contract on Sapphire creates it by salting (S_A^R) the actual reputation value (R_A), bounding the commitment to the entity $RC_A = H(ID_A \parallel R_A \parallel S_A^R)$, signing it, and emitting an event that a relayer listens to, which inserts it into the on-chain Merkle tree.

However, a single reputation commitment is not enough, as the reputation value may change. The same applies to connection commitments, which can be revoked. That is why there is a system of **epochs**. An epoch is a time period **lasting three to seven days**. This period is long enough for entities to establish paths while also maintaining the freshness of their reputations and connections.

Path proofs are tied to a specific epoch, and **proofs from different epochs cannot be combined**. The final path proof also includes the epoch in which the paths were proven. If an entity's reputation changes or a connection is revoked, this change will not take effect until the next epoch.

In the following sections, it will be explained that entities' reputations change with every epoch. Therefore, if an entity wants to prove paths in an epoch, it must always create a new reputation commitment. At the same time, the on-chain Merkle tree containing reputation commitments can be deleted and restarted from scratch every epoch.

6.5.2 ZK Random Walks

Each entity begins by constructing a path starting from itself and sends the proof to its connections, which then use it **recursively** in subsequent proofs. Such recursive proofs ultimately form a proof of a path through the social network. Proofs are bound to specific entities so that other entities cannot replay them. The following example describes a proof-of-path in which A receives a proof from X and produces a proof for B . To prevent the same entity appearing repeatedly on a single path, each entity that the path passes through adds its nullifier $N_{entity} = H(ID_{entity} \parallel epoch)$ to the Bloom filter (BF). The details of how the BF ensures uniqueness of entities on the path are explained in [Section 6.5.2](#) under the **Path Distinctiveness** subsection. If this is still confusing, think of BF as an array of nullifiers, where the ZK circuit ensures that each nullifier is correctly added to the array in the resulting proof and that there are no duplicates.

The ZKP has these **inputs** and **outputs**:

- **Public Inputs:** π_{prev} (recursive ZKP from a connection), connection and reputation MT root, connection revocation SMT root, L_{old} (accumulated path length so far), $pathR_{old}$ (accumulated path reputation so far), $epoch$, BF_{old} (the current bit-array)

state of the Bloom filter), $Dest_{old} = H(ID_X \parallel ID_A \parallel S_{XA}^{CC} \parallel epoch)$ (commitment specifying who is allowed to generate the next proof).

- **Private Inputs (Witnesses):** $ID_X, ID_A, SK_A, ID_B, CC_{XA}^s, CC_{AB}^s, S_{AB}^{CC}, R_A, S_A^R$, connection MIP, connection revocation MNIP, reputation MIP.
- **Public Outputs:** π_{new} (newly generated ZKP), $L_{new}, pathR_{new}, BF_{new}$ (updated Bloom filter bit-array with A 's bits flipped to 1), $Dest_{new} = H(ID_A \parallel ID_B \parallel S_{AB}^{CC} \parallel epoch)$ (commitment to the next entity in the path).

With these inputs and outputs, the ZKP must prove the following inside a circuit:

- π_{prev} with path length L_{old} and reputation $pathR_{old}$ is valid.
- A is the intended recipient of the π_{prev} by verifying that $Dest_{old} = H(ID_X \parallel ID_A \parallel S_{XA}^{CC} \parallel epoch)$. Note that S_{XA}^{CC} is only known to X and A . Therefore, if $Dest_{old}$ was wired by X , only A can use this proof.
- The $epoch$ of π_{prev} is the same as of the new proof.
- The connection and reputation MT root and connection revocation SMT root of the π_{prev} are the same as those of the new proof. This ensures that the roots of the MT and SMT remain the same, and that the final proof can verify that all intermediate proofs used the existing MT and SMT roots. The roots used are always from the beginning of the $epoch$.
- A has an unrevoked connection to the X by proving knowledge of S_{XA}^{CC} , so the leaf $CC_{XA}^s = H(H(\min(ID_X, ID_A), \max(ID_X, ID_A)) \parallel S_{XA}^{CC})$, proving that connection MIP results in connection MT root, proving that connection revocation MNIP is for the connection nullifier $N_{XA} = H(CC_{XA}^s \parallel S_{XA}^{CC})$, and the MNIP results in connection revocation SMT.
- A possesses a publicly committed reputation score R_A by proving $RC_A = H(ID_A \parallel R_A \parallel S_A^R)$ and reputation MIP results in reputation MT root.
- $N_A = H(ID_A \parallel epoch)$ is not already in BF_{old} , and A has correctly added N_A to the path's BF_{new} .
- A correctly calculated the new path length L_{new} , path reputation $pathR_{new}$, and securely locked the proof for B with $Dest_{new} = H(ID_A \parallel ID_B \parallel S_{AB}^{CC} \parallel epoch)$.

The first proof in the path is slightly different because there is no prior proof. $epoch$ is added to the Bloom filter nullifier and $Dest$ commitment, so they are different each in every epoch. This similarity could enable the detection of recurring patterns in these values and the deanonymization of connections.

The destination commitment $Dest = H(ID_{prover} \parallel ID_{dest} \parallel S_{prover,dest}^{CC} \parallel epoch)$ binds ID_{prover} as the proof provider and ID_{dest} as the proof receiver by including $S_{prover,dest}^{CC}$ as the specific connection for which this proof was intended. Note that even if the connection secret $S_{prover,dest}^{CC}$ is leaked, a malicious entity cannot use this proof because it is specifically bound to ID_{prover} and ID_{dest} .

When constructing the **final proof**, every entity that wishes to use a particular path designates itself in the $Dest$ and may choose any secret S_{entity} , i.e., $Dest = H(ID_{entity} \parallel$

$ID_{entity} \parallel S_{entity} \parallel epoch$). The secret and double ID_{entity} have no specific purpose here; they are only there to ensure that the commitment calculation remains the same, so the circuit does not have to be changed.

Path Sharing

The diagram in [Figure 6.7](#) illustrates the gradual creation of paths between four entities. The way the entities are connected is shown in the diagram on the right. Entities cannot see this graph; they are only aware of their direct connections. The edges indicate the path that an entity follows. For example, $[A, B, C]$ denotes a path starting with entity A , passing through entity B , and ending with entity C .

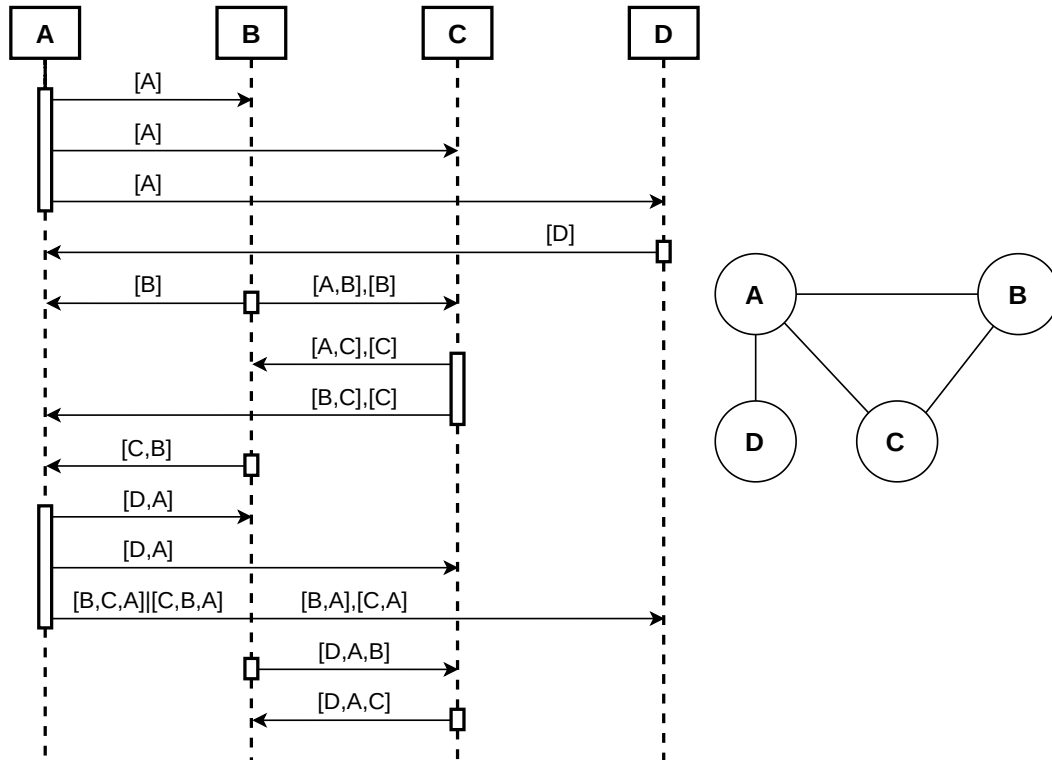


Figure 6.7: Zero-knowledge random walks in the social network.

A initiates a path and sends a proof to B . B extends this proof by itself and sends the path $[A, B]$ to C . All entities repeat this process, regardless of the order. Entities can wake up at any time, use the paths they have received, extend them, and send them to other entities.

Entities do not share paths if it turns out that their target entity cannot use them, because the nullifier for that entity is already present in the Bloom filter. Even if it were a false positive, which can occur in a Bloom filter.

Many such paths are created in the network, and many of them pass through the same entities. In the example in [Figure 6.7](#), this is evident when A sends the paths $[B, C, A]$ and $[C, B, A]$ to entity D . In this specific case, if the paths are identical, which can be determined

using the same Bloom filters, it is sufficient to propagate only one path, specifically the one with the higher reputation $pathR$. Otherwise, entities must share all paths to provide the best possible options for creating paths for other entities.

The best way to share path proofs would be P2P. In practice, however, P2P communication is problematic because entities would have to be constantly online, which is not feasible. An alternative is to use a *storage service* that allows entities to store their proofs so that other entities can access them at their convenience. However, a centralized service can determine which entities are communicating with each other and are therefore likely to be connected.

Length of Paths

According to the algorithms for analyzing Sybil nodes in a social graph discussed in [Section 5.4.1](#), the path length should be $\mathcal{O}(\log n)$, where n is the number of nodes in the network. This means that the length of paths needed grows no faster than some constant multiple of $\log n$. It is also related to small-world theory, where relatively short paths are usually sufficient to connect all entities to one another. Note that the logarithm base is not relevant to the big $\mathcal{O}$ notation, because the difference between two logarithms with different bases is just a constant multiplier. The required path length can be determined by a smart contract on Sapphire that provides a count of fully registered users.

Path Reputation

The reputation of the path is calculated as:

$$pathR_{new} = pathR_{old} \times \alpha + R_A \times w_A^B$$

, where:

- $pathR_{old}$ is the previous path reputation. The default value for a new path is 0.
- $\alpha \in \langle 0, 1 \rangle$ is a decay factor, so the reputation of preceding entities along the path is gradually decreasing.
- R_A is reputation committed and proven by entity A .
- $w_A^B \in \langle 0, 1 \rangle$ is the weight assigned by entity A to entity B . This value indicates the degree of trust that entity A places in entity B . A may change this value at any time.

Path Distinctiveness

Nodes must not be repeated along the path to prevent the formation of a cycle in which an entity could add its reputation to the path multiple times. To achieve this, a Bloom filter is used, and each entity on the path adds its nullifier to it, which is verified via a ZKP.

A Bloom filter [\[44\]](#) is a probabilistic data structure that consists of a bit array of size m . Initially, all the bits in the array are set to 0. One key feature of a Bloom filter is that it can always accurately confirm whether an item has been inserted into the filter. However, it may also indicate that an item is present in the filter when, in fact, it has not been inserted; this situation is known as a **false positive**.

An item is inserted into the filter by hashing it j times with hash functions; each hash is then modulo n , yielding n indices into the bit array. A value of 1 is set at each of these indices.

Verifying whether an item is present in the Bloom filter proceeds in exactly the same way. If all the obtained indices contain the value 1, the item is considered present in the filter. Due to collisions among set bits, false positives may occur. Furthermore, inserted items cannot be removed.

Inserting into the filter and the membership check is shown in Figure 6.8. Nullifiers 1 and 2 are inserted into the filter, and the corresponding indices are set to 1. There may be collisions, such as at index 5. When verifying whether nullifier 3 is in the filter, a false positive occurs. In the context of the ZKP circuit, this means that an entity with nullifier 3 cannot extend the path, even though it is not yet on the path.

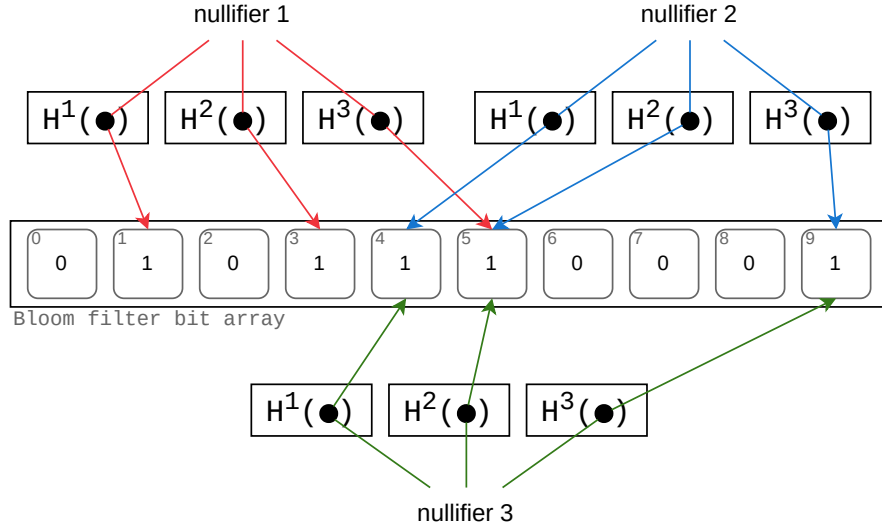


Figure 6.8: Illustration of the insertion and membership check operations of a Bloom filter [44].

The reason for using a Bloom filter instead of a simple array of nullifiers is to preserve the privacy of the connections in the graph. For example, if three entities A, B, C are connected in a cycle $A \rightarrow B \rightarrow C \rightarrow A$, then when the path $A \rightarrow B \rightarrow C$ is created, C realizes that A and B must be connected because it sees their nullifiers in the array. This problem also arises with larger cycles. Additionally, it is possible to observe which nullifiers recur most frequently in a given entity's paths, thereby revealing the structure of its connections. **Entities cannot use nullifiers with different salt for each path**, because it would be impossible to distinguish whether multiple proven paths are distinct. A Bloom filter hides who is on the path because the set bits do not reveal the nullifiers, and it can also produce false positives; the more entities are added, the harder this becomes.

However, for this Bloom filter to be effective, m and j must be set correctly. There is still an effort to minimize false positives, and this must be done in a way that keeps their rate as low as possible, even across the k paths. This is because the entities want to prove that they have k paths that do not share the same entity.

According to Christensen et al. [23], the probability of false positives for q inserted items is:

$$p_{false}(q, m, j) = \frac{m!}{m^{j(q+1)}} \sum_{i=1}^m \sum_{l=1}^i (-1)^{i-l} \frac{l^{jq} \times i^j}{(m-i)! \times l! \times (i-l)!}$$

However, minimizing this function is very difficult, so an approximation of this function is used: $p_{false} \approx (1 - e^{-jq/m})^j$. Minimizing this approximated function with respect to j yields an optimal value of j : $j = \lceil \frac{m}{q} \ln(2) \rceil$. The value of j is rounded, since it must be an integer. $q = \lceil \log n \rceil$, which corresponds to the path length. Now all that remains is to choose m based on the acceptable value of p_{false} : $m = \lceil -\frac{n \ln p_{false}}{(\ln 2)^2} \rceil$.

In the final verification, an entity has to prove k paths that do not share any entity except the entity that proves them. This can be achieved using Bloom filters by performing an AND operation on each pair of filters to compute their intersection. Paths always share one entity, so there are at least j common bits in the intersection. The expected number of bits in the intersection with j bits that are definitely present is Expected Bits $\approx j + (m-j)(\frac{j \times q}{m})^2$. If there are $2j$ common bits in the intersection, this could indicate two entities on the same path, or it could be a false positive. To minimize false positives, it is necessary to set m very large, which results in a very sparse Bloom filter; however, at a minimum, $j \leq \text{Expected Bits} < 2j$ should hold. After substituting Expected Bits and simplifying the inequality, $m > j \times q^2$.

In the event of a collision in the hash functions, **there may also be cases where there are fewer than $2j$ bits in the intersection, but the same entity, except the entity that is making the proof, is present in both paths.** This is a false negative, which poses a major problem, but the probability of it occurring is very low for a sparse Bloom filter and can be accepted. The Bloom filter size is the same for all entities. This prevents cheating and reliance on false positives. Furthermore, **entities cannot manipulate the Bloom filter's nullifier**, so they cannot influence the probability of false positives or false negatives.

The practical tests simulating this Bloom filter approach showed that for $Q = 10$, $K = 10$, and number of hashes $j = 3$, an expected false positive rate is $\approx 0.05\%$ and false negative rate is $\approx 0.01\%$ for a filter of size $m = 4096$ bits. With lower Q and K , the false-positive and false-negative rates are even lower. These rates look reasonable, and even a Bloom filter of size 4096 bits is usable. The simulation was performed using a simple Python script, `bloom_filter_path.py`, which is part of the implementation.

Motivation for Creating Paths

Path generation is computationally intensive. Furthermore, extending a path does not benefit the entity that extended it. However, the game-theoretic principle known as the prisoners' dilemma works well here, specifically the repeated version. It is advantageous for entities to cooperate, as they can collectively prove that they are not Sybil. If an entity stops cooperating and expects the other entities to generate proofs for it, those entities may stop communicating with that entity. Just as in the repeated prisoners' dilemma, the best strategy is to copy the opponent's decision from the previous move, and the same applies in this system. If an entity cooperates and creates paths, the other entities will also create paths for it; otherwise, they will not cooperate.

In practice, entities do not need to prove they are not Sybils every 3 to 7 days; they only need to do so in certain situations. Therefore, entities are rewarded with intrinsic reputation R_I for regularly proving their paths. Reputation is thus divided into two components:

- External $R_E \in \langle 0, R_{max} \rangle$, which is derived from the reputation system.
- Intrinsic $R_I \in \langle 0, R_{max} \rangle$, which is a reward for proving paths.

The R_I recalculated each epoch as:

$$R_{I,new} = \beta \times R_{I,old} + (1 - \beta) \times reward(pathRs, k)$$

, where:

- β serves as a decay factor. If an entity stops producing new proofs, it will gradually lose R_I .
- $R_{I,old}$ is the R_I from the previous epoch. The default value is 0.
- $reward(pathRs, k) = \frac{\sum_k pathRs}{k \times R_{max}}$, where $pathRs$ is set of path reputations from k requested paths. It is an average path reputation from k proven paths.

The resulting reputation R_{entity} is a function of these two reputation components and the value $\gamma \in \langle 0, R_{max} \rangle$, which specifies the maximum amount of R_I that an entity with $R_E = 0$ can obtain:

$$R_{entity}(R_E, R_I, \gamma) = R_E + R_I \left(\frac{\gamma}{R_{max}} + \frac{R_{max} - \gamma}{R_{max} \times R_{max}} \times R_E \right)$$

The higher R_E an entity has, the more R_I it can gain, while always receiving the full R_E . This restriction is primarily intended to prevent Sybil entities from accumulating reputation while rewarding honest entities all the more.

An important question is how to set the threshold T for the minimum reputation of a path. Reputation is primarily intended to distinguish between Sybil and honest regions. Its role is not to fully ensure Sybil resistance, but to make it harder for Sybil entities to be perceived as honest. The basic assumption is that a path consisting solely of entities with $R_E = 0$ cannot be considered valid, hence

$$T_{min} > \gamma \times \sum_{i=0}^{\lceil \log n - 1 \rceil} \alpha^i$$

However, the chosen value of T at which paths are considered sufficiently trustworthy can be dynamic. Systems with varying degrees of sensitivity to Sybil attacks can determine the acceptable value of T themselves.

This is also related to the maximum reputation value that a path can acquire. This value can be determined by evaluating the equation:

$$pathR_{max} = (R_{max} + \frac{pathR_{max}}{R_{max}}) \times \sum_{i=0}^{\lceil \log n - 1 \rceil} \alpha^i$$

R_{max} is a constant that determines the maximum range of reputation. For example, if $R_{max} = 10$, $\alpha = 0.8$, $\beta = 0.7$, $\gamma = 2$, then both R_E and R_I take values from the interval $\langle 0, 10 \rangle$. However, this is only true if paths are infinitely long. In practice, paths have a length of $\log n$. With a path length of 8, the $pathR_{max} \approx 71.266$. This also means that $R_{I,max} \approx 7.1266$ and $T_{min} > 8.32228$.

6.5.3 ZK Proof of Paths

Once an entity has accumulated the required number of paths, it can create a ZKP that aggregates these proofs into a single one. The smart contract on Sapphire verifies this proof and determines whether the account is non-Sybil. In addition, it modifies the entity's intrinsic reputation R_I .

It would also be possible to gradually prove all paths of the required parameters, though this would be computationally more demanding for the smart contract. The ZKP has the following inputs:

- **Public Inputs:** ID_{entity} , $pathR_{total}$, L_{req} path proofs $epoch$, $[\pi_1, \dots, \pi_k]$, where k is the number of paths.
- **Private Inputs:** SK_{entity} , S_{entity} (secret from the last path proof), public inputs to all path proofs.

With these inputs, the ZKP must prove the following inside a circuit:

- Verify all proofs $[\pi_1, \dots, \pi_k]$ using their public inputs.
- $\sum_{i=1}^k pathR_i = pathR_{total}$.
- $\forall i \in \{1, \dots, k\} : L_i = L_{req}$.
- $\forall i \in \{1, \dots, k\} : Dest_i = H(ID_{entity} \parallel ID_{entity} \parallel S_{entity} \parallel epoch)$.
- Verify that connection MT root, reputation MT root, and revocation SMT root are the same for all proven paths. The $epoch$ must also be the same for all paths, but this is related to the MT and SMT roots.
- Verify that paths are distinct (they do not share a common entity). Circuit pairs the Bloom filters ($\frac{k(k-1)}{2}$ combinations) and performs a bitwise AND operation on each pair, then sums the resulting 1 bits and enforces that the sum is **strictly** $< 2j$, where j is the number of hashes per nullifier. The details of the process were discussed in [Section 6.5.2](#).

The $Dest$ in this case verifies that the final entity in the path was ID_{entity} . The smart contract verifying this final proof **must** confirm that ID_{entity} is the wallet address executing the transaction. This verifies the ownership of the final proof as no other entity can construct this proof without S_{entity}^R in the last path extension and set $Dest = H(ID_{entity} \parallel ID_{entity} \parallel S_{entity} \parallel epoch)$ in all paths.

Required Number of Paths

Determining the correct number of paths in a social network can be quite challenging. The social graph analysis algorithms discussed in [Section 5.4.1](#) do not offer a definitive answer. Each of these algorithms employs a slightly different approach. However, it appears that using $\mathcal{O}(\log n)$, where n represents the number of nodes in the network, is a reasonable estimate. This means that the number of paths needed to prove is not growing faster than some constant multiple of $\log n$. As the network grows, an attacker naturally finds more opportunities to gain more connections. If the required number of paths $k = \log n$ and the number of attacker connections to honest entities is g , then a **Sybil attack is not possible** as long as $k > g$.

6.5.4 Sources of Trust

Solutions that utilize the social graph require an external source of trust to distinguish between honest and Sybil regions. It is desirable for these sources of external trust to be as decentralized as possible. However, in practice, this is very difficult to achieve. I have proposed two methods to get trust into the social graph: **community verification** and **trusted anchors**.

These methods can be used by an external reputation system. Entities will be able to prove their external reputation using a smart contract on Sapphire, which stores it in its state.

One can argue that a reputation system itself is a kind of SRIS, but the social graph is more robust. It is about social connections and mutual verification. Even entities with a smaller reputation score can be trusted if they have strong connections.

Community Verification

Proof of VCs, SBTs, and other valuable tokens. Entities can issue ZKPs attesting to the possession of credentials issued by trusted authorities and tokens, thereby demonstrating their trustworthiness. The network can assess the trustworthiness of individual tokens. Too many options can motivate entities to split VCs and SBTs into more fake accounts. Therefore, the community should be careful about which VCs and SBTs will be accepted and how many trust tokens will be assigned to entities that prove them.

Trusted Anchors

Trusted anchors are entities in the social graph that have disclosed their identity to the network, and other entities that have remained private place their trust in them. For example, public figures, association presidents, academics, and so on. These entities hold external trust, which is further distributed throughout the network via path proving.

Chapter 7

Security Analysis

This chapter discusses the properties of the proposed system, based on the primary properties of SRIS described in [Section 5.2](#). Furthermore, it discusses possible attack vectors within the proposed system and its potential shortcomings.

The most significant attacks occur when an attacker acquires a large number of Sybil identities, either by compromising the system or by stealing the identities of other users. At the same time, it is important to protect privacy and anonymity, and to prevent user tracking of users' actions, as this could lead to censorship.

7.1 Properties of Proposed System

Of the SRISs properties described in [Section 5.2](#), this system already fulfills uniqueness, humanness, integrity, anonymity & unlinkability, persistence, decentralization, transparency, inclusiveness, and equality.

Nevertheless, this system can still be improved by introducing mechanisms described below to enhance accountability, key recovery, and revocation, and to achieve better interoperability and scalability.

7.1.1 Sybil-Resistance/Uniqueness & Humanness

This is a key feature of the system. Facial biometric authentication and social graph analysis complement each other. While facial biometrics can, to a certain extent, ensure **uniqueness** and **humanness** on its own, its biggest problems are fraud, forgery, and deepfakes. This is where social graph analysis comes in, allowing users to collectively verify the authenticity of registered facial biometrics. There is no third-party authority verifying users, which makes the system highly decentralized.

On the other hand, social graph analysis alone would not be sufficient, as there is a theoretical attack in which an attacker could create multiple virtual identities and distribute their use across multiple social groups in which they are present. It is natural for people to belong to multiple social groups, such as family, colleagues, friends from a sports club, and so on. The attacker could thus create an identity for each such social group and gain connections within it for community verification. Facial biometric registration prevents this by maintaining a database of registered users. While it is possible to obtain registrations even when there is a match in registered facial biometrics, it is much more difficult for the attacker.

In practice, even the currently used centralized solutions that ensure Sybil resistance are not perfect. It is likely impossible to create a system that is 100% Sybil-resistant. However, with the right protocol design, it is possible to make it difficult for an attacker to acquire a significant number of Sybil virtual identities.

7.1.2 Non-transferability & Coercion Resistance

As previously stated, ensuring non-transferability is nearly impossible. It cannot be prevented that a user willfully or accidentally give up their private key.

In critical systems, such as in-person elections, a user can present a facial scan verified by ROFL, ensuring the person is registered and genuinely authentic. Even a stolen or sold identity would not be enough for an attacker; however, physical presence is a crucial factor here.

There is also no protection against coercion. As mentioned before, to some degree, coercion can be mitigated by a key-recovery mechanism, as the attacker cannot be sure that the user will not recover their private key. The same applies to identity trading, when the buyer cannot be sure that the seller of the identity will not restore it. However, the current design does not include any key-recovery or revocation mechanism.

7.1.3 Key-Recovery & Revocation

While the system currently lacks a key recovery or revocation mechanism, such a feature can certainly be implemented. Since users have established trusted connections, one of the simplest solutions is for each user to select a set of highly trusted connections through which they can recover their private key or generate a new one. A similar approach is used, for example, in BrightID [14].

7.1.4 Integrity & Decentralization & Transparency

Integrity, decentralization, and transparency are inherent characteristics of blockchain technology. Furthermore, both the protocol details and the source code are publicly available. Only the embedding database and the relayer are centralized. However, the database is encrypted and accessible only to ROFL, and the relayer can be bypassed by writing directly to the blockchain.

7.1.5 Anonymity & Unlinkability

Anonymity and unlinkability are key features of the proposed system. This is most evident in the proposed solutions for social graph analysis. Actions such as establishing and revoking connections, or performing ZK random walks, are designed to protect users' identities and prevent attackers from tracking their activities. Information about registered users, established connections, or other profile state information is accessible only in TEEs.

Additionally, a mechanism could exist where verified users create a commitment within a Merkle Inclusion Tree. Using a ZKP, they can demonstrate that they possess a verification confirming they are not a Sybil under any virtual identity, not just their registered virtual identity in the system. This adds an extra layer of privacy protection and ensures unlinkability. However, there is also a risk of a replay attack against such proof.

7.1.6 Persistence

The registered identity is permanent, but it is necessary to submit ZK proof of paths each epoch. Fortunately, this renewal process is fully automatic and requires no user action.

7.1.7 Accountability

There are two types of accountability in the system. The first type is accountability to the users who utilize the system. Due to a strong emphasis on privacy and unlinkability, it is not feasible to track malicious users who may misuse the system and hold them accountable for their actions.

The second type is accountability to the entities that operate the system. While most components of the system are decentralized, specific elements, such as the embeddings database and the relay, can be held accountable for censorship or actions that may harm the system.

Furthermore, these centralized components can be replicated and made partially decentralized. However, the existing design currently offers no specific solution to address this issue.

7.1.8 Interoperability

The interoperability of the proposed system is not discussed, but the smart contract on Sapphire can issue a VC after successfully proving the ZK proof of paths. The VCs are well recognized and can be used in most decentralized systems. Therefore, the proposed system can be extended to have good interoperability.

7.1.9 Inclusiveness & Equality

To join the system and create an identity, the only requirements are to have a modern phone with a camera, trusted hardware, and support for C2PA manifests. However, because the C2PA standard is relatively new, meeting these requirements may be challenging for some users. Nevertheless, it is anticipated that most phones will support these features in the future. Looking ahead, this system is designed to be very inclusive and equal for everyone, with no special entities or privileges.

7.1.10 Scalability

The most critical aspect of scalability is the sharing of ZK random walks. Entities with numerous connections must generate many computationally intensive proofs. However, other parts of the system scale effectively. By design, blockchain technology scales well with user numbers, and the database with embeddings can be sharded using the Inverted File Index (IVF) technique.

7.2 Facial Biometric Fraud, Forgeries, and Deepfakes

This issue has already been addressed through the C2PA standard and trusted hardware in modern smartphones. However, a highly sophisticated attacker may still be able to bypass the trusted hardware or replace the C2PA manifest. There is significant dependence on smartphone manufacturers to ensure the secure operation of the camera and trusted

hardware. Therefore, the problem of such attacks has not been completely resolved, but it takes a high level of skill to subvert the system. Additionally, even if the attacker registers multiple identities, they still have to prove paths in the social graph, which requires having connections with a reputation.

7.3 Facial Biometrics Theft

It may be easier for an attacker to steal 3D facial scans of others for registration than to bypass trusted hardware. This could involve scanning people’s faces without their knowledge or creating sophisticated masks. However, the attacker still faces challenges in establishing honest connections, except for those registered with stolen facial biometrics.

A significant issue arises when the rightful owners of these facial biometrics attempt to register. They may inadvertently match their facial biometric data to their already-stolen data, leading to limited registration. Although there is a mechanism to get full registration, this type of attack could effectively lead to a denial-of-service situation.

One potential solution is to introduce a financial incentive that requires users to stake a certain amount of money during registration. This money would be returned after they successfully create the first ZK proof of paths. However, this approach may impact inclusivity and does not prevent wealthy attackers from carrying out such an attack.

7.4 Attacks on Social Graph Analysis

The fundamental requirement for a Sybil attack to even occur is that the Sybil region needs to have k attack edges, where k is the number of required distinct paths. Furthermore, the distribution of these attack edges between Sybil entities also matters. The best scenario for a Sybil cluster is when the attack edges are spread among Sybil entities. This is because if there were only k attack edges and one Sybil entity possesses two of these k attack edges, then for the other Sybil entities, these two attack edges effectively become one, because the paths to the honest region must pass through this one Sybil entity, and thus it will no longer be possible to prove two distinct paths, but only one.

Therefore, it can be challenging for a single person to create multiple Sybil identities, but a group of malicious users may find it easier to do so. If more than k individuals collaborate, their real identities can be used as attack edges by the Sybils they generate. This concept is illustrated in [Figure 7.1](#), with $k = 3$. Such a coordinated group can leverage the reputation of their real identities to have their Sybil identities verified.

Addressing this issue is challenging. Without centralized authorities, users must take responsibility for their connections, but this reliance on user behavior is not ideal for system design. A public blacklist could help identify malicious users or groups, but maintaining accountability in a privacy-preserving system remains difficult.

7.5 Censorship Attacks

There are two types of possible censorship. The first is from the relayer, which adds commitments to the public blockchain, and the second is from users, who may refuse to share paths to a specific individual.

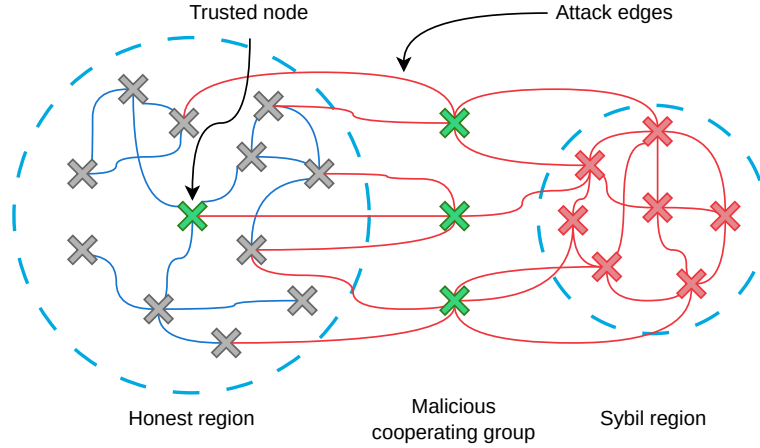


Figure 7.1: An attack by a coordinated group on the proposed social graph analysis method.

7.5.1 Relay Censorship

As a centralized component relay, commitments issued by the smart contract on Sapphire can be censored when sent to a public blockchain. While users can bypass this relay by directly submitting their commitments to the blockchain, doing so reveals the identities of those involved in the commitment.

This issue is particularly significant for connection commitments, as it is crucial to maintain the parties' identities confidential. Although it may not be a privacy concern at the moment, when users connected by these commitments request a Merkle Inclusion Proof (MIP) from an indexer service, that service can track the connections associated with them.

To address this issue, users can utilize mixing services or consider using a different relay to add their commitments to the blockchain. Additionally, when using the indexer service to obtain an MIP, users should request multiple commitments, even those unrelated to them, to further obscure their connections.

7.5.2 Entities Censorship

For the system to function properly, users must share ZK random walks with one another. However, users may choose to censor certain individuals. Such censorship is hard to detect because only specific high-reputation paths may be censored. Meanwhile, low-reputation paths that hold less value are still shared.

When an entity establishes a connection, there should be a basic level of trust that both parties will act honestly with each other. Additionally, it can be assumed that everyone has a few close connections they can rely on. The system's decentralized nature ensures that even if some users opt to censor connections, others will always be willing to participate.

Chapter 8

Implementation and Testing

This chapter presents the practical realization of the Proof-of-Concept (PoC) for the system designed in [Chapter 6](#). It covers the tools, libraries, implementation details, and technical challenges encountered throughout the process. The implementation is divided into three parts:

1. **Simulation of the social network analysis:** This simulation shows how the trust distribution and equations from [Section 6.5](#) work in practice. While the equation describes the system theoretically, in practice, there are many probabilistic variables, such as the graph structure, user reputation scores, and edge weights assigned by users to one another. The purpose is to test the limits of the social graph analysis.
2. **Implementation of circuits for ZK random walks and ZK proof of paths:** This is the core for the social graph analysis. Two different circuit approaches are implemented for ZK random walks, and for both ZK random walks and ZK proof of paths, there are practical examples.
3. **PoC implementation of the registration and connection establishment system:** This is a simplified implementation of all system components and their interactions based on the proposed design, as illustrated in [Figure 6.2](#).

8.1 Social Graph Simulation

The mathematical principles outlined in [Section 6.5](#) for social graph analysis, along with the verification requirements, are theoretically sound. However, the primary motivation for designing the system in this way was to allow users without an external reputation to achieve verification. This leads to a question: What percentage of users must have an external reputation, given the number of user connections, and how high must this reputation be for users with no external reputation to be able to verify?

Note that in an ideal situation, when the users are well connected and there is enough trust in the network, the system always works. The simulation tests edge cases, such as situations with insufficient trust or network connections, so entities may struggle to get verified.

It is also possible to simulate Sybil attacks. However, the capacity of Sybil entities to acquire attack edges, meaning connections, is entirely dependent on the actions of honest users and the weights they assign to these edges. Such dynamics are challenging to predict and therefore difficult to simulate.

The simulation is implemented in Python 3.12 with libraries **NetworkX**¹ for creation, manipulation of social network, and **Pyvis**² for interactive network visualization. The implementation is structured into the following components:

- **Configuration:** Definitions for simulation parameters.
- **Graph Builder:** Module for constructing the network using the **NetworkX** library.
- **Simulation:** Models the evolution of reputation scores for entities in the network over discrete-time epochs.
- **Visualizer:** Module for interactive visualization of simulation results using the **Pyvis** library.

8.1.1 Configurations

Various parameters can be adjusted to test different scenarios. To separate them logically, they are divided into categories: **reputation distribution parameters**, **network’s structure parameters**, **Sybil attack parameters**, **simulation flow parameters**. Understanding these parameters demonstrates what can be simulated.

Reputation Distribution Parameters

These include parameters from equations introduced in [Section 6.5](#). The maximum reputation $R_{max} = 10$, minimum reputation is $R_{min} = 0$, path reputation decay factor $\alpha = 0.8$, intrinsic reputation decay factor $\beta = 0.7$, and maximum intrinsic reputation when external reputation is zero $\gamma = 2$. These parameters remain consistent across all simulations, as they are constants that must be selected in a reasonable way (e.g., $\alpha = 0.8$ means that each entity passes 80% of the path’s reputation from the previous hop).

Network’s Structure Parameters

These include the number of honest entities in the network, the graph generator used, the percentage of entities with an external reputation, and whether this reputation is *spread* using a normal distribution of values from the interval $\langle 0, R_{max} \rangle$ or *seeded*, meaning that each chosen entity has reputation R_{max} .

These two models of reputation strategies simulate possible solutions to sources of trust described in [Section 6.5.4](#). The first model, known as the seeded model, relies on trusted anchors, where a select few entities (seeds) with high reputations. In contrast, the second model, called the spread model, emphasizes community verification, allowing anyone to establish a degree of external reputation. In both models, it is interesting to simulate the percentage of entities that require a reputation, so that a larger portion of the network can be verified by proving paths through these entities.

Regarding the connection (edge) weights between nodes, there is no parameter to set them; instead, they are randomly sampled from the interval $\langle 0, 1 \rangle$ according to a normal distribution.

The second interesting thing to simulate is the network’s structure. The **NetworkX** library offers many graph generators for this purpose. The most notable for social networks are [\[43\]](#):

¹<https://networkx.org/>

²<https://pyvis.readthedocs.io/>

- **Watts–Strogatz Graph (WS):** Represents a conservative, low clustering social network, where entities keep the closest connections. Each entity starts with k connections, and additional connections are added via random rewiring.
- **Barabási–Albert Graph (BA):** Introduces *hubs*, which are entities with many connections. They correspond to well-connected entities, such as public figures or influencers in social networks.
- **Holme–Kim Graph (HK):** A more sophisticated version of BA. It tries to mimic the real structure of a social network. In this model, entities that share a common neighbor are more likely to be connected, following the friend-of-a-friend principle. Entity clustering occurs in a manner similar to BA.

The simulation can use a combination of WS and BA models (with a user-defined ratio), or operate with either WS or HK alone. Using only the BA model is illogical, as social networks do not consist only of hubs. The WS-BA combination simulates a social network with low clustering, conservative users, and a subset of highly connected individuals. In contrast, the WS model alone creates a social graph that lacks these highly connected users, which serve as connection hubs. The HK model can be used independently, as it forms clusters of connected entities, including those that function as hubs.

Sybil Attack Parameters

Each simulation has one Sybil region, whose network is always fully connected, with edges of weight 1, as this gives the Sybils the best chance to find the best possible paths. Furthermore, it is possible to specify the number of Sybil entities in the region, the number of attack edges to the honest region, and the attack strategy, which can be random or focused on honest entities with more connections.

Additionally, there is a special parameter specifying the number of gateways. A *gateway* is a real entity with standard connections to the honest region and a reputation. However, this gateway is also fully connected to the Sybil region. It represents an attacker who uses their real identity to facilitate connections to the Sybil cluster they have created.

Simulation Flow Parameters

These are the remaining technical parameters, including the number of simulated epochs, the selected random seed for the random number generator, the logarithm base used to determine the required length and number of paths (e.g., 2, e , or 10), and the number of parallel processors used for simulation, as it is possible to speed up the simulation by allocating multiple CPUs.

8.1.2 Graph Builder

The graph builder utilizes the `NetworkX` to construct the honest and Sybil regions based on the given configuration. This includes setting up entities as graph nodes, connecting them with weighted edges, assigning reputations, and adding attack edges. The result is an initialized graph that can be used in the simulation.

8.1.3 Simulation

In this module, the equations outlined in [Section 6.5](#) are converted into code and utilized for real computations.

The simulation operates in discrete time across a defined number of epochs. In each epoch, a snapshot of the current state is taken, and for each entity:

1. The best combination of k distinct and valid paths is selected based on the network state from the previous epoch. A path is deemed valid if it meets the required length and reputation criteria (as described in [Section 6.5](#)).
2. A new intrinsic reputation R_I is calculated.
3. If the entity has the required number of valid paths with the required reputation, it is determined as verified.

The results from all entities are applied to update the overall state, and metrics such as the ratio of verified honest entities to all honest entities are calculated. Furthermore, the graph state and relevant metrics are captured for visualization.

Note that each entity’s state is calculated independently, allowing the process to run in parallel across multiple CPUs.

Finding the best combination of k distinct paths is a non-polynomial (NP) problem. In the implementation, it is solved with linear programming using the PuLP³ library. However, this is computationally intensive for large networks with a high k . A possible improvement would be to replace the solution with linear programming using a sufficiently good heuristic.

Nevertheless, it is impossible to conduct such an analysis on a real social network with millions of users. However, in practice, all entities only compute their best combination of k distinct paths individually, which is not particularly difficult for reasonable values of $k \in \{6, 7, 8, 9, 10\}$. This leads to an interesting observation that the distribution of this computation actually makes such analysis feasible.

8.1.4 Visualizer

The state of captured epochs can be interactively visualized using the Pyvis library.

There are interesting visual features that can also be seen in the screenshot from the simulation in [Figure 8.1](#). On the right side of the interface are statistics and the simulation settings used. There is also a slider that allows navigation through the epochs and observation of the development of verified entities over time. When hovered over a node (representing an entity) in the graph, it displays the values for R_E , R_I , and the total reputation. Similarly, hovering over an edge reveals the weights in both directions.

Also, different types of nodes, honest nodes, Sybil nodes, verified nodes, and attack edges are color-coded for better clarity. However, the best feature is that clicking on a node highlights the paths used for verification in that specific epoch. This function enables a manual recalculation of the results.

8.1.5 Experiments

The primary objective of the experiments was to assess the ability of entities without external reputation to achieve verification. The experiments involved different graph models

³<https://coin-or.github.io/pulp/>

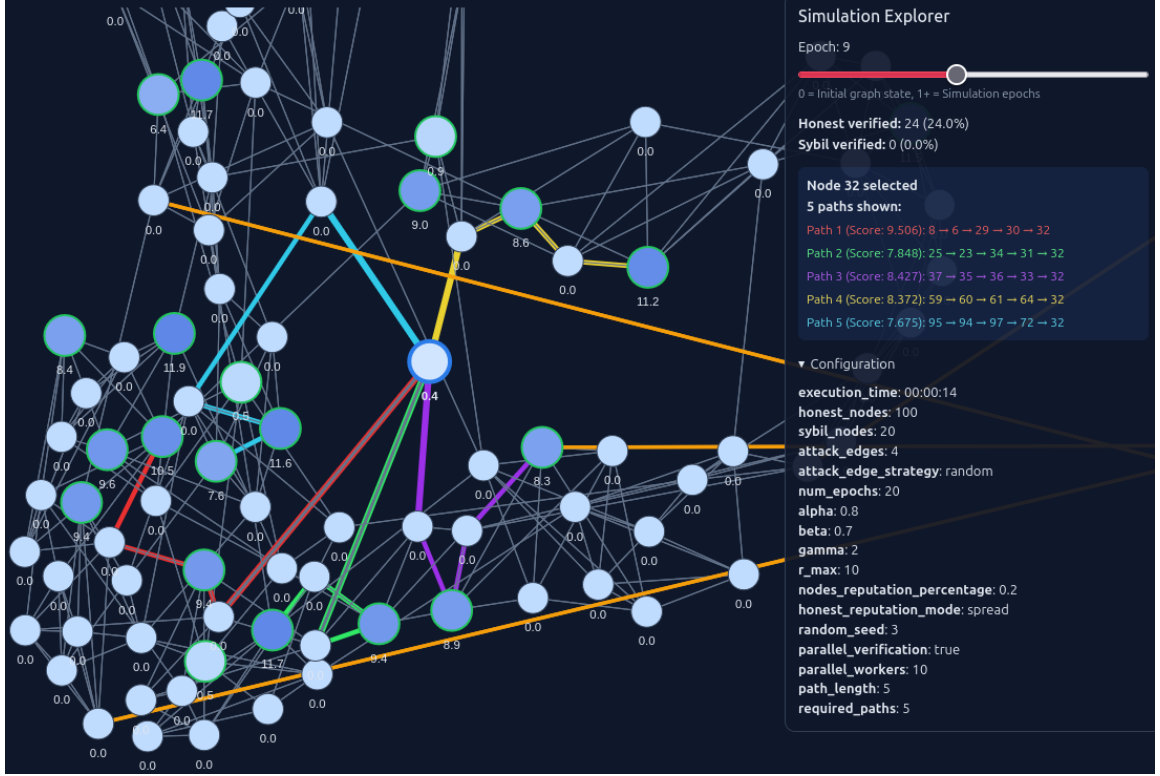


Figure 8.1: Screenshot from the social network simulation visualizer.

consisting of 100 entities, conducted over 30 epochs, with a required path length of 5 and a necessary number of paths also 5.

Simulations were carried out in both spread and seed modes, utilizing varying initial percentages of external reputation. In spread mode, the initial reputation percentages ranged from 10% to 30%. Conversely, in seed mode, the percentages were lower, between 5% and 10%, since it is not anticipated that more than 10% of the network will consist of seed entities. The results of these experiments are presented in [Table 8.1](#).

Graph / ex. rep. %	Spread				Seed		
	10%	15%	20%	30%	5%	8%	10%
HK	10%	85%	85%	100%	5%	100%	100%
WS + BA	9%	14%	14%	63%	5%	9%	12%
WS	10%	13%	25%	60%	5%	8%	18%

Table 8.1: Results of social network simulations with 100 entities over 30 epochs using different graph models and percentage of initial external reputation. The table shows the percentage of verified entities.

The HK (friend-of-a-friend) model performs well in both the spread and seed modes. It is mainly because entities in this model are very well connected.

Conversely, the conservative WS graph model requires a higher level of external reputation in spread mode and fails completely in seed mode. This is expected since there are no entities acting as hubs through which reputation can be distributed.

The WS-BA graph model performs the worst. This is primarily due to most entities in this graph operating according to the WS model, with only a small portion aligned with the BA model (specifically in the first 10%, seeded by the initial level of external reputation). This small portion only forms a cluster and does not significantly assist the other entities.

The results indicate that when entities maintain very close connections, more than 30% of them need to have an external reputation for the majority of the network to successfully verify against the system. In contrast, with a larger number of connections, the spread model performs well when only 15% of the entities have external reputations, and the model with seed entities begins to function effectively even at 8%.

This highlights the cold-start problem: at the beginning, when entities lack sufficient connections, they struggle to verify. However, a higher percentage of external reputation in the network can help mitigate this issue.

8.2 ZK Random Walks

The most crucial part was choosing the right proof system, because recursive proofs can be very computationally intensive. While there are many ZK proving systems, the development is very fast, and many implementations lack good documentation. I decided to use **Plonky2**⁴ [70] mainly because of very fast recursive proofs and Poseidon hashing, as the designed circuit requires computing three Merkle proofs. Additionally, there is good documentation with many examples, even with recursive proofs.

As a notable alternative, I was suggesting Nova [46], but here is one very important catch. While recursion via folding in Nova is very efficient, even more efficient than in Plonky2, each fold requires a private witness from the previous fold, and only the final proof is zero-knowledge. This makes Nova unsuitable for the designed system, as the private witness cannot be disclosed at each recursive step.

Because Plonky2 is implemented in Rust (Nightly), the implementation of ZK random walks has to be as well. There is one notable abstraction compared to the original design, as it is a PoC implementation. The Bloom filter for ensuring path distinctiveness is replaced with a simple array of nullifiers. This simplifies the implementation, but does not necessarily make the proof more efficient. The circuit has to iterate and compare nullifiers in the array, but Poseidon hashing used in the Bloom filter is also very fast.

Apart from the mentioned abstraction, the ZK circuit is implemented exactly as described in Section 6.5.2. In addition, there are two different approaches to implement this circuit:

- **Linear:** One separate compiled circuit per recursion depth, so it is required to have $l+1$ circuits, where l is the required length of the path. This approach is possible because the number of recursive steps is predictable. Each circuit embeds the Verification Key (VK) of the previous circuit (a specific, public component used in non-interactive ZKPs), but there must also be a base circuit without prior recursion. Linear setup: `BaseCircuit` $\rightarrow$ `StepCircuit[0]` $\rightarrow$ `StepCircuit[1]` $\rightarrow \dots \rightarrow$ `StepCircuit[l]`. The advantage is that each circuit is smaller and simpler compared to the Cyclic circuit. This positively affects the proof time, verification time, and circuit size.
- **Cyclic:** A single self-referential circuit for all recursion steps that verifies its own previous proof via a special Plonky2's recursive method. This method uses a dummy

⁴<https://github.com/0xPolygonZero/plonky2>

inner proof that bypasses all constraints during the first step, and a recursive real proof with the full constraint set in every other step. The advantage is that there is only one universal circuit for any required path length. However, it is more complex, which negatively affects the proof time, verification time, and circuit size.

The implementation is divided into the following parts:

- **Merkle tree utils:** Utilities for constructing Merkle tries and sparse Merkle tries, primarily for testing. The Poseidon implementation of Plonky2 is used as a hash function.
- **Merkle proof gadgets for ZK circuits:** Both linear and cyclic circuits use Merkle inclusion proofs and Merkle non-inclusion proofs; these circuit gadgets are separated here and used in both approaches, so the implementation is not repeated.
- **ZK random walk circuit:** Implementation of both linear and cyclic circuits with the use of Merkle gadgets.
- **Prover and verifier:** Implementation of prover and verifier APIs for both linear and cyclic approaches with the same template. There are three prover methods:
 - **setup():** Compiles the circuit and, in the linear variant, builds the base circuit and a separate circuit for each step. In the cyclic variant, a single unified circuit is built.
 - **prove_base():** Generates the base proof for the linear variant and the dummy proof for the cyclic variant.
 - **prove_step():** Takes the previous proof (or the base/dummy proof) and the necessary inputs, and extends the proof by one step.

For the verifier, there is just one method:

- **verify_walk_proof():** Takes the circuit and the proof as parameters and verifies their validity.

8.2.1 Examples

Two examples were made for both the linear and cyclic variants. **three_hop** example shows a ZK random walk with base plus three steps, and **large_tries** example shows only base plus one step, but with Merkle tries of depth 20 and the sparse Merkle tree of depth 32. The three-step walk is a sufficient example, as each step requires the same computational effort. The **large_tries** example demonstrates the efficiency when more hash operations are required, as the **three_hop** is trivial with only four elements in Merkle tries. The results are shown in [Table 8.2](#). The experiments were performed on a laptop with an AMD Ryzen 5 5600H processor and 16GB of RAM.

The first half of the table shows circuit compilation time and circuit size. Note that, in the linear variant, the base circuit is 8KB and each step is an additional 4.2MB. The cyclic variant has only one universal circuit of size 8.4 MB, but because it is more complex, compilation time is longer.

The second half of the table shows **the proof time for each step**, the verification time of the proof, and the size of the proof. While verification time and proof size are

Example	Circuit		Proof		
	Comp.	Size	Prove	Ver.	Size
three_hop_linear	1.56s	8KB + $3 \times 4.2\text{MB}$	312.92ms	4.46ms	124KB
large_tries_linear			328.56ms	4.44ms	124KB
three_hop_cyclic	2.39s	8.4MB	625.44ms	4.57ms	130KB
large_tries_cyclic			630.27ms	4.68ms	130KB

Table 8.2: Results of ZK random walk `three_hop` and `large_tries` examples for the linear and cyclic variants.

comparable between the two variants, the complexity of proof time is almost double in the case of the cyclic variant.

The very interesting observation is that there is almost no difference between proofs in `three_hop` and `large_tries` examples, despite the significant difference in Merkle proof sizes. This is a feature of the Plonky2 proof system, where Poseidon hashing is very efficient.

8.3 ZK Proof of Paths

ZK proof of paths is essentially an aggregation of ZK random walks into a single proof. The implementation also uses the Plonky2 proof system, as in ZK random walks. Again, I was considering Nova, as this is the perfect situation to use it. However, Plonky2 and Nova use fundamentally different mathematical principles, and it is not possible to use Plonky2's proof inside Nova.

The ZK circuit is implemented exactly as described in [Section 6.5.3](#). The only difference is replacing Bloom filters with an array of nullifiers, similar to the ZK random walks implementation. The structure of implementation is also divided similarly:

- **ZK proof of paths circuit:** Implementation of the ZK circuit.
- **Prover and verifier:** Implementation of prover and verifier APIs. There are two prover methods:
 - `setup()`: Compiles the aggregator circuit based on the inner (ZK random walks) proof data, the number of aggregated proofs k , and required random walk lengths.
 - `prove()`: Takes k ZK random walk proofs and generate an aggregated proof.

For the verifier, there is just one method:

- `verify_aggregated_proof()`: Takes the circuit and the proof as parameters and verifies their validity.

According to the original design, the final ZK proof of paths should be verified in the Sapphire smart contract. However, the Ethereum Virtual Machine (EVM) does not have native support to directly verify Plonky2 proofs. It might be possible, but it would cost millions of gas. The standard solution is to add a proof conversion step that wraps the Plonky2 proof in an SNARK that the EVM can verify cheaply (essentially another recursion), such as Groth16, Gnark, or Halo2. Nevertheless, this conversion is not implemented in the current PoC.

8.3.1 Examples

There are two examples: the first aggregates three ZK random walks, and the second aggregates six ZK random walks. Both examples use the linear variant of the ZK random walk, but the cyclic variant would be very similar. The results are shown in Table 8.3. The experiments were performed on a laptop with an AMD Ryzen 5 5600H processor and 16GB of RAM.

Example	Circuit		Proof		
	Comp.	Size	Prove	Ver.	Size
aggregate_three	1.25s	16.9MB	1.58s	7.19ms	143KB
aggregate_six	2.65s	33.8MB	2.82s	12.41ms	149KB

Table 8.3: Results of ZK proof of paths examples aggregating three and six ZK random walks.

The first notable thing is that the circuit for six aggregated paths is almost double the size of the circuit for three aggregated paths. This is because of the linear variant of ZK random walks, which uses a different circuit for each step with a size of 4.2MB. The circuit aggregating the cyclic variant of ZK random walks would be much smaller.

However, the proof and verification times are fast, considering that recursion is usually very expensive in ZKPs. Note that this proof is done once an epoch, so a few seconds of computation is a very good result.

8.4 Registration and Connection Establishment

The architecture of implementation corresponds to the model of communication with all components illustrated in Figure 6.2. Together, there are five components, namely: **Client**, **ROFL Mock**, **Sapphire Mock**, **Relayer**, and **Merkle Tree Smart Contract**. Using this implementation, it is possible to **locally** test identity registration and connection establishment between two entities.

However, as it is a PoC implementation, there are a few notable abstractions.

- Instead of using a 3D facial scan, only 2D face photos are used for registration and connection establishment, with no liveness detection.
- There are no C2PA manifests, as the standard is relatively new and not yet widely supported.
- Revocation of connections is not implemented.
- There is no Merkle tree for reputation commitments.
- The confidential features of smart contracts on Sapphire are abstracted as they are deployed only to the local EVM testnet.

8.4.1 Client

Client is a program on the entity side that communicates with ROFL and calls transactions on Sapphire smart contracts. It is implemented in Python 3.12 using `httpx`⁵ library to

⁵<https://www.python-httpx.org/>

send POST request to ROFL, `Web3.py`⁶ and `eth_account`⁷ libraries for handling blockchain interaction.

It supports two command-line interface (CLI) commands:

- **register**: Registers a new user identity using 2D photo. A required parameter is a path to a JSON file containing *SK* and *PK*, and a path to a 2D face photo. The Client reads the JSON file, loads the photo, signs the payload, and sends a POST request to ROFL. If ROFL finds a match, the registration is rejected without an on-chain transaction. If no match is found, then the client saves the returned embedding from ROFL and uses the signed ROFL result to call an on-chain transaction to register a new identity.
- **connect**: Initialize a connection with other entity. A required parameter is a path to a JSON file containing *SK*, *PK*, the *ID* of the entity to connect with, and a path to a 2D face photo of this entity. The Client reads the JSON file, loads the photo, loads the entity's stored embedding, signs the payload, and sends a POST request to ROFL. If there is a match between the sent embedding and the embedding from the sent photo, then ROFL returns a signed connection commitment that is used to call an on-chain transaction to register a new connection. If there is no match, no on-chain transaction is sent.

8.4.2 ROFL Mock

ROFL Mock is a component that represents a program running inside the Oasis ROFL TEE. Oasis ROFL requires a Docker or Podman container with a program to execute. Therefore, the ROFL Mock is implemented as a Docker container with a program that supports identity registration and verification during connection establishment. There is also a Docker container with a PostgreSQL database for storing encrypted embeddings.

The implementation is in Python 3.12 using `FastAPI`⁸ library to make a simple HTTP server, `FaceNet`⁹ and `PyTorch`¹⁰ libraries to extract and compare facial embeddings, and `cryptography`¹¹ and `asyncpg`¹² libraries to encrypt/decrypt embeddings (using AES symmetric cypher) and communicate with the PostgreSQL database.

Apart from the mentioned abstractions, ROFL Mock works the same as described in Sections 6.3.1 and 6.4.2. It processes and responds to the **register** and **connect** requests from the Client and signs the result with its private key. The connection commitment created during connection establishment uses Keccak, as there are no good Python libraries implementing the Poseidon hash. This is not a problem, as the smart contract for connection establishment verifies the connection commitment using Keccak and then recomputes it with Poseidon.

⁶<https://web3py.readthedocs.io/>

⁷<https://eth-account.readthedocs.io/>

⁸<https://fastapi.tiangolo.com/>

⁹<https://github.com/timesler/facenet-pytorch>

¹⁰<https://pytorch.org/>

¹¹<https://cryptography.io/>

¹²<https://github.com/MagicStack/asyncpg>

8.4.3 Sapphire Mock

Sapphire Mock is a component that represents confidential smart contracts on the Oasis Sapphire network. The Oasis Sapphire includes a specialized library for making signatures in smart contracts and generating random values. However, these features do not function on a local test net. As a result, the generation of random values has been abstracted, and the signing of smart contract messages has been completely omitted. Apart from the mentioned abstractions, the implementation of identity registration and connection establishment is the same as described in [Sections 6.3.2](#) and [6.4.3](#).

The logic is separated into two smart contracts implemented in **Solidity**:

- **IdentityRegistry**: On-chain registry of identities and their states. Entities can call the `registerProfile()` function to register a new identity.
- **ConnectionManager**: On-chain registry of established connections. Entities can call the `establishConnection()` function to establish a new connection. As described in the design ([Section 6.4.3](#)), both entities have to submit a transaction on-chain. Both parties call the same smart contract function. The initial call sets the connection to the `INITIALIZED` state. Only after the second party makes their call does the state transition to `COMPLETED`, indicating that the connection has been successfully established. As mentioned, the connection commitment must be recalculated using the Poseidon hash, since ROFL Mock uses Keccak. Poseidon hash is important because it is a ZKP-friendly hash, and connection commitments are indeed proved using a Merkle tree inclusion proof within the circuit.

The ROFL Mock address is hardcoded into both deployed smart contracts, allowing them to verify ROFL Mock's signature.

8.4.4 Relayer & Merkle Tree Smart Contract

Relayer is a simple component implemented in **Python 3.12** that listens to events emitted by the **ConnectionManager** using the **Web3.py** library and specifically the event signaling the establishment of a new connection. Relayer takes connection commitments emitted in these events and calls the `insert()` function in the Merkle tree smart contract.

The Merkle tree smart contract is an implementation of an Incremental Merkle Tree (IMT) in **Solidity** for storing connection commitments. Note that this contract should validate the signature from the **ConnectionManager** smart contract, so the insertion into the IMT is controlled by it. However, since the signing functionality is omitted, this signature verification has not been implemented as well.

8.4.5 Testing

As this implementation serves as a PoC, testing demonstrates how communication between components works in practice rather than evaluating performance and scalability. This involves five steps:

1. Setup of all components in the systems. This involves deploying all three smart contracts on a local Ethereum test net provided by Anvil¹³, using Docker Compose to build and run ROFL Mock and the Postgres database, and starting the Relayer. All future on-chain actions can be confirmed by a log in the Anvil terminal.

¹³<https://www.getfoundry.sh/anvil>

2. Registration of entity A by executing the **register** command in Client with SK_A , PK_A , face photo of A (P_A).
3. Registration of entity B by executing the **register** command in Client with SK_B , PK_B , face photo of B (P_B).
4. Connection establishment between A and B by node A by executing the **connect** command in Client with SK_A , PK_A , ID_B (address on Ethereum test net), and P_B .
5. Connection establishment between A and B by node B . Execution of **connect** command in Client with SK_B , PK_B , ID_A , and P_A . After this execution, the Relayer writes the log message in its terminal and inserts the connection commitment into the on-chain Merkle tree.

During testing, all steps were successfully executed and yielded the expected results, confirming the correctness of the implementation.

Chapter 9

Discussion

This chapter discusses potential limitations, areas for improvement, and unresolved questions.

9.1 Facial Scan False Positives

False positives during identity registration can complicate the process for affected users. Therefore, it is crucial to minimize them as much as possible. Achieving this requires high-quality models for extracting facial embeddings.

Additionally, it is important to have an effective mechanism for addressing false positives that arise during registration. The current solution proposes a system where each registered user has a vote that serves as an endorsement for potentially affected users. While this approach may work in practice, it could be improved. On the one hand, there is a risk that endorsements could be bought; on the other hand, it may be difficult for honest users to obtain them.

9.2 Liveness Detection in ROFL

In the proposed design, liveness detection is conducted on a 3D facial scan. However, it may be possible to create a more advanced protocol for liveness detection utilizing live video. This could involve real-time interaction between the program in ROFL TEE and the user, where the program presents the user with random challenges to verify their liveness and authenticity.

9.3 Trust in the Social Graph

The proposed design requires an external reputation system to convey the level of trust in entities. The main effort was to minimize dependence on such a system, but it is ultimately unavoidable.

Except for the potential sources of trust, it is not discussed how such a system can be designed. However, future solutions should be stringent rather than lenient and must also consider the expiration of external reputation.

Regarding the ZK random walks, there remains an unresolved question regarding how to establish the threshold for path reputation. A formula for a minimum threshold has been suggested, based on the maximum possible intrinsic reputation an entity can attain without

any external reputation. However, it may be more suitable to set the optimal threshold dynamically, as reputation levels in the network evolve over time.

The disadvantage is that newly registered users may find it challenging to get verified as the threshold for path reputation rises. Therefore, the dynamics must be adjusted based on multiple factors, making it difficult to set the threshold accurately.

9.4 Social Network Groups and Bootstrapping

Systems that use social graph analysis face an efficiency problem in bootstrapping new users. It takes time for these users to establish enough connections for the system to assess them as legitimate (non-Sybil) users. This issue is most pronounced when there are not yet enough users in the system, and the social network they created is not sufficiently connected.

Therefore, techniques that enhance the efficiency of this bootstrapping process are significant for such systems. A possible area of investigation is the analysis of social groups in the social network. While it is not necessary for all entities in a network to be connected, users tend to form groups that can be viewed as sub-social networks. An analysis can be performed within these sub-social networks and between them.

However, this approach may present new challenges since a Sybil region is itself a sub-social network, weakly connected to other parts of the social network. In the current system, external reputation is used to distinguish between honest and Sybil regions, but the social network is analyzed as a whole. This means that the more entities are in the network, the stricter the criteria for receiving verification from the system.

9.5 Persistence of Merkle Proofs

In the proposed ZK random walk, three Merkle proofs are required. However, there is a challenge associated with using Merkle proofs in ZKPs. The Merkle tree's root, which serves as public input, must remain recorded on-chain until the proof is verified. This is important because the verifier must be able to verify it in the corresponding Merkle tree smart contract.

The issue arises because each time a new commitment is added to the on-chain Merkle tree, the root changes. To address this, the Merkle tree smart contract maintains a set of previous roots, allowing verification that the root was valid at the time the proof was created.

This set of previous roots functions as a cyclic buffer. In the context of the proposed system, this buffer must be sufficiently large to last for the entire epoch. This is almost impossible to achieve, as it is unpredictable how many commitments will be inserted in an epoch. However, a more efficient solution would be to apply Merkle tree updates at the start of each new epoch. While this approach is feasible with a Relayer, it introduces the risk of censorship because the Relayer is a centralized component. Therefore, it is essential to explore alternative solutions to mitigate this issue.

Chapter 10

Conclusion

The purpose of this work was to design and implement a decentralized identity management system that is Sybil-resistant, meaning it is resistant to the creation of false identities. This goal was achieved through a detailed analysis of existing approaches, from which the combination of facial biometric verification and social graph analysis was selected as the most effective means to ensure the uniqueness of entities within the system. The proposed solution emphasizes maintaining the privacy, anonymity, and security of personal information. In this context, the protocol utilizes zero-knowledge cryptography, blockchain technology, and trusted execution environments. The system's functionality is demonstrated through a proof-of-concept implementation that includes identity registration and entity connection establishment, as well as a simulation of social graph analysis. Despite the complexity of the designed system, the implementation demonstrated the usability of the designed solution.

The resulting solution provides one of the few alternatives to the predominantly used centralized systems. It represents a significant step toward achieving self-sovereign identity management, which is crucial in today's world where personal data is a valuable commodity.

The key solution involves combining facial biometric registration with a unique protocol for establishing connections. This protocol requires a face-to-face meeting where the parties involved can verify each other's registered identities, thereby eliminating the need for a centralized authority. The connections formed create a social graph that can be analyzed, which is the purpose of the second part of the solution. For privacy reasons, the social graph itself is never stored. Instead, recursive zero-knowledge proofs of established connections are used to demonstrate paths in the social graph. Users only keep track of their own connections and can share zero-knowledge proofs of these connections, enabling other users to use them recursively in their own proofs. Ultimately, these recursive proofs create a path proof of varying length that illustrates deeper involvement in the social graph, facilitating its analysis.

Future work should focus on developing solutions for a decentralized reputation system, which is essential for social graph analysis. Additionally, there is a need for further research in the area of the proposed social graph analysis itself. There remain unresolved questions regarding how to dynamically set analysis parameters to ensure efficient verification and bootstrapping while preventing the verification of Sybil entities. Finally, the current implementation serves merely as a proof-of-concept. In the future, I would like to complete the implementation of the entire application, incorporating all abstracted components.

Bibliography

- [1] AHMED, M. R.; ISLAM, A. K. M. M.; SHATABDA, S. and ISLAM, S. Blockchain-Based Identity Management System and Self-Sovereign Identity Ecosystem: A Comprehensive Survey. *IEEE Access* online. Dhaka 1212, Bangladesh: IEEE, october 2022, vol. 10, p. 113436–113481. ISSN 2169-3536. Available at: <https://doi.org/10.1109/ACCESS.2022.3216643>. [cit. 2025-11-02].
- [2] AL QURISHI, M.; AL RAKHAMI, M.; ALAMRI, A.; ALRUBAIAN, M.; RAHMAN, S. M. M. et al. Sybil Defense Techniques in Online Social Networks: A Survey. *IEEE Access* online, january 2017, vol. 5, p. 1200–1219. revised 2017-03-13. ISSN 2169-3536. Available at: <https://doi.org/10.1109/ACCESS.2017.2656635>. [cit. 2025-12-21].
- [3] BALDIMTSI, F.; CHALKIAS, K. K.; JI, Y.; LINDSTRØM, J.; MARAM, D. et al. ZkLogin: Privacy-Preserving Blockchain Authentication with Existing Credentials. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* online. New York, NY, USA: Association for Computing Machinery, 2024, p. 3182—3196. CCS '24. ISBN 9798400706363. Available at: <https://doi.org/10.1145/3658644.3690356>. [cit. 2026-02-20].
- [4] BELOTTI, M.; BOŽIĆ, N.; PUJOLLE, G. and SECCI, S. A Vademecum on Blockchain Technologies: When, Which, and How. *IEEE Communications Surveys & Tutorials*. IEEE, 2019, vol. 21, no. 4, p. 3796–3838. ISSN 1553-877X.
- [5] BEN SASSON, E.; BENTOV, I.; HORESH, Y. and RIABZEV, M. Scalable, transparent, and post-quantum secure computational integrity. *Cryptology ePrint Archive, Paper 2018/046* online, october 2018. revised 2018-03-06. Available at: <https://eprint.iacr.org/2018/046>. [cit. 2026-02-20].
- [6] BENICHE, A. *A Study of Blockchain Oracles* online. arXiv, 2020. revised 2020-07-14. Available at: <https://doi.org/10.48550/arXiv.2004.07140>. [cit. 2026-01-19].
- [7] BENNETT, A. *Going Parabolic: Analyzing Sybil Resistance in Quadratic Voting Mechanisms for Blockchain-Based DAOs* online. Stanford, CA, US, 2025. 40 p. Dissertation. Department of Economics, Stanford University. Supervisor STRNAD, J. Available at: <https://doi.org/10.25740/hj860vc2584>. [cit. 2025-12-04].
- [8] BERENTSEN, A.; LENZI, J. and NYFFENEGGER, R. An Introduction to Zero-Knowledge Proofs in Blockchains and Economics. *Federal Reserve Bank of St. Louis Review* online, Fourth Quarter 2023, vol. 105, no. 4, p. 280–94. Available at: <https://doi.org/10.20955/r.105.280-94>. [cit. 2026-01-21].

- [9] BERGE, M.; KOKORIS KOGIAS, E.; JOVANOVIĆ, P.; GASSER, L.; GAILLY, N. et al. Proof-of-Personhood: Redemocratizing Permissionless Cryptocurrencies. In: *2017 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)* online. IEEE, April 2017, p. 23–26. ISBN 978-1-5386-2244-5. Available at: <https://doi.org/10.1109/EuroSPW.2017.46>. [cit. 2025-11-02].
- [10] BOSHMAF, Y.; LOGOTHETIS, D.; SIGANOS, G.; LERÍA, J.; LORENZO, J. et al. Integro: Leveraging victim prediction for robust fake account detection in OSNs. In: *NDSS* online. February 2015, vol. 15, p. 8–11. Available at: http://lersse-dl.ece.ubc.ca/record/296/files/boshmaf_ndss_2015.pdf. [cit. 2026-01-01].
- [11] BOWE, S.; GRIGG, J. and HOPWOOD, D. Halo: Recursive proof composition without a trusted setup. *IACR Cryptol. ePrint Arch.* online, 2019, no. 1021. Available at: <https://eprint.iacr.org/2019/1021.pdf>. [cit. 2026-02-20].
- [12] BRENZIKOFER, A. *Encointer – Local Community Cryptocurrencies with Universal Basic Income* online. version v2. arXiv, 14. december 2019. revised 2020-12-09. Available at: <https://doi.org/10.48550/arXiv.1912.12141>. [cit. 2025-12-06].
- [13] BREWER, E. A. Towards robust distributed systems. In: Portland, OR. *PODC*. 2000, vol. 7, 10.1145, p. 343–477.
- [14] BRIGHTID. *BrightID: Universal Proof of Uniqueness* online. whitepaper. January 2022. Available at: <https://www.brightid.org/whitepaper>. [cit. 2025-10-03].
- [15] BRIGHTID. *BrightID Gitbook* online. 2024. revised 2024-12-11. Available at: <https://brightid.gitbook.io/brightid>. [cit. 2025-11-24].
- [16] BUTERIN, V. DAOs, DACs, DAs and More: An Incomplete Terminology Guide. *Ethereum Foundation Blog* online. 6. may 2014. Available at: <https://blog.ethereum.org/2014/05/06/daos-dacs-das-and-more-an-incomplete-terminology-guide>. [cit. 2026-02-14].
- [17] BUTERIN, V. What do I think about biometric proof of personhood? *Vitalik Buterin’s website* online. 24. july 2023. Available at: <https://vitalik.eth.limo/general/2023/07/24/biometric.html>. [cit. 2025-11-02].
- [18] BÜNZ, B.; BOOTLE, J.; BONEH, D.; POELSTRA, A.; WUILLE, P. et al. Bulletproofs: Short Proofs for Confidential Transactions and More. In: *2018 IEEE Symposium on Security and Privacy (SP)* online. IEEE, May 2018, p. 315–334. ISBN 978-1-5386-4353-2. Available at: <https://doi.org/10.1109/SP.2018.00020>. [cit. 2026-02-20].
- [19] CAO, Q. and YANG, X. *SybilFence: Improving Social-Graph-Based Sybil Defenses with User Negative Feedback* online. 13. april 2013. Available at: <https://doi.org/10.48550/arXiv.1304.3819>. [cit. 2026-01-01].
- [20] CARONNI, G. Walking the Web of trust. In: *Proceedings IEEE 9th International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises (WET ICE 2000)* online. Gaithersburg, MD, USA: IEEE, June 2000, p. 153–158. ISSN 1080-1383. Available at: <https://doi.org/10.1109/ENABL.2000.883720>. [cit. 2025-11-29].

- [21] CHALKIAS, K. and HEARN, M. Demonstrate how Zero-Knowledge Proofs work without using maths. In: *CORDACON 2017 Conference*. February 2019, p. 5.
- [22] CHAUM, D. L. Untraceable electronic mail, return addresses, and digital pseudonyms. *Communications of the ACM*. ACM New York, NY, USA, 1981, vol. 24, no. 2, p. 84–90.
- [23] CHRISTENSEN, K.; ROGINSKY, A. and JIMENO, M. A new analysis of the false positive rate of a Bloom filter. *Information Processing Letters* online, october 2010, vol. 110, no. 21, p. 944–949. ISSN 0020-0190. Available at: <https://doi.org/10.1016/j.ipl.2010.07.024>. [cit. 2026-04-26].
- [24] COALITION FOR CONTENT PROVENANCE AND AUTHENTICITY. *C2PA Specification* online. 2026. Available at: <https://spec.c2pa.org/specifications/specifications/2.4/index.html>. [cit. 2026-04-18]. Version 1.3.
- [25] DANEZIS, G. and MITTAL, P. Sybilinfer: Detecting sybil nodes using social networks. In: San Diego, CA. *Ndss* online. February 2009, vol. 9, p. 1–15. Available at: <https://crysp.uwaterloo.ca/courses/pet/W11/cache/netfiles.uiuc.edu/mittal2/www/sybilinfer-ndss09.pdf>. [cit. 2026-01-01].
- [26] DOUCEUR, J. R. The Sybil Attack. In: DRUSCHEL, P.; KAASHOEK, F. and ROWSTRON, A., ed. *Peer-to-Peer Systems* online. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, p. 251–260. ISBN 978-3-540-45748-0. Available at: https://doi.org/10.1007/3-540-45748-8_24. [cit. 2025-12-27].
- [27] EFFENDY, S. and YAP, R. H. C. The Strong Link Graph for Enhancing Sybil Defenses. In: *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)* online. IEEE, July 2017, p. 944–954. ISBN 978-1-5386-1792-2. Available at: <https://doi.org/10.1109/ICDCS.2017.117>. [cit. 2026-01-01].
- [28] ERNSTBERGER, J.; CHALIASOS, S.; ZHOU, L.; JOVANOVIĆ, P. and GERVAIS, A. *Do You Need a Zero Knowledge Proof?* online. Cryptology ePrint Archive, Paper 2024/050, 2024. Available at: <https://eprint.iacr.org/2024/050>. [cit. 2026-01-21].
- [29] EUROPEAN UNION. *Consolidated text: Regulation (EU) No 910/2014 of the European Parliament and of the Council of 23 July 2014 on electronic identification and trust services for electronic transactions in the internal market and repealing Directive 1999/93/EC* online. 23. july 2014. revised 2024-04-30. Available at: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:02014R0910-20241018>. [cit. 2025-12-21]. Official Journal of the European Union, L 257, 28. 8. 2014, p. 73.
- [30] EUROPEAN UNION. *Regulation (EU) 2024/1183 of the European Parliament and of the Council of 11 April 2024 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework* online. 11. april 2024. Available at: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=celex%3A32024R1183>. [cit. 2025-12-21]. Official Journal of the European Union, L 1183, 30.4.2024.
- [31] FENG, Q.; HE, D.; ZEADALLY, S.; KHAN, M. K. and KUMAR, N. A survey on privacy protection in blockchain system. *Journal of Network and Computer Applications*

- online. Elsevier, 2019, vol. 126, p. 45–58. ISSN 1084-8045. Available at: <https://doi.org/10.1016/j.jnca.2018.10.020>. [cit. 2026-01-16].
- [32] FENG, Z.; LI, Z.; CUI, H. and WHITTY, M. T. Identity Management Systems: A Comprehensive Review. *Information* online, 2025, vol. 16, 9, 778. ISSN 2078-2489. Available at: <https://doi.org/10.3390/info16090778>. [cit. 2026-02-06].
- [33] FICHTER, K. What’s a Sparse Merkle Tree? online. Medium, september 2018. Available at: <https://medium.com/@kelvinfichter/whats-a-sparse-merkle-tree-acda70aeb837>. [cit. 2026-04-26].
- [34] FORD, B. *Identity and Personhood in Digital Democracy: Evaluating Inclusion, Equality, Security, and Privacy in Pseudonym Parties and Other Proofs of Personhood* online. arXiv, november 2020. Available at: <https://doi.org/10.48550/arXiv.2011.02412>. [cit. 2025-12-01].
- [35] FURUTANI, S.; SHIBAHARA, T.; AKIYAMA, M. and AIDA, M. Interpreting Graph-Based Sybil Detection Methods as Low-Pass Filtering. *IEEE Transactions on Information Forensics and Security* online. IEEE, january 2023, vol. 18, p. 1225–1236. ISSN 1556-6021. Available at: <https://doi.org/10.1109/TIFS.2023.3237364>. [cit. 2026-01-01].
- [36] GABIZON, A.; WILLIAMSON, Z. J. and CIOBOTARU, O. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. *Cryptology ePrint Archive, Paper 2019/953* online, august 2019. revised 2025-07-17. Available at: <https://eprint.iacr.org/2019/953>. [cit. 2026-02-20].
- [37] GILBERT, S. and LYNCH, N. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *SIGACT News* online. New York, NY, USA: Association for Computing Machinery, june 2002, vol. 33, no. 2, p. 51—59. ISSN 0163-5700. Available at: <https://doi.org/10.1145/564585.564601>. [cit. 2026-01-16].
- [38] GOEL, A. and RAHULAMATHAVAN, Y. A Comparative Survey of Centralised and Decentralised Identity Management Systems: Analysing Scalability, Security, and Feasibility. *Future Internet* online, 2025, vol. 17, 1, 1. ISSN 1999-5903. Available at: <https://doi.org/10.3390/fi17010001>. [cit. 2026-02-06].
- [39] GROTH, J. On the Size of Pairing-Based Non-interactive Arguments. In: FISCHLIN, M. and CORON, J.-S., ed. *Advances in Cryptology – EUROCRYPT 2016* online. Berlin, Heidelberg: Springer Berlin Heidelberg, April 2016, p. 305–326. ISBN 978-3-662-49896-5. Available at: https://doi.org/10.1007/978-3-662-49896-5_11. [cit. 2026-02-20].
- [40] HOMOLIAK, I.; VENUGOPALAN, S.; REIJSBERGEN, D.; HUM, Q.; SCHUMI, R. et al. The Security Reference Architecture for Blockchains: Toward a Standardized Model for Studying Vulnerabilities, Threats, and Defenses. *IEEE Communications Surveys & Tutorials* online. IEEE, 2021, vol. 23, no. 1, p. 341–390. ISSN 1553-877X. Available at: <https://doi.org/10.1109/COMST.2020.3033665>. [cit. 2026-01-16].

- [41] IDENA. *Idena documentation* online. 2024. Available at: <https://docs.idena.io/>. [cit. 2025-11-24].
- [42] IMMORLICA, N.; JACKSON, M. O. and WEYL, E. G. Verifying Identity as a Social Intersection. online. SSRN, april 2019, p. 20. Available at: <https://doi.org/10.2139/ssrn.3375436>. [cit. 2025-11-29].
- [43] JAIN, S. NetworkX for Python – A Practical Guide to Graph Generators. online. Medium, february 2026. Available at: <https://medium.com/@jainsnehasj6/networkx-for-python-a-practical-guide-to-graph-generators-1932664f545e>. [cit. 2026-05-14].
- [44] KIM, N. Bloom Filters Explained. *System Design* online. 6. march 2023. Available at: <https://systemdesign.one/bloom-filters-explained/>. [cit. 2026-04-26]. Path: Home;Archive;Fundamentals;Bloom Filters Explained.
- [45] KLEROS. *The Internet Of Humans: A decentralized, sybil-resistant, list of humans* online. 2021. Available at: <https://proofofhumanity.id/>. [cit. 2025-12-12].
- [46] KOTHAPALLI, A.; SETTY, S. and TZIALLA, I. Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. *Cryptology ePrint Archive, Paper 2021/370* online, march 2021. revised 2024-07-20. Available at: <https://eprint.iacr.org/2021/370>. [cit. 2026-02-20].
- [47] KUMAR, B. and BHUYAN, B. Game Theoretic Defense Framework Against Sybil Attacks. *SN Computer Science* online. Springer Nature, september 2024, vol. 5, no. 857, p. 16. Available at: <https://doi.org/10.1007/s42979-024-03153-0>. [cit. 2025-12-07].
- [48] KÖPPELMANN, M. Introducing Circles – Universal Basic Income. online, june 2018. Available at: <https://ourbasicincome.wordpress.com/2015/06/18/circles-universal-basic-income/>. [cit. 2025-10-03].
- [49] KÖPPELMANN, M.; BOES, P. and ERNST, F. *Circles – money for a multipolar world* online. whitepaper, Version 2.2.1. Available at: <https://whitepaper.aboutcircles.com/>. [cit. 2025-10-03].
- [50] LAVIN, R.; LIU, X.; MOHANTY, H.; NORMAN, L.; ZAAROUR, G. et al. *A Survey on the Applications of Zero-Knowledge Proofs* online. arXiv, 2024. Available at: <https://doi.org/10.48550/arXiv.2408.00243>. [cit. 2026-01-21].
- [51] LIANG, J.; HU, D.; WU, P.; YANG, Y.; SHEN, Q. et al. *SoK: Understanding zk – SNARKs: The Gap Between Research and Practice*. In: *34th USENIX Security Symposium (USENIX Security 25)* online. Seattle, WA, USA: [b.n.], August 2025, p. 2085–2104. ISBN 978-1-939133-52-6. Available at: <https://www.usenix.org/conference/usenixsecurity25/presentation/liang-sok>. [cit. 2026-02-20].
- [52] LIU, J. K.; WEI, V. K. and WONG, D. S. Linkable Spontaneous Anonymous Group Signature for Ad Hoc Groups. In: WANG, H.; PIEPRZYK, J. and VARADHARAJAN, V., ed. *Information Security and Privacy* online. Berlin, Heidelberg: Springer Berlin

- Heidelberg, 2004, p. 325–335. ISBN 978-3-540-27800-9. Available at: https://doi.org/10.1007/978-3-540-27800-9_28. [cit. 2026-01-19].
- [53] MARAM, D.; MALVAI, H.; ZHANG, F.; JEAN LOUIS, N.; FROLOV, A. et al. CanDID: Can-Do Decentralized Identity with Legacy Compatibility, Sybil-Resistance, and Accountability. In: *2021 IEEE Symposium on Security and Privacy (SP)* online. IEEE, May 2021, p. 1348–1366. ISBN 978-1-7281-8934-5. Available at: <https://doi.org/10.1109/SP40001.2021.00038>. [cit. 2025-11-02].
- [54] MARIANI, J. *Privacy-Preserving Consensus Protocol based on Social Capital*. Brno, CZ, 2025. Dissertation. Brno University of Technology, Faculty of information technology. Supervisor HOMOLIAK, I.
- [55] MAZZOCCA, C.; ACAR, A.; ULUAGAC, S.; MONTANARI, R.; BELLAVISTA, P. et al. A Survey on Decentralized Identifiers and Verifiable Credentials. *IEEE Communications Surveys & Tutorials* online. IEEE, february 2025, vol. 27, no. 6, p. 3641–3671. ISSN 1553-877X. Available at: <https://doi.org/10.1109/COMST.2025.3543197>. [cit. 2026-02-08].
- [56] MERKLE, R. C. Digital signature system and method based on a conventional encryption function, november 1989. U.S. Patent 4 881 264.
- [57] MÜHLE, A.; GRÜNER, A.; GAYVORONSKAYA, T. and MEINEL, C. A survey on essential components of a self-sovereign identity. *Computer Science Review* online, november 2018, vol. 30, p. 80–86. ISSN 1574-0137. Available at: <https://doi.org/10.1016/j.cosrev.2018.10.002>. [cit. 2026-02-19].
- [58] NAINWAL, A.; KAMBLE, A. and AWATHARE, N. A Comparative Analysis of zk-SNARKs and zk-STARKs: Theory and Practice. *Cryptography and Security* online. arXiv, december 2025. Available at: <https://doi.org/10.48550/arXiv.2512.10020>. [cit. 2026-02-20].
- [59] NAKAMOTO, S. *Bitcoin: A Peer-to-Peer Electronic Cash System* online. October 2008. Available at: <http://www.bitcoin.org/bitcoin.pdf>. [cit. 2026-01-16].
- [60] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. *Face Recognition Technology Evaluation (FRTE) 1:N Identification* online. U.S. Department of Commerce, march 2026. Available at: <https://pages.nist.gov/frvt/html/frvt1N.html>. [cit. 2026-04-21].
- [61] OASIS PROTOCOL FOUNDATION. *The Oasis Blockchain Platform* online. Oasis Labs, june 2020. Available at: https://assets.website-files.com/5f59478e350b91447863f593/628ba74a9aee37587419cf65_20200623%20The%20Oasis%20Blockchain%20Platform.pdf. [cit. 2026-04-16]. White paper.
- [62] OASIS PROTOCOL FOUNDATION. *Oasis Network Documentation* online. 2026. Available at: <https://docs.oasis.io>. [cit. 2026-04-16].
- [63] OHLHAVER, P.; NIKULIN, M. and BERMAN, P. *Compressed to 0: The Silent Strings of Proof of Personhood* online. SSRN, 6. march 2024. revised 2024-04-11. Available at: <https://doi.org/10.2139/ssrn.4749892>. [cit. 2025-12-02].

- [64] OHLHAVER, P.; WEYL, E. G. and BUTERIN, V. Decentralized Society: Finding Web3’s Soul. online. SSRN, may 2022, p. 37. revised 2024-02-06. Available at: <https://doi.org/10.2139/ssrn.4105763>. [cit. 2025-11-29].
- [65] OLEKŠÁK, S. *The analysis of cryptographic techniques for offloading computations and storage in blockchains*. Brno, CZ, 2024. 62 p. Dissertation. Brno University of Technology, Faculty of information technology. Supervisor PEREŠÍN, M.
- [66] ORACLE CORPORATION. Understand Inverted File Flat Vector Indexes. *Oracle AI Vector Search User’s Guide* online. April 2026. Available at: <https://docs.oracle.com/en/database/oracle/oracle-database/26/vecse/understand-inverted-file-flat-vector-indexes.html>. [cit. 2026-04-22]. Path: Home;Database;Oracle AI Database;Release 26.
- [67] ÖZSU, M. T.; VALDURIEZ, P. et al. *Principles of distributed database systems*. 3rd ed. Springer, 1999. ISBN 978-3-030-26252-5.
- [68] PLATT, M.; PLATT, D. and MCBURNEY, P. Sybil attack vulnerability trilemma. *International Journal of Parallel, Emergent and Distributed Systems* online. Taylor & Francis, june 2024, vol. 39, no. 4, p. 446–460. ISSN 1744-5760. Available at: <https://doi.org/10.1080/17445760.2024.2352740>. [cit. 2025-12-19].
- [69] POLGAR, S. *The Next Generation of Anonymous Credentials and Zero-Knowledge Proofs of Knowledge for Private Digital Identity* online. 2025. 103 p. Dissertation. The University of Sydney, Faculty of Engineering, School of Computer Science. Available at: <https://hdl.handle.net/2123/34195>. [cit. 2025-11-02].
- [70] POLYGON ZERO TEAM. Plonky2: Fast Recursive Arguments with PLONK and FRI. online, september 2022, p. 16. Available at: <https://docs.rs/crate/plonky2/latest/source/plonky2.pdf>. [cit. 2026-05-11].
- [71] POROBOV, P. *Upala* online. 6. march 2019. revised 2022-10-18. Available at: <https://docs.upala.id/en/latest/>. [cit. 2025-12-02].
- [72] RAJASEKARAN, A. S.; AZEES, M. and AL TURJMAN, F. A comprehensive survey on blockchain technology. *Sustainable Energy Technologies and Assessments* online, 2022, vol. 52, p. 102039. ISSN 2213-1388. Available at: <https://doi.org/10.1016/j.seta.2022.102039>. [cit. 2026-01-16].
- [73] RICH, M. *Introducing Humanity DAO* online. 9. march 2019. Available at: <https://medium.com/marbleorg/introducing-humanity-90ddf9ead235>. [cit. 2025-12-06].
- [74] RIVEST, R. L.; SHAMIR, A. and TAUMAN, Y. How to Leak a Secret. In: BOYD, C., ed. *Advances in Cryptology — ASIACRYPT 2001* online. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, p. 552–565. ISBN 978-3-540-45682-7. Available at: https://doi.org/10.1007/3-540-45682-1_32. [cit. 2026-01-19].
- [75] SHAHAF, G.; SHAPIRO, E. and TALMON, N. Genuine Personal Identifiers and Mutual Sureties for Sybil-Resilient Community Growth. In: AREF, S.; BONTCHEVA, K.; BRAGHIERI, M.; DIGNUM, F.; GIANNOTTI, F. et al., ed. *Social Informatics* online. Cham: Springer International Publishing, October 2020, vol. 12467, p. 320–332. ISBN 978-3-030-60975-7. Available at: https://doi.org/10.1007/978-3-030-60975-7_24. [cit. 2025-11-29].

- [76] SHARIF, A.; RANZI, M.; CARBONE, R.; SCARRETTA, G.; MARINO, F. A. et al. The eIDAS Regulation: A Survey of Technological Trends for European Electronic Identity Schemes. *Applied Sciences* online, december 2022, vol. 12, 24, 12679, p. 28. revised 2022-12-16. ISSN 2076-3417. Available at: <https://doi.org/10.3390/app122412679>. [cit. 2025-12-21].
- [77] SHARMA, S.; SINGH, P. and KUMAR, A. Fake Profile Detection on Social Networks—A Survey. In: ILLÉS, Z.; VERMA, C.; SEQUEIRA, P. J. G. and SINGH, P. K., ed. *Proceedings of International Conference on Recent Innovations in Computing* online. Singapore: Springer Nature Singapore, 2024, p. 403–416. ISBN 978-981-97-3442-9. Available at: https://doi.org/10.1007/978-981-97-3442-9_28. [cit. 2025-12-21].
- [78] SHEYBANI, N.; AHMED, A.; KINSY, M. and KOUSHANFAR, F. Zero-Knowledge Proof Frameworks: A Systematic Survey. *Cryptography and Security* online. arXiv, february 2025. revised 2025-04-27. Available at: <https://doi.org/10.48550/arXiv.2502.07063>. [cit. 2026-02-20].
- [79] SHI, L.; YU, S.; LOU, W. and HOU, Y. T. SybilShield: An agent-aided social network-based Sybil defense among multiple communities. In: *2013 Proceedings IEEE INFOCOM* online. IEEE, July 2013, p. 1034–1042. ISSN 0743-166X. Available at: <https://doi.org/10.1109/INFCOM.2013.6566893>. [cit. 2026-01-01].
- [80] SIDDARTH, D.; IVLIEV, S.; SIRI, S. and BERMAN, P. Who Watches the Watchmen? A Review of Subjective Approaches for Sybil-Resistance in Proof of Personhood Protocols. *Frontiers in Blockchain* online. Frontiers, november 2020, vol. 3, p. 16. ISSN 2624-7852. Available at: <https://doi.org/10.3389/fbloc.2020.590171>. [cit. 2025-11-02].
- [81] SONG, R. *LinkDID: A Privacy-Preserving, Sybil-Resistant and Key-Recoverable Decentralized Identity Scheme* online. version v2. arXiv, 2024. revised 2024-01-24. Available at: <https://doi.org/10.48550/arXiv.2307.14679>. [cit. 2025-11-16].
- [82] SPORNY, M.; LONGLEY, D.; CHADWICK, D. and HERMAN, I. *Verifiable Credentials Data Model v2.0* online. World Wide Web Consortium, may 2025. Available at: <https://www.w3.org/TR/vc-data-model/>. [cit. 2026-01-16].
- [83] SPORNY, M.; LONGLEY, D.; SABADELLO, M.; REED, D.; STEELE, O. et al. *Decentralized Identifiers (DIDs) v1.0* online. World Wide Web Consortium, july 2022. Available at: <https://www.w3.org/TR/did-1.0/>. [cit. 2026-01-16].
- [84] SZABO, N. Smart contracts: building blocks for digital markets. *EXTROPY: The Journal of Transhumanist Thought*, (16), 1996, vol. 18, no. 2, p. 28.
- [85] TRAN, N.; LI, J.; SUBRAMANIAN, L. and CHOW, S. S. Optimal Sybil-resilient node admission control. In: *2011 Proceedings IEEE INFOCOM* online. IEEE, June 2011, p. 3218–3226. ISBN 978-1-4244-9921-2. Available at: <https://doi.org/10.1109/INFCOM.2011.5935171>. [cit. 2026-01-01].
- [86] UN SYSTEM CHIEF EXECUTIVES BOARD FOR COORDINATION. *UN Digital ID* online. Available at: <https://unsceb.org/topics/un-digital-id>. [cit. 2025-12-21].

- [87] UNIQUE IDENTIFICATION AUTHORITY OF INDIA. *Aadhaar* online. revised 2025-12-22. Available at: <https://uidai.gov.in/en>. [cit. 2025-12-22].
- [88] VALIANT, P. Incrementally Verifiable Computation or Proofs of Knowledge Imply Time/Space Efficiency. In: CANETTI, R., ed. *Theory of Cryptography* online. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, p. 1–18. ISBN 978-3-540-78524-8. Available at: https://doi.org/10.1007/978-3-540-78524-8_1. [cit. 2026-04-22].
- [89] WANG, S. *What Is Entity Resolution: Techniques, Tools & Use Cases* online. 19. september 2025. Available at: <https://www.puppygraph.com/blog/entity-resolution>. [cit. 2025-12-21].
- [90] WEI, W.; XU, F.; TAN, C. C. and LI, Q. SybilDefender: A Defense Mechanism for Sybil Attacks in Large Social Networks. *IEEE Transactions on Parallel and Distributed Systems* online. IEEE, january 2013, vol. 24, no. 12, p. 2492–2502. ISSN 1558-2183. Available at: <https://doi.org/10.1109/TPDS.2013.9>. [cit. 2026-01-01].
- [91] WORLD. *World whitepaper* online. March 2026. Available at: <https://whitepaper.world.org/>. [cit. 2026-04-03].
- [92] YAGA, D.; MELL, P.; ROBY, N. and SCARFONE, K. Blockchain Technology Overview. *CoRR* online. arXiv, 2019, abs/1906.11078. Available at: <https://doi.org/10.6028/NIST.IR.8202>. [cit. 2026-01-16].
- [93] YU, H.; GIBBONS, P. B.; KAMINSKY, M. and XIAO, F. SybilLimit: A Near-Optimal Social Network Defense against Sybil Attacks. In: *2008 IEEE Symposium on Security and Privacy (sp 2008)* online. IEEE, May 2008, p. 3–17. ISBN 978-0-7695-3168-7. Available at: <https://doi.org/10.1109/SP.2008.13>. [cit. 2026-01-01].
- [94] YU, H.; KAMINSKY, M.; GIBBONS, P. B. and FLAXMAN, A. D. SybilGuard: Defending Against Sybil Attacks via Social Networks. *IEEE/ACM Transactions on Networking* online. IEEE, june 2008, vol. 16, no. 3, p. 576–589. ISSN 1558-2566. Available at: <https://doi.org/10.1109/TNET.2008.923723>. [cit. 2026-01-01].
- [95] ZCASH. *The halo2 Book* online. 2024. Available at: <https://zcash.github.io/halo2>. [cit. 2026-02-20]. Official technical documentation for the halo2 protocol.
- [96] ZHAO, C.; ZHAO, S.; ZHAO, M.; CHEN, Z.; GAO, C.-Z. et al. Secure Multi-Party Computation: Theory, practice and applications. *Information Sciences* online, 2019, vol. 476, p. 357–372. ISSN 0020-0255. Available at: <https://doi.org/10.1016/j.ins.2018.10.024>. [cit. 2026-01-19].